Universidad de Oviedo

ESCUELA POLITÉCNICA DE INGENIERÍA DE GIJÓN

GRADO EN
INGENIERÍAINFORMÁTICA EN TECNOLOGÍAS DE LA INFORMACIÓN

GESTIÓN DEL TRANSPORTE DE USUARIOS DE UN CENTRO DE DÍA DE PERSONAS MAYORES

DÑA. RODRÍGUEZ BARES, SILVIA
TUTOR:
D. MUÑIZ SÁNCHEZ, RUBÉN

FECHA: Julio 2024

Contents

List of Figures

List of Tables

1. Report

1.1.- Objectives and scope

1.1.1.- Introduction

The main objective of this Bachelor's thesis is to design and deploy an automated transportation management system that enhances the quality and efficiency of the service provided to elderly users of a day center. The project is inspired by the challenges faced by AFADLA, an organization focused on addressing Alzheimer's and other ailments in Villablino, where manual registration of center users and planning of daily pickup and return routes from their homes have been identified as major pain points. To address these issues, the proposed system will incorporate advanced optimization algorithms that enable automated route planning, taking into account various constraints such as space availability , and user-specific needs.

1.1.2.- Scope

The scope of this Bachelor's thesis is to develop a transportation management system for a day center for elderly people, which includes the creation of an Android mobile application and a web-based platform. The system will allow the organization to efficiently register center users and schedule their pickups and returns. Additionally, the system will calculate the a route for transportation, taking into account various constraints, such as space limitations and user-specific needs, such as users who cannot sit together. The system will be designed to provide a user-friendly and intuitive interface, which will enhance the overall quality and efficiency of the transportation service.

Figure 1.1: Representation of picking up a center user.

In addition, the proposed transportation management system must ensure the security and privacy of the data handled by the service. To achieve this goal, the system will implement robust data encryption and authentication mechanisms, in compliance with relevant

regulatory requirements. The system will also feature role-based user profiles, which will allow users to interact with the system based on their specific needs and privileges.

1.2.- Previous studies and analysis

1.2.1.- Current situation

Based on the preliminary research, the current process for managing transportation services for the elderly population in AFADLA involves manual registration and data entry using paper forms and Excel spreadsheets. The administrative staff is responsible for collecting user data and manually matching them to predetermined transportation routes, often with the assistance of a driver and/or caregiver. The decision-making process for route assignment is based on the driver's familiarity with the area and the caregiver's observations of the user's behavior (such as aggression, dizziness, or other special needs). The routes are not recorded or documented, leaving the information solely in the driver's memory, and in some cases, communicated via text message when dealing with a new user.

It is important to note that users are typically picked up in a fixed order, without consideration for the most optimal route or any special accommodations required for specific user characteristics, such as medical conditions or behavioral tendencies. AFADLA operates throughout the Laciana area, with its facilities located at one end of the valley in the village of Villaseca.

The current process lacks a systematic approach to route planning, relying heavily on the driver's memory and occasional text messages to manage new users.

A map of the Laciana valley can be used to illustrate the area covered by these transportation services, highlighting the geographic challenges faced by the administrative staff in coordinating effective and timely transportation for the elderly population. The red points on the map represent the habitation areas. This situation underscores the need for an improved system that can ensure more efficient and user-centric transportation management.

Figure 1.2: Cartographic map of the Laciana Valley

Another significant aspect of the current transportation management process in Villablino is that the notification of a user's absence is communicated through a phone call or text message to the director of the center. The director then informs the rest of the staff through a phone call or message.

If the driver arrives at the user's pick-up location and the user is not present, they usually wait for a reasonable amount of time (5 minutes) before calling the family member in charge of the elderly person. If this is unsuccessful, the driver will try to contact the user through the intercom system. If they are unable to contact anyone through these means, they will move on to the next stop, informing the center director of the user's absence. The director must then try to contact the user's family or caregiver to determine their whereabouts.

It is also important to emphasize that the area where the center is located is a mountainous region where there are several road accidents caused by wild animals, and it can become impassable due to meteorological factors such as heavy snow or frost. Currently, the closure of the center is communicated through a phone call, as well as any breakdowns that occur with the transportation system used (a van).

This process can be time-consuming, error-prone, and lacks the ability to track or analyze user data to make informed decisions regarding service improvements. Therefore, there is a need for a more efficient, streamlined, and secure transportation management system that can automate the registration process, manage user data, and generate optimized transportation routes based on various factors such as time, space, and user-specific needs.

In the lower image, you can see a summary of the sequence of actions that would be carried out when a new user enters the day center. The actions are presented in summary form as they have already been explained in previous paragraphs with a higher level of detail.

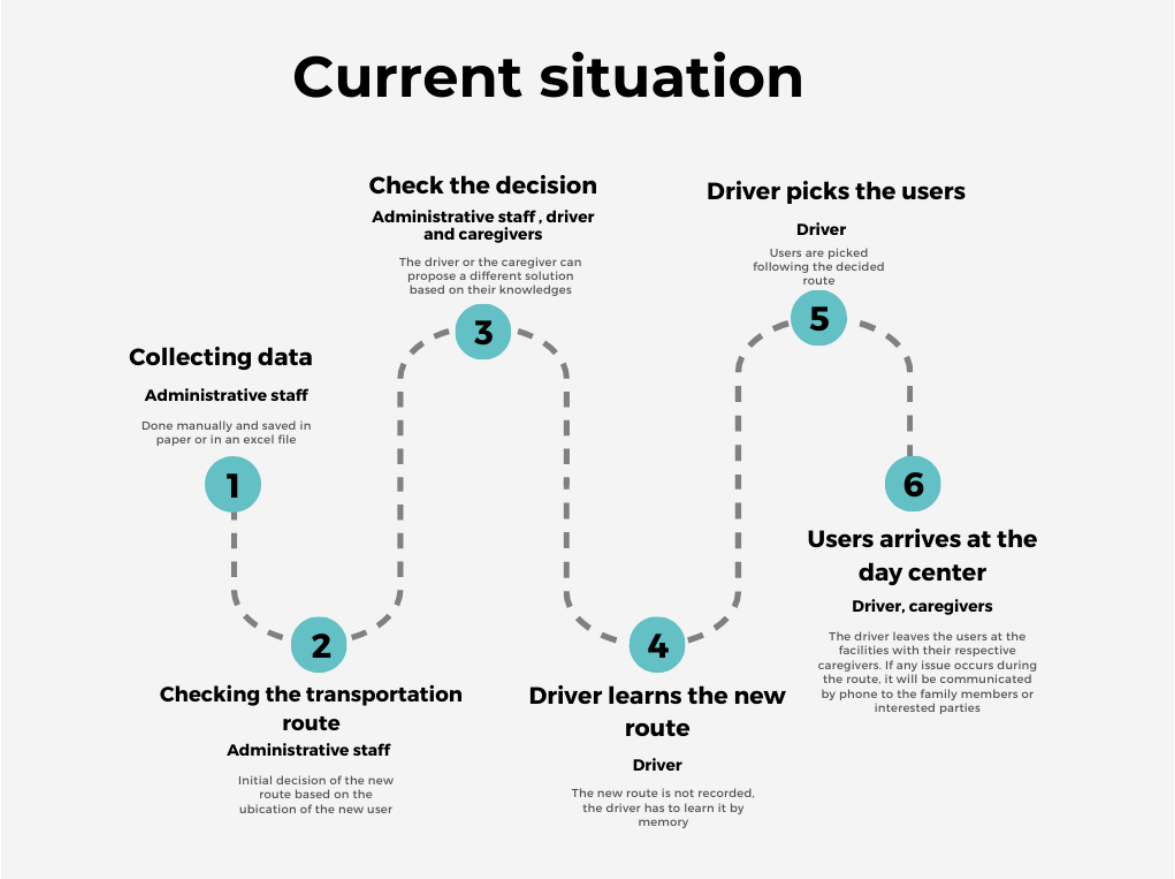

Figure 1.3: Current situation for adding a new user process flow

1.2.2.- Project Analysis

The objective of this project is to design and deploy an automated transportation management system that enhances the quality and efficiency of the service provided to elderly users of a day center.

Before analyzing the more technical aspects of the project, we should assess the potential stakeholders in order to design a solution tailored to their needs. To do this, we must consider the rural environment of Villablino, located within the Laciana Valley (where the AFADLA association operates). In this aging population area, there are several day care centers and nursing homes, as well as multiple associations related to various degenerative diseases.

It can be asserted that the interest of a significant portion of the local population in these types of services is quite high. The identified stakeholders are the following ones:

In this particular case, it is important to consider the following information when formulating the best proposal and meeting all the requirements:

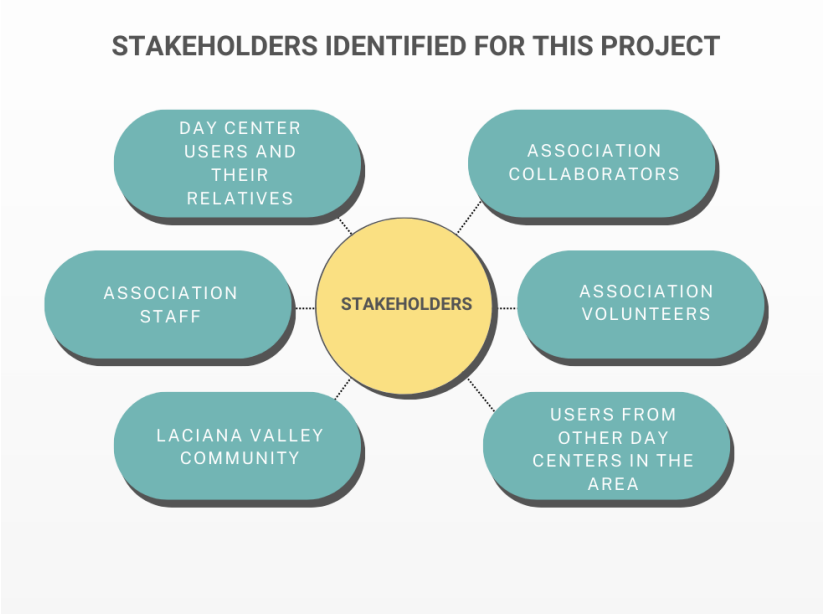

Figure 1.4: Stakeholders identified for this project

- All the information will be stored in a common database, making it easier for its update, maintenance, and protection.
- Multiple profiles will be created, each with different functions. Each profile will be able to view the corresponding information without having access to what they should not. Four different profiles have been identified for this project:
  - Administrator access rights:
    - Full visibility of user data and workers' data.
    - Add, delete, and update user and worker details.
    - View available routes.
    - Create new routes
    - Create new buses distributions
  - Staff access rights :
    - Full visibility of user data and workers' data.
    - View user data.
    - View available routes.
    - Cannot edit or delete user data.
  - Driver access rights:
    - No access to personal data of users.
    - View routes.

    - View map of the route.
    - Set the user to state picked or not and add observations (being this last field optional)
    - view all buses with its seats arrangements of the buses
    - Reset route if wanted
  - Daycare center user access rights :
    - View and edit their own profile.
    - Access a history log with various pick-up or drop-off dates.
    - Cannot access or modify data of other users.

1.2.3.- Study of alternatives

In order to design an effective automated transportation management system, it is essential to evaluate various potential solutions. This subsection explores different approaches and technologies that could be implemented to address the challenges identified in the current manual system used by AFADLA. The alternatives will be assessed based on their feasibility, cost, ease of implementation, and potential impact on improving service efficiency and user satisfaction.

1.2.3.1.- Commercial Transportation Management Systems

One option is to adopt a commercial Transportation Management System (TMS). These systems, used widely in logistics, can be adapted for managing transportation for elderly users. Key features include automated route planning, real-time tracking, and data management. Notable TMS providers include SAP Transportation Management, Oracle Transportation Management, and C.H. Robinson TMS.

SAP Transportation Management[10] SAP TM is a comprehensive solution designed to streamline logistics and improve transportation efficiency with features such as:

- **Automated Route Planning:** Generates efficient routes based on distance, traffic, and user needs, significantly reducing manual planning time and effort.
- **Real-Time Tracking:** Allows administrative staff to monitor vehicle locations and quickly respond to delays or emergencies, enhancing user safety.
- **Integration with SAP Modules:** Enables seamless data management, ensuring all user information is up-to-date and reducing manual data entry errors.

- **Automated Notifications:** Notifies relevant staff and caregivers via SMS or email in case of schedule changes, reducing the need for manual communication.
- **Handling Complex Logistics:** Coordinates multiple vehicles and dynamically adjusts routes based on real-time information, useful in areas with rapidly changing road conditions.

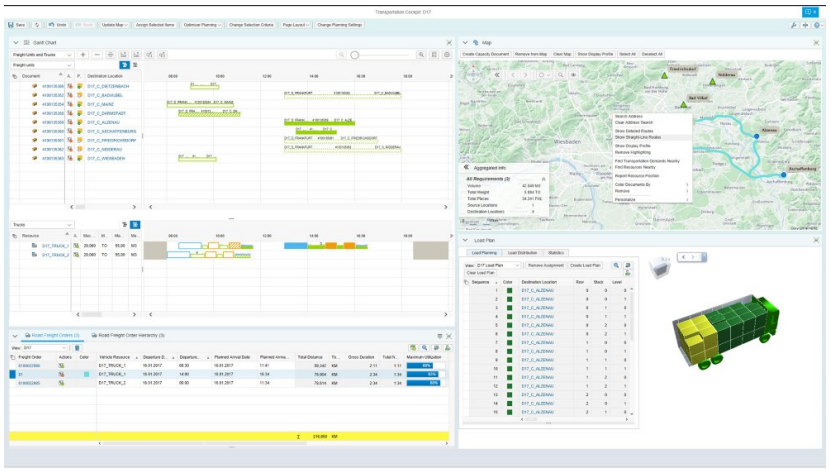

Figure 1.5: Example of a TMS implemented with SAP

However, there are significant drawbacks. AFADLA lacks the technical profile to maintain the SAP TM system and does not have the budget to support such a comprehensive solution, making it non-viable given the current low number of users.

Pros	Cons
High integration capability with other SAP products	High initial cost and ongoing maintenance expenses
Robust and reliable platform	Requires significant customization for specific needs
Comprehensive support and regular updates	Complex implementation process
Automated route planning and real-time tracking	AFADLA lacks a technical profile to maintain the product
Centralized data management	Budget constraints due to the low number of users
Automated notifications	Not viable for AFADLA's current scale

For more information about SAP Transportation Management, refer to the following link: https://www.sap.com/spain/products/scm/transportation-logistics

Oracle Transportation Management [9] Oracle Transportation Management (OTM) provides a flexible and scalable solution for managing transportation needs. It includes features such as route planning, freight payment, and logistics network modeling. Oracle Transportation Management Cloud could enhance AFADLA's transportation services with features like:

- **Automated Route Planning:** Optimizes routes based on distance, traffic, and user needs, reducing manual planning effort.
- **Real-Time Tracking:** Allows staff to monitor vehicle locations and quickly adjust plans in case of delays or emergencies.
- **Integration with Oracle Products:** Ensures seamless data management, keeping user information up-to-date and easily accessible.
- **Automated Notifications:** Notifies staff and caregivers via SMS or email about schedule changes, reducing manual communication.
- **Analytical and Reporting Capabilities:** Helps analyze transportation data for informed decision-making.

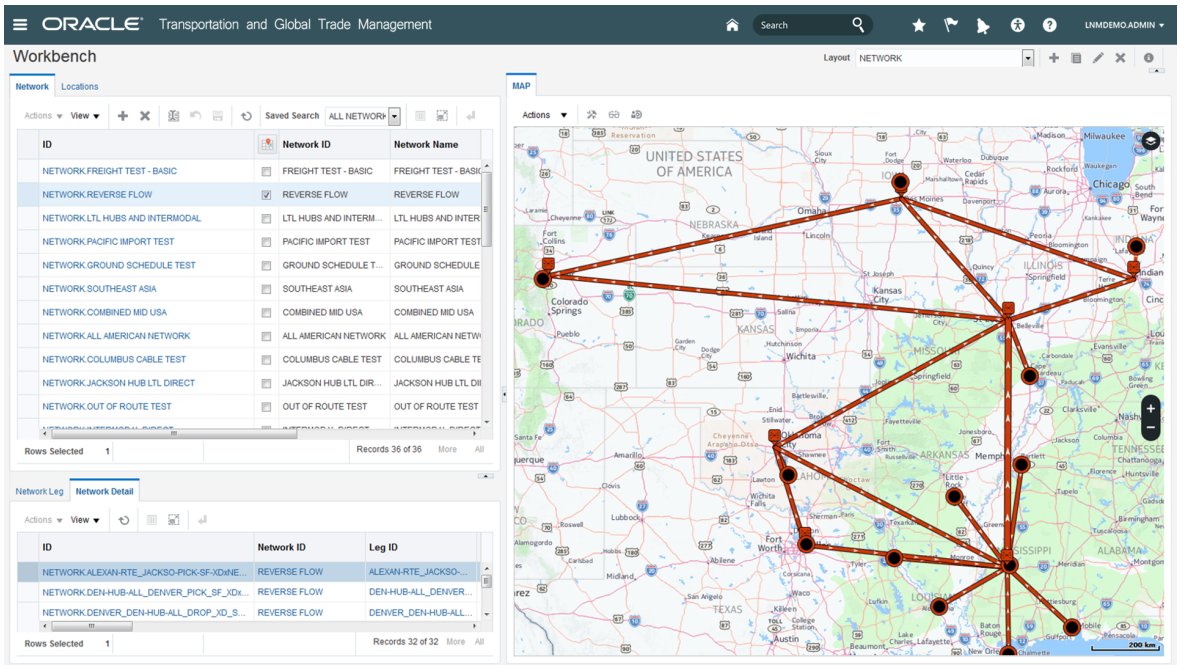

Figure 1.6: Example of a TMS implemented with Oracle transportation management cloud

However, similar to SAP TM, Oracle TM Cloud has challenges such as high implementation and maintenance costs, a steep learning curve, and budget constraints, making it potentially unviable for AFADLA's current scale.

Pros	Cons
Scalable and flexible architecture	High cost of implementation and maintenance
Strong analytical and reporting capabilities	Steep learning curve for users
Good integration with other Oracle products	Customization may be required for specific requirements
Automated route planning and real-time tracking	AFADLA lacks a technical profile to maintain the product
Centralized data management	Budget constraints due to the low number of users
Automated notifications	Not viable for AFADLA's current scale

For more information about Oracle Transportation Management Cloud, refer to the following link: https://www.oracle.com/webfolder/s/quicktours/scm/gqt-scm-log-overview/

C.H. Robinson TMS [5] C.H. Robinson offers a transportation management system, known as Navisphere, that focuses on optimizing supply chain logistics. It features automated route planning, real-time visibility, and extensive carrier network integration.

Navisphere, by C.H. Robinson, could enhance AFADLA's transportation services with features like:

- **Automated Route Planning:** Optimizes routes using a vast network of carriers, reducing manual planning effort.
- **Real-Time Visibility:** Allows staff to monitor vehicle locations and make quick adjustments in case of delays or emergencies.
- **User-Friendly Interface:** Facilitates ease of use for administrative staff, reducing the learning curve.

Implementation steps would include evaluating current processes, customizing Navisphere to fit specific needs, transferring existing data, training staff, and deploying the system.

Figure 1.7: Example of a TMS implemented with C.H. Robinson transportation management system AKA Navisphere

However, potential limitations include high costs and limited customization options, which may not fully meet AFADLA's specific requirements.

Pros	Cons
Extensive network of carriers and logistics partners	Potentially high costs depending on scale
Real-time visibility and tracking	Limited customization options compared to fully custom solutions
User-friendly interface and ease of use	May require integration with existing systems (not an issue for AFADLA)
Automated route planning	

For more information about C.H. Transportation (Navisphere), refer to the following link: https://www.chrobinson.com/es-es/technology/shipper-technology/navisphere/

1.2.3.2.- Hybrid Solution through Consultancy

Given AFADLA's unique challenges, a hybrid solution combining commercial TMS platforms with customized consultancy features is recommended. This approach leverages the strengths of commercial TMS while addressing specific needs through tailored customization and ongoing support. The consultancy would begin with an initial assessment to evaluate

AFADLA's transportation processes, identify pain points, and define specific requirements. Based on this assessment, the consultancy would recommend the most suitable commercial TMS platform and customize it to address specific needs such as route optimization, real-time tracking, automated notifications, and data management. The customized TMS would then be integrated with AFADLA's existing data sources and processes. Comprehensive training and ongoing support would be provided to ensure system proficiency and smooth operation. There are limitations to this approach, including the lack of local consultancy firms with the required expertise in Villablino, which means AFADLA would need to seek consultancy services from outside the locality. Additionally, AFADLA's inexperience in subcontracting consultancy projects and managing remote work could complicate the implementation and ongoing management of the hybrid solution.

Pros	Cons
Combines strengths of commercial TMS with custom features	Potentially higher initial costs due to customization
Tailored to specific needs of AFADLA	Dependence on consultancy for ongoing support
Ongoing support and maintenance from consultancy	Complexity in managing a hybrid system
Flexible and scalable solution	Lack of local consultancy firms
	No experience subcontracting consultancy projects, No experience with remote work

Table 1.4: Pros and Cons of Hybrid Solution through Consultancy

1.2.3.3.- Small Local Team Development

An alternative approach for AFADLA is to hire a small local team to develop a tailored transportation management system. This team would consist of individuals from the area who have a close relationship with the association, offering lower costs and fostering a relationship of trust. While this approach may take more time, it is not a major concern for AFADLA.

To implement this solution, AFADLA would identify and hire local talent with the necessary skills in software development and transportation management. The small team would work closely with the association to develop a system that meets their specific needs, ensuring a high level of customization and flexibility.

The development process would involve the following steps:

- **Team Formation:** Identify and hire local professionals with expertise in software development, transportation logistics, and project management. This team should have a close relationship with AFADLA to ensure a collaborative and cost-effective approach.
- **Needs Assessment:** Conduct a thorough assessment of AFADLA's current transportation processes and identify specific requirements and pain points.
- **System Design:** Develop a detailed design for the transportation management system, including key features such as route optimization, real-time tracking, automated notifications, and data management.
- **Development and Testing:** The local team would develop the system incrementally, testing each component to ensure it meets AFADLA's requirements and is free of bugs.
- **Training and Support:** Provide comprehensive training for AFADLA's staff to ensure they are proficient in using the new system. The local team would also offer ongoing support and maintenance.
- **Pilot Testing:** Conduct a pilot test to validate the system, identify any potential issues, and make necessary adjustments.
- **Full Implementation:** After successful pilot testing, the system would be fully implemented across all transportation operations.

This approach offers several advantages, including lower costs due to the local team's close relationship with AFADLA and a high level of customization to meet specific needs. Additionally, the team being local ensures better communication and collaboration.

However, there are some limitations to consider. The development process might take longer compared to using a commercial TMS platform. Additionally, finding local talent with the necessary skills could be challenging, and the local team might need additional training and resources.

Pros	Cons
Lower costs due to local team's relationship with AFADLA	Longer development time
High level of customization	Challenge in finding local talent with necessary skills

Pros	Cons
Better communication and collaboration	Local team might need additional training and resources
Ongoing support and maintenance from local team	

Table 1.5: Pros and Cons of Small Local Team Development

1.2.4.- Chosen option- Small Local Team Development

Given AFADLA's specific needs and constraints, the chosen approach is to hire a small local team to develop a tailored transportation management system. This subsection details the various technical decisions that need to be made within this approach, including the selection of databases, data storage solutions, programming languages for both back-end and front-end development, development methodology, and code repository management.

1.2.4.1.- Database Type: Relational vs Non-Relational

In creating a web service for managing the pickup of users from a senior center, it is crucial to select the appropriate type of database. The options include relational and non-relational databases, each with its own advantages and disadvantages.

Relational Database Relational databases use a structured schema and tables to organize data. These are ideal for highly structured data and complex transactions. Ex: MySQL, PostgreSQL, Oracle...

Advantages: High data consistency and integrity are key benefits, along with support for complex transactions and multiple join operations. Additionally, there is wide support and a strong community available.

Disadvantages: There is less flexibility for handling unstructured or dynamic data, and the development and maintenance time tend to be longer. Furthermore, scalability is limited compared to non-relational databases.

Non-Relational Database Non-relational databases, also known as No-SQL, are more flexible and can handle unstructured and semi-structured data. Ex: MongoDB, DynamoDB, Elasticsearch...

Advantages: Non-relational databases handle unstructured data without needing to define a schema beforehand. They are faster and cheaper to create and maintain, improving performance and efficiency. Additionally, they offer better scalability and flexibility to adapt to changing requirements.

Disadvantages: They may exhibit lower consistency in some cases compared to relational databases and have limitations in executing complex queries and transactions. In some instances, they also consume higher storage space.

Choosing a Non-Relational Database For this project, a non-relational database is the best option for several reasons:

- **Handling Unstructured Data:** It can store any type of data without needing to define a schema beforehand, which is useful for handling complex and varied information such as user profiles, preferences, locations, schedules, etc.
- **Speed and Cost:** It is faster and cheaper to create and maintain, as it does not require complex queries or joins to access or manipulate data, improving the performance and efficiency of the app and web service.
- **Scalability and Flexibility:** It can easily accommodate large amounts of data and adapt to changing requirements, ensuring that the app and web service can handle growing demand and provide consistent service.

After selecting non-relational databases, various options were considered. The table below shows the studied options with their advantages and disadvantages.

Database	Type	Advantages	Disadvantages
MongoDB	Document-oriented	Schema-less and flexible Supports rich queries and indexes Scalable and reliable Widely used and supported	Not suitable for complex transactions May consume more storage space May suffer from data inconsistency

Database	Type	Advantages	Disadvantages
DynamoDB	Key-value and document-oriented	Fully managed and serverless Fast and consistent performance Highly scalable and durable Supports encryption at rest	Expensive for large datasets Limited query capabilities No support for joins or aggregations Vendor lock-in with AWS
Apache Cassandra	Wide-column	Highly scalable and distributed Optimized for write-intensive workloads High availability and fault tolerance Supports tunable consistency levels	Complex to configure and maintain Not suitable for read-intensive workloads or complex queries Lacks built-in security features
Couchbase	Document-oriented	Schema-less and flexible Supports SQL-like query language (N1QL) and indexes Provides replication, caching, and full-text search	Expensive for large datasets Not suitable for complex transactions or analytics May suffer from data inconsistency

Table 1.6: Comparison of Non-Relational Databases

DynamoDB

DynamoDB has been chosen as the best option for this project for several reasons. First, DynamoDB is a key-value and document-oriented database, making it very flexible and scalable. This is important in this project since there will be many different types of data to store, and it needs to be easy to add or modify the data structure as needed.

Second, DynamoDB is designed to work well with unstructured data, which is also important in this project since the data collected will likely not always conform to a strict schema. This allows for a more natural and dynamic way of storing and retrieving data,

which is more aligned with the requirements of this project.

Finally, DynamoDB has an excellent performance track record, especially in terms of scalability and handling large amounts of data. This is crucial in a project like this where a lot of data will be generated and stored over time, and it needs to be easily accessible for analysis and reporting purposes.

1.2.4.2.- Data Storage Solutions

Storing data securely and efficiently is vital. The options for data storage include:

- **Local Servers:** Data can be stored on local servers managed by AFADLA. This provides complete control over data but requires significant maintenance and security management.
- **Cloud Storage:** Using cloud storage solutions like AWS S3, Google Cloud Storage, or Azure Blob Storage offers high availability, scalability, and reduced maintenance efforts.

Feature	AWS S3	Azure Blob Storage	Google Cloud Storage
Scalability	Virtually unlimited	Virtually unlimited	Virtually unlimited
Security	Supports encryption at rest and in transit, IAM	Supports encryption at rest and in transit, role-based access control	Supports encryption at rest and in transit, IAM
Integration with DynamoDB	High integration with AWS DynamoDB	Limited integration with DynamoDB	Limited integration with DynamoDB
Cost (per GB)	$0.023 USD for the first 50 TB/month	$0.0184 USD for hot tier, $0.01 USD for cool tier	$0.020 USD for the first 50 TB/month

Table 1.7: Comparison of Cloud Storage Solutions

AFADLA currently does not have local servers or a suitable location to store them. They would need to re-purpose one of the rooms at the day center and purchase all the necessary equipment. Additionally, being in a high mountain area, there are frequent power outages and internet connectivity issues. Hosting the data on AWS ensures high availability and reliability. AWS was particularly attractive due to its free tier for the first year, providing a cost-effective solution.

Chosen Solution: *Cloud Storage (AWS S3)* is selected due to its reliability, scalability, and ease of management. It ensures that data is securely stored and accessible from anywhere, which is crucial for remote management and disaster recovery. Furthermore, the compatibility with DynamoDB and having the entire backend on the same platform made AWS the chosen option. This integration simplifies the architecture and enhances the performance and security of the system.

1.2.4.3.- Programming Languages for Back-End Development

The back-end of the system will handle data processing, business logic, and integration with the database. The following programming languages are considered:

- **Python:** Known for its simplicity and readability, Python has extensive libraries and frameworks like Django and Flask, which are suitable for rapid development.
- **Java:** A robust, platform-independent language that is widely used in enterprise environments. Frameworks like Spring Boot facilitate building scalable and secure applications.
- **Node.js:** Allows for building fast and scalable server-side applications using JavaScript. It is particularly good for real-time applications.

Python offers simplicity and readability with its clean syntax, reducing the learning curve for new developers and speeding up the development process. This simplicity also makes code maintenance and updates easier. Python supports rapid development and has an extensive library ecosystem and strong community support, facilitating the implementation of various functionalities. Additionally, Python integrates well with other technologies and services, ensuring a cohesive system interaction.

Chosen Language *Python* is selected for its simplicity, rapid development capabilities, and strong community support. For seat assignment algorithm development, C++ was used to gain better control over pointers and memory management, and to leverage its superior speed in operations on various matrices. Detailed explanations will follow in a subsequent subsection.

1.2.4.4.- Programming Languages for Front-End Development

The front-end of the system will provide the user interface and interact with the back-end services. The following options are considered:

- **React.js:** A JavaScript library for building user interfaces, known for its component-based architecture and performance.
- **Vue.js:** A progressive JavaScript framework that is easy to integrate and offers flexibility in building user interfaces.
- **Angular:** A robust framework for building dynamic web applications, backed by Google.

AFADLA already has a website developed in React that primarily displays information and does not integrate with any backend. The existing website can be viewed at `https://alzheimerlaciana.com/`. Leveraging React.js for the new transportation management system ensures consistency in technology and reduces the learning curve for developers already familiar with React.

React.js enhances performance with its virtual DOM, ensuring faster and more efficient updates, crucial for dynamic user interfaces. Its component-based architecture promotes modular development, making the codebase more manageable, maintainable, and reusable. React.js integrates easily with other libraries and frameworks, facilitating data flow between the front-end and back-end. The large, active community provides extensive resources, libraries, and tools, speeding up development and offering quick solutions to problems. Despite a learning curve, React.js is considered easier to learn compared to frameworks like Angular, thanks to the simplicity of JSX and its declarative nature.

Feature	React.js	Vue.js	Angular
Performance	High (virtual DOM)	High (virtual DOM)	Medium (real DOM with optimizations)
Learning Curve	Moderate	Low	High
Community Support	Very strong	Strong	Strong
Component-Based Architecture	Yes	Yes	Yes
Ease of Integration	High	High	Moderate
Existing Use in AFADLA	Yes	No	No

Table 1.8: Comparison of Front-End Frameworks

React.js is selected due to its performance, ease of integration, and strong community support. In conclusion, React.js is the most suitable choice for AFADLA's front-end development due to its performance, ease of integration, and strong community support. The fact

that AFADLA already uses React.js for their existing website further justifies this choice, ensuring consistency in technology and reducing the learning curve for the development team.

1.2.4.5.- Development Methodology

Adopting a suitable development methodology is essential for ensuring efficient project management and successful delivery. The following methodologies are considered:

- **Agile:** An iterative approach that promotes flexible responses to change and continuous improvement through iterative cycles or sprints.
- **Waterfall:** A linear and sequential approach, where each phase must be completed before moving on to the next.
- **Scrum:** A subset of Agile, focusing on delivering iterative improvements through defined roles, events, and artifacts.

In the table below, a detailed comparison of the three considered development methodologies is presented. This comparison highlights the features of Agile, Waterfall, and Scrum methodologies, focusing on their approach, flexibility, suitability for managing risks and uncertainty, and how they facilitate client feedback. This detailed comparison aims to provide a clear understanding of why Scrum is the most appropriate choice for AFADLA's project, particularly given the high level of uncertainty and the need for frequent adjustments based on client feedback.

Feature	Agile	Waterfall	Scrum
Approach	Iterative	Linear	Iterative
Flexibility	High	Low	High
Client Feedback	Continuous	At the end of the project	Continuous
Risk Management	Adaptive to changes	Fixed scope, harder to manage changes	Adaptive to changes
Suitability for Uncertainty	High	Low	High
Task Tracking Tool	Varies	Typically Gantt charts	Microsoft Project

Feature	Agile	Waterfall	Scrum
Team Roles	Flexible	Defined roles	Defined roles (Product Owner, Scrum Master, Development Team)
Focus	Delivering functional software	Completing predefined phases	Delivering functional software in sprints

Table 1.9: Comparison of Development Methodologies

Scrum is selected for its structured yet flexible approach, allowing the team to adapt to changes and continuously improve the system through regular feedback loops. Using Scrum, we can implement client feedback effectively, which is crucial as AFADLA has never undertaken a project of this nature and is uncertain about their exact needs and the factors that are most important to them. Scrum is particularly well-suited for AFADLA because it allows for frequent adjustments based on client feedback, which is essential in a project with high uncertainty. By delivering the project in iterative sprints, the team can continuously incorporate feedback from AFADLA, ensuring that the final product closely aligns with their needs and expectations. Additionally, the structured roles and events in Scrum provide a clear framework for collaboration and communication, which is crucial for the success of the project.

Microsoft Project will be used to track tasks and manage the project timeline. This tool will help in planning, scheduling, and tracking the progress of tasks, ensuring that the project stays on track and any issues are addressed promptly.

1.2.4.6.- Code Repository Management

Managing the source code effectively is crucial for collaboration, version control, and deployment. The following options are considered:

- **GitHub:** A cloud-based Git repository hosting service, offering collaboration tools and continuous integration/continuous deployment (CI/CD) capabilities.
- **GitLab:** A web-based DevOps lifecycle tool providing a Git repository manager with CI/CD pipeline features.

- **Bitbucket:** A Git repository management solution with built-in CI/CD, suitable for both small and large teams.

Chosen Solution *GitHub* is selected due to its widespread adoption, robust features, and strong integration with various development tools and services.

GitHub Enterprise will be utilized to create an organization, within which all project repositories will be housed. This strategy is particularly beneficial for AFADLA for several reasons:

Creating an organization in GitHub enhances collaboration through features like pull requests, code reviews, and discussions. It provides enterprise-grade security and compliance controls, ensuring protection against unauthorized access. GitHub integrates seamlessly with various development tools and services, maintaining a smooth workflow and continuous delivery. It scales with AFADLA's needs, offering robust performance and reliability. Centralized management of repositories simplifies project oversight, policy enforcement, and permissions management. Additionally, GitHub encourages good documentation practices and knowledge sharing, maintaining a central repository of knowledge and best practices accessible to all team members.

GitHub Enterprise can be more expensive than GitLab and Bitbucket, particularly for smaller teams. Teams unfamiliar with GitHub may experience a learning curve, but extensive documentation and community support can help. Dependency on internet connectivity could be a concern for areas with unstable access, though offline capabilities and local repositories can mitigate this issue.

While GitLab offers comprehensive DevOps tools and CI/CD capabilities, it may not integrate as seamlessly with other development tools and services as GitHub. GitLab's user interface and community support are also less extensive. Bitbucket, although suitable for teams using Atlassian products, lacks some advanced collaboration features and integration capabilities of GitHub, and its interface is less intuitive, which could challenge new users.

Feature	GitHub	GitLab	BitBucket
Cost	Higher for Enterprise	Lower	Lower
Collaboration	Excellent	Good	Good
Security	High	High	Medium
Integration	Excellent with various tools	Good, mainly with GitLab ecosystem	Good, mainly with Atlassian products

Feature	GitHub	GitLab	BitBucket
Scalability	Excellent	Good	Good
User Interface	Intuitive	Moderate	Moderate
Community Support	Extensive	Extensive	Moderate
CI/CD Capabilities	Excellent	Excellent	Good

Table 1.10: Comparison of Code Repository Management Solutions

GitHub Enterprise is a robust and comprehensive solution for managing source code. Its strong integration capabilities, security features, and ability to facilitate collaboration make it an excellent choice for our project. Despite the higher cost and potential learning curve, the benefits it offers in terms of efficiency, security, and scalability outweigh these drawbacks, making it the preferred option for our code repository management.

1.2.4.7.- Summary

In summary, the small local team development approach will leverage the following technical choices, ensuring a robust, scalable, and maintainable transportation management system tailored to the needs of AFADLA:

Aspect	Choice
Database	DynamoDB
Data Storage	AWS S3
Back-End Development Language	Python
Front-End Development Language	React.js
Development Methodology	Scrum
Code Repository Management	GitHub

Table 1.11: Summary of Technical Choices

1.3.- Temporal planning

1.3.1.- Project Planning Overview

The temporal planning phase is essential for ensuring that the project progresses smoothly from start to finish. For the development of the transportation management system, a detailed plan has been created using Microsoft Project. This tool provides an extensive timeline and resource allocation, helping to ensure that all aspects of the project are accounted for and managed efficiently.

This phase involves a collaborative effort from a team with specific roles, ensuring all elements of the project are addressed. Regular meetings with a mentor are scheduled to review progress, discuss changes, and clarify technical issues, which helps keep the project aligned with its goals.

1.3.1.1.- Project Roles and Responsibilities

The team for this project includes a variety of roles to cover all necessary aspects:

- **Project manager:** Oversees the project, ensuring that it remains on track and aligned with the overall objectives. The Project Manager is responsible for coordinating the team, managing the schedule, and facilitating communication among all team members and stakeholders.
- **Analyst:** Conducts an in-depth analysis of the current processes and explores different technological solutions. The Analyst's findings will guide the selection of the most appropriate technologies and methodologies for the project.
- **Backend developer:** Focuses on developing the server-side logic, managing databases, and creating APIs to support the application. The Backend Developer ensures that the backend infrastructure is robust and scalable.
- **Frontend developer:** Responsible for designing and implementing the user interface. This role ensures that the application is user-friendly and visually appealing.
- **UX/UI designer:** Works closely with the Frontend Developer to create an intuitive and engaging user experience. The UX/UI Designer focuses on the layout, navigation, and overall aesthetics of the application.
- **Quality assurance (QA) Specialist:** Ensures that the project meets quality standards at every stage. The QA Specialist tests the application for bugs and issues, ensuring

that it is reliable and performs well.

The Gantt chart and planning table presented below provide a comprehensive visual and numerical outline of the project's scope. These detailed charts display task durations, start and finish dates, and associated costs, offering a clear and structured view of the entire project timeline. This meticulous planning aids in visualizing the sequence of activities and their interdependencies, ensuring that every phase of the project is systematically organized. By serving as a roadmap, these tools guide the team through each developmental stage, helping to maintain focus on the project's objectives. Additionally, they facilitate efficient resource allocation and management, ensuring that all project milestones are achieved within the stipulated timeframes and budget constraints. The clarity provided by this planning approach is crucial for maintaining project momentum and achieving successful outcomes.

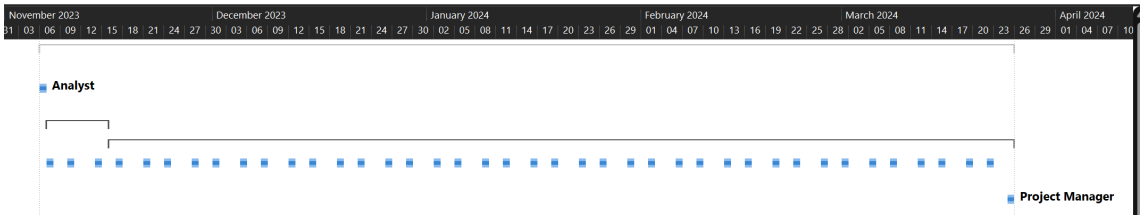

Figure 1.8: Planning: Gantt project overview

Task Name	Duration	Start	Finish	Predecessors	Resource Names	Cost
AFADLA Transportation management system	**101 days**	**Mon 06/11/23**	**Mon 25/03/24**			**€44,352.00**
1 Initial meeting with afadla directives	1 day	Mon 06/11/23	Mon 06/11/23		Analyst	€304.00
2 Project Analysis	**7 days**	**Tue 07/11/23**	**Wed 15/11/2**			**€2,128.00**
3 Development	**93 days**	**Thu 16/11/23**	**Mon 25/03/2**			**€32,080.00**
4 Bi-weekly project status meeting	**99 days**	**Tue 07/11/23**	**Fri 22/03/24**			**€9,600.00**
5 Final delivery meeting	1 day	Mon 25/03/2	Mon 25/03/2	3	Project Manager	€240.00

Figure 1.9: Planning: Project overview

1.3.2.- Project Analysis

The project analysis phase involves a comprehensive evaluation of the current transportation management practices at AFADLA. This phase includes:

- Assessing the existing manual processes.
- Identifying pain points and inefficiencies.

- Gathering requirements from stakeholders.
- Defining the project scope and objectives.
- Developing a project plan with milestones and deliverables.

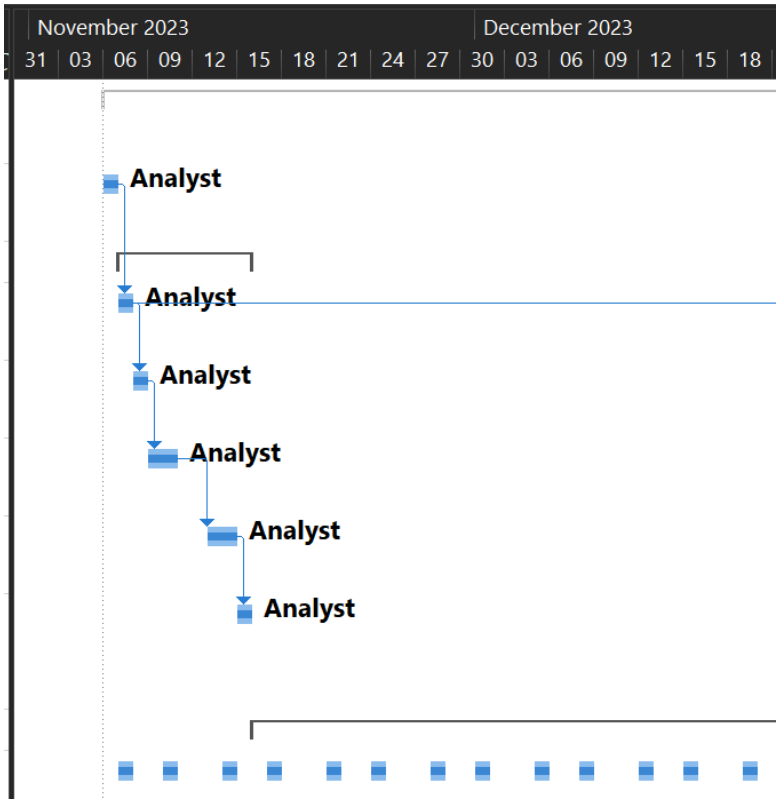

Figure 1.10: Planning: Gantt project analysis

Task Name	Duration	Start	Finish	Predecessors	Resource Names	Cost
AFADLA Transportation management system	**101 days**	**Mon 06/11/23**	**Mon 25/03/24**			**€44,352.00**
1 Initial meeting with afadla directives	1 day	Mon 06/11/23	Mon 06/11/23		Analyst	€304.00
2 Project Analysis	**7 days**	**Tue 07/11/23**	**Wed 15/11/2**			**€2,128.00**
2.1 Assessing the existing manual processes	1 day	Tue 07/11/23	Tue 07/11/23	1	Analyst	€304.00
2.2 Identifying pain points and inefficiencies	1 day	Wed 08/11/23	Wed 08/11/23	3	Analyst	€304.00
2.3 Gathering requirements from	2 days	Thu 09/11/23	Fri 10/11/23	4	Analyst	€608.00
2.4 Defining the project scope and objectives	2 days	Mon 13/11/23	Tue 14/11/23	5	Analyst	€608.00
2.5 Developing a project plan with milestones and deliverables	1 day	Wed 15/11/23	Wed 15/11/23	6	Analyst	€304.00

Figure 1.11: Planning: Analysis overview

This phase is crucial as it sets the foundation for the subsequent development phases by ensuring a clear understanding of the project's requirements and goals.

1.3.3.- Development Phase

The development phase is divided into three main phases to manage the project effectively and ensure a structured approach to building the system.

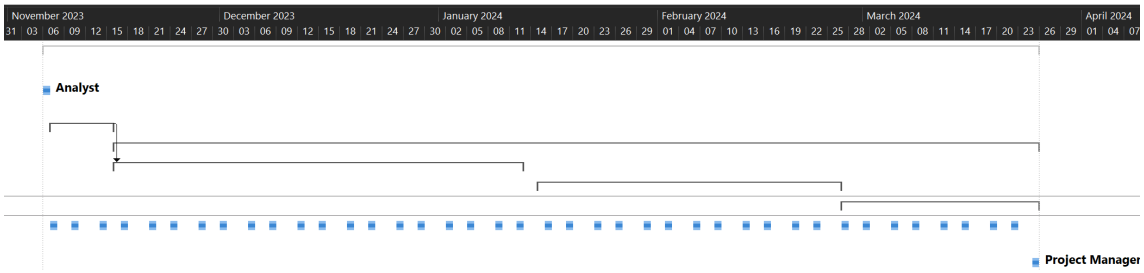

Figure 1.12: Planning: Gantt Development phase overview

Task Name	Duration	Start	Finish	Predecessors	Resource Names	Cost
AFADLA Transportation management system	**101 days**	**Mon 06/11/23**	**Mon 25/03/24**			**€44,352.00**
1 Initial meeting with afadla directives	1 day	Mon 06/11/23	Mon 06/11/23		Analyst	€304.00
2 Project Analysis	**7 days**	**Tue 07/11/23**	**Wed 15/11/2**			**€2,128.00**
3 Development	**93 days**	**Thu 16/11/23**	**Mon 25/03/2**			**€32,080.00**
3.1 Development phase 1	**42 days**	**Thu 16/11/23**	**Fri 12/01/24**	**2**		**€14,160.00**
3.2 Development phase 2	**31 days**	**Mon 15/01/2**	**Mon 26/02/2**			**€11,400.00**
3.3 Development phase 3	**20 days**	**Tue 27/02/24**	**Mon 25/03/2**			**€6,520.00**

Figure 1.13: Planning: Development phase overview

1.3.3.1.- Development Phase 1

Development Phase 1 focuses on setting up the core infrastructure and initial functionalities. This includes:

- Creating wire-frames and mock-ups for the web page UI/UX design.
- Setting up the development environment and tools.
- Designing the database schema and integrating DynamoDB.
- Implementing the initial backend services using Python.
- Developing basic frontend components with React.js.

- Establishing the CI/CD pipeline using GitHub.
- Conducting initial testing and quality assurance.

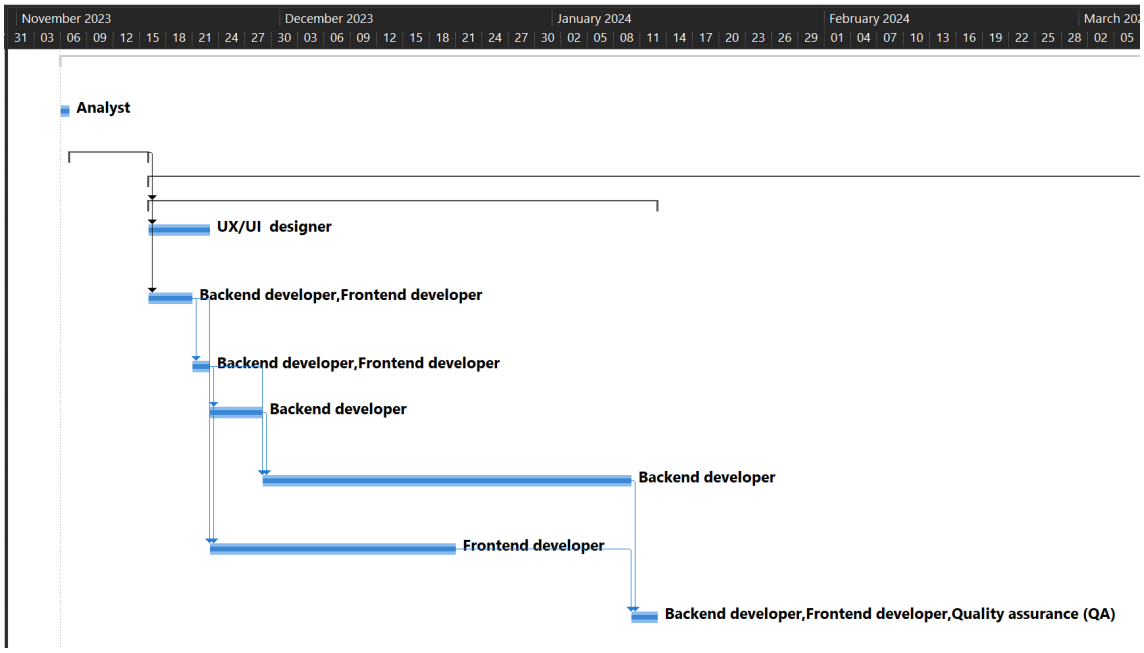

Figure 1.14: Planning: Gantt Development phase 1

3.1 Development phase 1	42 days	Thu 16/11/2	Fri 12/01/24	2		€14,160.00
3.1.1 Creating wire-frames and mock-ups for the web	5 days	Thu 16/11/23	Wed 22/11/23	2	UX/UI designer	€800.00
3.1.2 Setting up the development environment and tools	3 days	Thu 16/11/23	Mon 20/11/23	2	Backend developer Frontend	€1,200.00
3.1.3 Establishing the CI/CD pipeline using	2 days	Tue 21/11/23	Wed 22/11/23	11	Backend developer	€800.00
3.1.4 Designing the database schema and integrating DynamoDB	4 days	Thu 23/11/23	Tue 28/11/23	12	Backend developer	€800.00
3.1.5 Implementing the initial backend services using Python	30 days	Wed 29/11/23	Tue 09/01/24	13,12	Backend developer	€6,000.00
3.1.6 Developing basic frontend components with React.js	20 days	Thu 23/11/23	Wed 20/12/23	11,12	Frontend developer	€4,000.00
3.1.7 Conducting initial testing and quality	3 days	Wed 10/01/24	Fri 12/01/24	14,15	Backend developer[33%]	€560.00

Figure 1.15: Planning: Development phase 1

1.3.3.2.- Development Phase 2

Development Phase 2 builds on the foundation laid in Phase 1, adding more advanced features and enhancements. This includes:

- Adding route optimization algorithm.
- Enhancing the user interface for better user experience.
- Implementing security features and data encryption.
- Implement feedback from the stakeholders

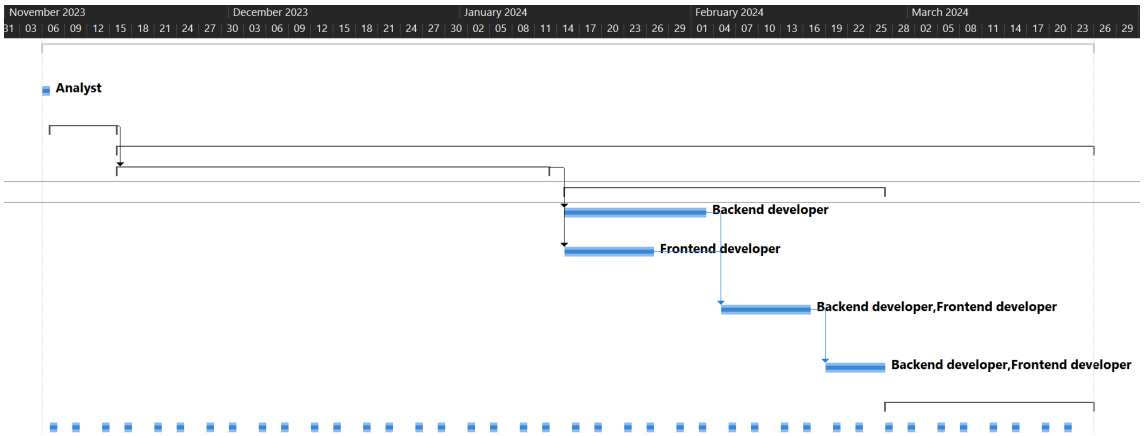

Figure 1.16: Planning: Gantt Development phase 2

3.2 Development phase 2	31 days	Mon 15/01/2	Mon 26/02/2			€11,400.00
3.2.1 Adding route optimization algorithm	15 days	Mon 15/01/24	Fri 02/02/24	9	Backend developer	€3,000.00
3.2.2 Enhancing the user interface for better user experience	10 days	Mon 15/01/24	Fri 26/01/24	9	Frontend developer	€2,000.00
3.2.3 Implementing security features and data encryption	10 days	Mon 05/02/24	Fri 16/02/24	18,19	Backend developer Frontend	€4,000.00
3.2.4 Implement feedback from the	6 days	Mon 19/02/24	Mon 26/02/24	20	Backend developer	€2,400.00

Figure 1.17: Planning: Development phase 2

1.3.3.3.- Final Phase

The final phase focuses on polishing the system, ensuring stability, and preparing for deployment. This includes:

- Conducting final system testing and user acceptance testing (UAT).
- Preparing deployment scripts and documentation.
- Training AFADLA staff on using the new system.

- Deploying the system to the production environment.
- Providing post-deployment support and maintenance.

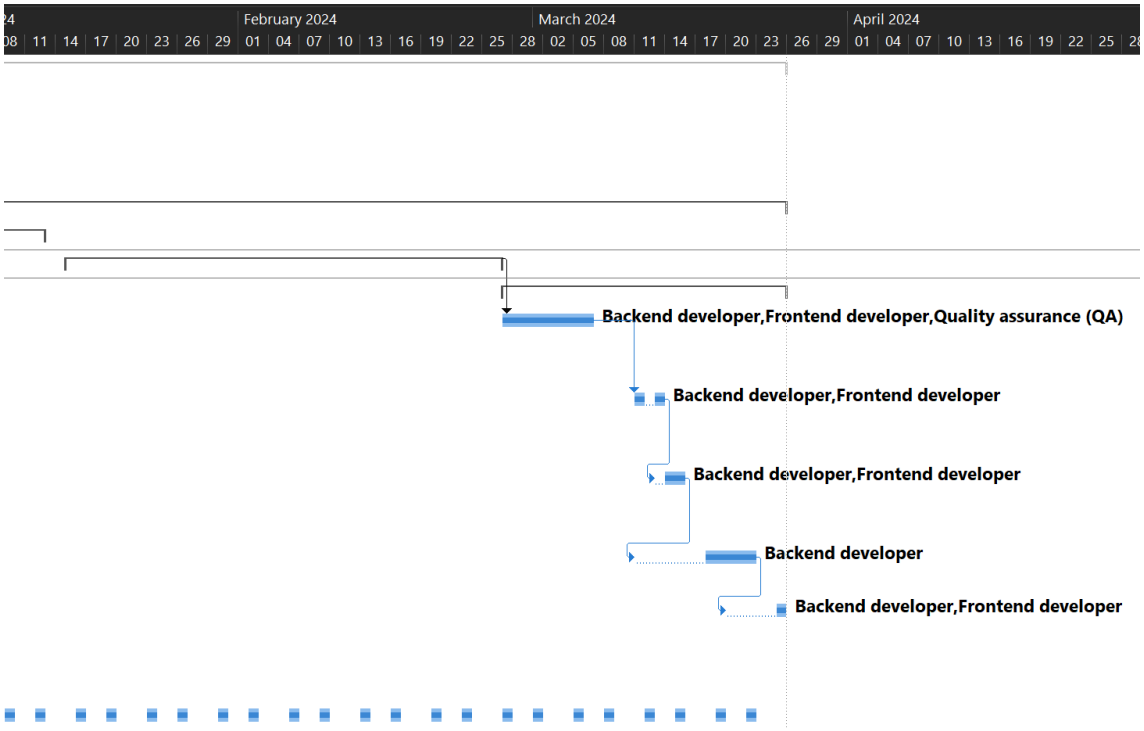

Figure 1.18: Planning: Gantt Development final phase

3.3 Development phase 3	20 days	Tue 27/02/24	Mon 25/03/2			€6,520.00
3.3.1 Conducting final system testing and user acceptance testing (UAT)	7 days	Tue 27/02/24	Wed 06/03/24	17	Backend developer Frontend	€3,920.00
3.3.2 Preparing deployment scripts and documentation	2 days	Mon 11/03/24	Wed 13/03/24	23	Backend developer Frontend	€400.00
3.3.3 Deploying the system to the production environment	2 days	Wed 13/03/24	Fri 15/03/24	24	Backend developer Frontend	€800.00
3.3.4 Training AFADLA staff on using the new	5 days	Mon 11/03/24	Fri 22/03/24	25	Backend developer	€1,000.00
3.3.5 Providing post-deployment support and maintenance	1 day	Wed 20/03/24	Mon 25/03/24	26	Backend developer Frontend developer	€400.00

Figure 1.19: Planning: Development final phase

1.3.3.4.- Management control Phase

The Management Control Phase is crucial for ensuring the project stays on track and aligns with the client's needs and expectations. This phase involves regular bi-weekly project status

meetings between the project manager, the development team, and the client. These meetings are designed to integrate feedback from the client into the various iterations of the project and to adhere to Scrum ceremonies.

During each bi-weekly meeting, the project manager reviews the progress made, discusses any challenges faced by the team, and ensures that the project is moving in the right direction. These meetings also provide an opportunity for the client to give feedback, which is then used to make necessary adjustments in subsequent iterations. This iterative process helps in refining the project and ensuring that the final product meets the client's requirements.

The final task in this phase is the delivery meeting, where the project manager presents the completed project to the client. This meeting marks the conclusion of the project, where the client reviews the final product and provides final approval.

Below are the visual representations of this phase:

4 Bi-weekly project status meeting	**99 days**	**Tue 07/11/23**	**Fri 22/03/24**			**€9,600.00**
5 Final delivery meeting	1 day	Mon 25/03/2	Mon 25/03/2	3	Project Manager	€240.00

Figure 1.20: Planning: Management tasks

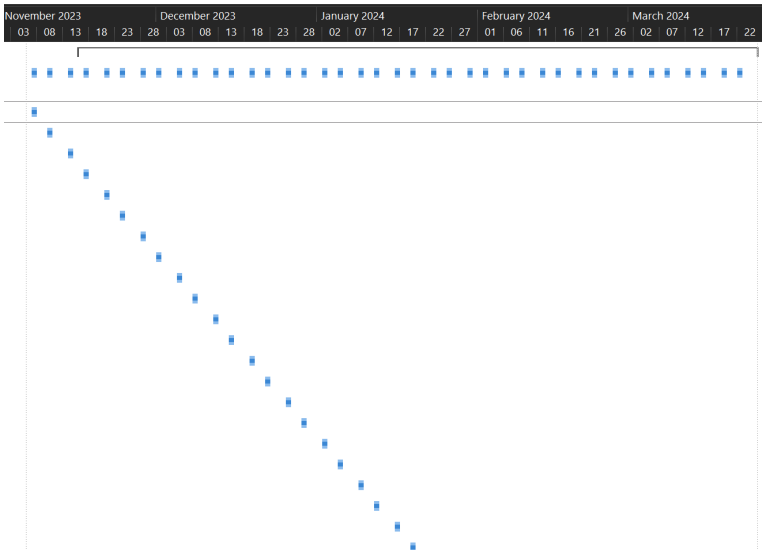

Figure 1.21: Planning: Example bi-weekly project status meeting Gantt

4 Bi-weekly project status meeting	99 days	Tue 07/11/23	Fri 22/03/24			€9,600.00
4.1 Bi-weekly project statu	1 day	Tue 07/11/23	Tue 07/11/23		Project Manager	€240.00
4.2 Bi-weekly project statu	1 day	Fri 10/11/23	Fri 10/11/23		Project Manager	€240.00
4.3 Bi-weekly project statu	1 day	Tue 14/11/23	Tue 14/11/23		Project Manager	€240.00
4.4 Bi-weekly project statu	1 day	Fri 17/11/23	Fri 17/11/23		Project Manager	€240.00
4.5 Bi-weekly project statu	1 day	Tue 21/11/23	Tue 21/11/23		Project Manager	€240.00
4.6 Bi-weekly project statu	1 day	Fri 24/11/23	Fri 24/11/23		Project Manager	€240.00
4.7 Bi-weekly project statu	1 day	Tue 28/11/23	Tue 28/11/23		Project Manager	€240.00
4.8 Bi-weekly project statu	1 day	Fri 01/12/23	Fri 01/12/23		Project Manager	€240.00
4.9 Bi-weekly project statu	1 day	Tue 05/12/23	Tue 05/12/23		Project Manager	€240.00
4.10 Bi-weekly project stat	1 day	Fri 08/12/23	Fri 08/12/23		Project Manager	€240.00
4.11 Bi-weekly project stat	1 day	Tue 12/12/23	Tue 12/12/23		Project Manager	€240.00
4.12 Bi-weekly project stat	1 day	Fri 15/12/23	Fri 15/12/23		Project Manager	€240.00
4.13 Bi-weekly project stat	1 day	Tue 19/12/23	Tue 19/12/23		Project Manager	€240.00
4.14 Bi-weekly project stat	1 day	Fri 22/12/23	Fri 22/12/23		Project Manager	€240.00

Figure 1.22: Planning: Example bi-weekly project status meeting

These figures illustrate the scheduled management tasks, the timeline for bi-weekly status meetings, and the specific dates and durations of each meeting. By adhering to this structured approach, the project manager ensures continuous communication and alignment with the client, ultimately leading to a successful project delivery.

1.4.- Future Work

As a continuation of this work, several enhancements and new features are planned:

Firstly, a notification system will be developed to allow users to see if they have new events for the day upon logging into their profile. Additionally, within each user's profile, there will be an option to upload photographs of activities they participate in at the center. These photographs can be uploaded by the staff and viewed by the user's family members or guardians, providing a more engaging and transparent experience.

In the administrator view, a feature will be implemented to create fully customized buses. Administrators will also have the ability to edit existing buses and view a comprehensive list of all buses. This will provide greater flexibility and control over bus management, ensuring that the system can adapt to various needs and scenarios.

Moreover, a targeted notification system will be developed for drivers. This system will ensure that only drivers assigned to specific routes and buses can see their assignments. This focused approach will help streamline communication and reduce confusion, allowing drivers to easily access the information relevant to their duties.

Additionally, a project will be undertaken to implement a higher level of security across the system. This will involve enhancing data protection measures and ensuring that user information is safeguarded against unauthorized access. Alongside security improvements, the graphical user interface (GUI) will be revamped to present a more professional and user-friendly design. This update will aim to improve the overall user experience, making the interface more intuitive and visually appealing.

Together, these enhancements will significantly improve the functionality, security, and user experience of the system, ensuring that it meets the evolving needs of its users.

1.5.- Conclusions

This final project has been an invaluable learning experience, bridging the gap between theoretical knowledge and practical application. Throughout this project, I have had the opportunity to delve into new technologies, such as non-relational databases like DynamoDB, and gain hands-on experience interacting with Amazon Web Services (AWS). This exposure has highlighted the importance of understanding and leveraging cloud-based services to develop scalable and efficient solutions.

Creating a custom algorithm was a crucial part of this project. This task required not only technical skill but also highlighted the importance of developing custom solutions to meet specific project needs. Additionally, teaching myself React has given me the skills to build dynamic and responsive user interfaces, which are essential for modern web development.

Understanding the associated costs of development was another crucial aspect of this project. By meticulously planning and accounting for hardware, software, and personnel expenses, I gained insights into the financial intricacies of software development. This knowledge is vital for ensuring that projects remain within budget and that resources are allocated effectively.

Moreover, this project provided a comprehensive understanding of the various roles within a development team. From backend and frontend development to quality assurance and project management, each role is integral to the project's success. This awareness fosters better collaboration and efficient task delegation within a team setting.

Project management played a pivotal role in balancing my study time, work commitments, and the development of this final project. Regular meetings with stakeholders, including the project mentor and AFADLA representatives, ensured that the project stayed on

track and that feedback was effectively incorporated. Utilizing Microsoft Project for planning and tracking emphasized the importance of accurate estimations and continuous monitoring. This approach allowed for better control of expenses and adherence to deadlines.

The knowledge applied throughout this project covers several subjects from my academic career, including Algorithmics, Web Technologies, Human-Computer Interaction (HCI), Databases, and Testing. This interdisciplinary approach was crucial for addressing the project's diverse challenges.

On a personal level, this project holds significant sentimental value. As a native of Villablino and a long-time collaborator with the AFADLA association, I was deeply invested in its success. This personal connection motivated me to go above and beyond to ensure the final product would genuinely benefit the community I care about.

In conclusion, this project has been a profound learning experience that has equipped me with a well-rounded skill set. It has enhanced my technical abilities, financial acumen, and project management skills. More importantly, it has reaffirmed my commitment to using my knowledge and skills to make a positive impact in my community. This journey has prepared me to tackle complex projects in the future with a balanced and effective approach to both development and management.

Bibliography

[1] Muhammad Qasim et al. "Test case prioritization techniques in software regression testing: An overview". In: *International Journal of advanced and applied sciences* 8.5 (2021), pp. 107–121.

[2] Python Documentation Team. *typing — Support for type hints — Python 3.10.6 documentation*. Accessed: 2024-07-10. 2023. URL: https://docs.python.org/es/3/library/typing.html.

[3] Amazon Web Services. *Amazon Elastic Block Store (EBS)*. Accessed: 2024-07-10. 2024. URL: https://aws.amazon.com/es/ebs/.

[4] Amazon Web Services. *AWS Pricing Calculator*. Accessed: 2024-07-10. 2024. URL: https://calculator.aws/#/addService.

[5] C.H. Robinson Documentation Team. *C.H. Robinson Global TMS*. Accessed: 2024-07-13. 2024. URL: https://www.chrobinson.com/es-es/technology/shipper-technology/global-tms/.

[6] npm Documentation Team. *cookies-next*. Accessed: 2024-07-10. 2024. URL: https://www.npmjs.com/package/cookies-next.

[7] npm Documentation Team. *nookies*. Accessed: 2024-07-10. 2024. URL: https://www.npmjs.com/package/nookies.

[8] OpenStreetMap Documentation Team. *Develop*. Accessed: 2024-07-13. 2024. URL: https://wiki.openstreetmap.org/wiki/Develop.

[9] Oracle Documentation Team. *Oracle Transportation Management*. Accessed: 2024-07-13. 2024. URL: https://www.oracle.com/es/scm/logistics/transportation-management/.

[10] SAP Documentation Team. *SAP Transportation Management*. Accessed: 2024-07-13. 2024. URL: https://www.sap.com/spain/products/scm/transportation-logistics.html.

[11] OSMnx Documentation Team. *OSMnx Documentation*. Accessed: 2024-07-13. 2024. URL: https://osmnx.readthedocs.io/en/stable/py-modindex.html.

[12] pnpm Documentation Team. *Motivations for pnpm*. Accessed: 2024-07-13. 2024. URL: https://pnpm.io/es/motivation.

[13] Poetry Documentation Team. *Python Poetry*. Accessed: 2024-07-13. 2024. URL: https://python-poetry.org/.

[14] PyPI Documentation Team. *bcrypt*. Accessed: 2024-07-13. 2024. URL: https://pypi.org/project/bcrypt/.

[15] TypeScript Documentation Team. *TypeScript Documentation*. Accessed: 2024-07-10. 2024. URL: https://www.typescriptlang.org/docs/.

[16] Vercel Inc. *Vercel Pricing*. Accessed: 2024-07-10. 2024. URL: https://vercel.com/pricing.

[17] Archlinux. *Archlinux home page*. URL: https://archlinux.org.

[18] Cloudflare. *Cloudflare Registrar - At-cost pricing for registration and renewal*. URL: https://www.cloudflare.com/products/registrar.

[19] GitHub. *GitHub Pricing*. URL: https://github.com/pricing.

[20] Slimbook. *Laptop PROX14*. URL: https://slimbook.com/en/shop/product/prox-14-5700u-635?category=7.

2. Budget

A well-defined budget is crucial for the successful execution and completion of the AFADLA transportation management system project. This section provides a detailed breakdown of the costs associated with each aspect of the project, ensuring transparency and efficient allocation of resources. By carefully planning and documenting the financial requirements, we aim to avoid unforeseen expenses and ensure that the project stays within the allocated budget.

The budget section is divided into four main categories: hardware, software, development and the final total cost of the project.

2.1.- Hardware

This section outlines the costs related to the physical components necessary for the project. It includes expenditures for servers, networking equipment, computers, and any other hardware required to support the development and deployment of the system. The hardware budget ensures that all necessary physical infrastructure is in place to handle the project's demands.

Item	Quantity	Unit Price	Amortization Period	Subtotal
Development PC - Slimbook PROX14 [20]	6	994.00 €	1/4	1,491.00 €
		Subtotal(without VAT)		**1,491.00 €**

Table 2.1: Budget - Hardware

2.2.- Software

In this section the expenses associated with the software tools and platforms needed for the project will be detailed. This includes costs for operating systems, development tools, database management systems, and any third-party software licenses required. The software budget is essential for acquiring the right tools to build and maintain the transportation management system efficiently.

Item	Unit	Quantity	Unit Price	Subtotal
Archlinux [17]	License fees	6	0.00 €	0.00 €
Cloudflare - Domain registration [18]	€/Year	1	10.00 €	10.00 €
Vercel - Website hosting [16]	€/Year/person	6	18.40 €	110.04 €
GitHub Enterprise [19]	€/Year/Contributor	6	231.00 €	1,386.00 €
Amazon Web Services				
Simple Storage Service				
Amazon S3 Storage [4]	Storage (GB-Mo)	28,980	0.021 €/GB	608.58 €
Amazon S3 Storage Requests [4]	Requests	24	0.0038 €/10K	0.01 €
Elastic Compute Cloud				
Amazon Elastic Compute Cloud running Linux/UNIX [4]	€/hour	8760	0.0079 €	69.21 €
EBS [4, 3]	GB-month	203.4	0.1045 €	21.26 €
Virtual Private Cloud				
Amazon Virtual Private Cloud Public IPv4 Addresses [4]	€/hour	8760	0.005 €	43,80 €
CloudWatch				
Amazon CloudWatch [4]	Storage (GB-Mo)	0	0.00 €/GB	0.00 €
Data Transfer				
Bandwidth [**aws_datatransfer**]	GB	0	0.00 €/GB	0.00 €
DynamoDB				
DynamoDB Storage [4]	Storage (GB-Mo)	0	0.00 €/GB	0.00 €
DynamoDB Read Requests [4]	Read Request Units	192.5	0.24 €/M	0.05 €
DynamoDB Write Requests [4]	Write Request Units	1	1.22 €/M	0.00 €
DynamoDB IA Read Requests [4]	IA Read Request Units	317	0.31 €/M	0.10 €
DynamoDB IA Write Requests [4]	IA Write Request Units	8	1.53 €/M	0.01 €
		Subtotal (without VAT)		**2,249.06 €**

Table 2.2: Budget - Software

2.3.- Development

The development section covers the costs related to the human resources involved in the project. This includes salaries for the project manager, analysts, developers, UX/UI designers, and quality assurance specialists. Additionally, it includes any training or professional development needed to ensure the team is well-equipped to handle the project tasks. The development budget ensures that we have the right expertise and manpower to bring the project to fruition.

Item	Unit	Nº Hours	Unit Price (€/h)	Units	Subtotal
Project Manager	Hours	328 h	30.00 €/h	1	9,840.00 €
Analyst	Hours	64 h	38.00 €/h	1	2,432.00 €
Backend developer	Hours	696 h	25.00 €/h	1	17,400.00 €
Frontend developer	Hours	504 h	25.00 €/h	1	12,600.00 €
UX/UI designer	Hours	40 h	20.00 €/h	1	800.00 €
Quality Assurance (QA)	Hours	64 h	20.00 €/h	1	1,280.00 €
			Subtotal (without VAT)		**44,352.00 €**

Table 2.3: Budget - Personnel

In the development section, it's important to highlight that the analyst has a higher cost compared to the project manager. This is because the analyst will work only for a relatively short period to conduct the initial analysis and then will exit the project. This specialized and temporary work has increased their cost to €38 per hour. On the other hand, the project manager, who usually has the highest cost in a typical project, has a lower cost of €30 per hour in this specific situation. This is because the project manager's involvement spans the entire duration of the project, thereby diluting their overall cost compared to that of the analyst.

2.4.- Total

The total section provides a comprehensive summary of all the costs incurred across the hardware, software, and development categories.

The table below provides a detailed breakdown of the project's total costs, encompassing personnel, hardware, and software expenses. Personnel costs, totaling 44,352.00 €, form the largest component of the budget, reflecting the extensive human resources required for the project. Hardware expenses amount to 1,491.00 €, covering the necessary physical infrastructure. Software costs, detailed at 2,249.06 €, include licenses and subscriptions essential for development and deployment. After accounting for general expenses and industrial profit,

the subtotal before VAT is 59,292.34 €. Including a 21% VAT of 12,451.39 €, the overall project cost amounts to 68,665.84 €.

Item	Cost
Personnel	44,352.00 €
Hardware	1,491.00 €
Software	2,249.06 €
Subtotal (without VAT)	48,092.06 €
General expenses (12%)	5,771.05 €
Industrial Profit (6%)	2,885.52 €
Total (without VAT)	56,748.63 €
VAT (21%)	11,917.21 €
Total	**68,665.84 €**

Table 2.4: Budget - Total

3. Technical documents

3.1.- User Requirements

ID	Name	Description
UR1	User Access Rights	
UR1.1	View User Profile	The daycare center user will be able to access their profile, allowing them to view and edit it.
UR1.2	Personal Information	The daycare center user will be able to view their personal information. The available information will include: • Profile photo • First name • Last name • Username • National ID (DNI) • Phone number • Address • Allergies • Disabilities • Illnesses • Medication • Other relevant details The daycare center user can edit all their personal information except the username and National ID (DNI). For these changes, they must contact their administrator.
UR1.3	Change password	The daycare center user will be able to change their personal password
UR1.4	Access History Log	The daycare center user will be able to view all pickup/drop-off events and view additional comments if desired.

ID	Name	Description
UR1.5	Filter History Log	The daycare center user will be able to filter and sort the history log alphabetically
UR1.6	Data Restriction	Daycare center users must not be able to access or modify data of other users.
UR2	Admin Access Rights	
UR2.1	View Staff List	The administrator will be able to view a list of all staff members, showing their: 1. National ID (DNI) 2. First name 3. Last name 4. Phone number 5. Address 6. Roles at the day center (there can be multiple roles, e.g., an assistant can also be a kitchen helper) 7. Username 8. Whether they have administrative privileges

ID	Name	Description
UR2.2	View Drivers List	The administrator will be able to view a list of all drivers, showing their: 1. National ID (DNI) 2. First name 3. Last name 4. Phone number 5. Address 6. Role 7. Username 8. Driver's license number
UR2.3	View Users List	The administrator will be able to view a list of all users, showing their: 1. Profile photo 2. First name 3. Last name 4. Username 5. National ID (DNI) 6. Phone number 7. Address 8. Allergies 9. Disabilities 10. Illnesses 11. Medication 12. Other relevant details

ID	Name	Description
UR2.4	View Routes List	The administrator will be able to view a list of all routes, showing the: 1. Name of the route 2. Assigned driver 3. Estimated time to complete the route 4. Total distance (round trip) 5. Description of the route
UR2.5	Create New User	The administrator will be able to create a new user, adding the following items: 1. Profile photo 2. First name 3. Last name 4. Username 5. National ID (DNI) 6. Phone number 7. Address 8. Allergies 9. Disabilities 10. Illnesses 11. Medication 12. Other relevant details 13. Password 14. Users they cannot sit with in the van

ID	Name	Description
UR2.6	Create New Staff	The administrator will be able to create a new staff member, adding the following items: 1. National ID (DNI) 2. First name 3. Last name 4. Phone number 5. Address 6. Roles at the day center (there can be multiple roles, e.g., an assistant can also be a kitchen helper) 7. Username 8. Password 9. Whether they have administrative privileges
UR2.7	Create New Driver	The administrator will be able to create a new driver, adding the following items: 1. National ID (DNI) 2. First name 3. Last name 4. Phone number 5. Address 6. Role 7. Username 8. Driver's license number 9. Password

ID	Name	Description
UR2.8	Create New Route	The administrator will be able to create a new route, adding the following items: 1. Name of the route 2. Assigned driver 3. Day center users included in the route 4. Description of the route
UR2.9	Edit Information	The administrator will be able to edit the following information: 1. Assign administrative permissions to staff members who do not have them 2. Revoke administrative permissions from staff members who currently have them 3. Change the password for any user (staff, day center users, drivers) 4. Edit any information entered when creating a day center user, staff member, or driver (except their DNI and username) 5. Delete staff members, day center users, or drivers
UR2.10	Create new bus	The administrator will be able to create a new bus distribution: 1. Name of the bus 2. Assigned driver 3. Day center users included in the bus 4. Description of the bus
UR3	Staff Access Rights	

ID	Name	Description
UR3.1	View User Data	Staff members must be able to view user data as enumerated in the "View Users List" section for administrators.
UR3.2	View Worker Data	Staff members must be able to view data of other workers as enumerated in the "View Staff List" section for administrators.
UR3.3	View Driver Data	Staff members must be able to view data of drivers as enumerated in the "View Driver List" section for administrators.
UR3.4	View Routes	Staff members must be able to view all available routes as described in the "View Routes List" section for administrators.
UR3.5	Data Restriction	Staff members must not be able to edit or delete any data.
UR4	Driver Access Rights	
UR4.1	No Personal Data Access	Drivers must not have access to the personal data of users.
UR4.2	View Routes	Drivers must be able to view all available routes.
UR4.3	View Route Map	Drivers must be able to view the map of the route.
UR4.4	User Pick-Up Status	Drivers must be able to set the status of the user as picked or not picked and add any observation if its is wanted.
UR4.5	Reset trip	Drivers must be able to reset the trip.
UR4.6	Make a return trip	The drivers, after marking all users as picked up or on, must be able to perform the same actions to return them to their homes.
UR4.7	View Bus list	Drivers must be able to view the different bus options that exist.
UR4.8	View Bus distribution	Drivers must be able to view the different bus distributions that exist.
UR4.9	Select route	Drivers must be able to select one route.
UR5	General User Requirements	
UR5.1	Login	Users must be able to log in using their National ID (DNI) and password.
UR5.2	Logout	Users must be able to logout in any moment.
UR5.3	Secure Data Storage	User data must be stored securely and only accessible to authorized personnel.

ID	Name	Description
UR5.4	Select type of user	Users must be able to select their user profile option before login.

Table 3.1: User Requirements for AFADLA Project

3.2.- System Requirements Analysis

ID	Name	Description	User Requirement
FR1	Log In	The system must allow users to log in to the application. To successfully log in, the user must enter their National ID (DNI) and password. If there is a mismatch, a message will be displayed indicating the error.	UR5.1
FR2	Log Out	The system must allow users to log out.	UR5.2
FR3	Password Reset (Admin)	The system must allow users with administrator permissions to reset the passwords of any user.	UR2.9
FR4	Password Reset (User)	The system must allow day center users to change their own password.	UR1.3
FR5	User Management	The system must be able to display different views based on the logged-in user type and show only the relevant information to them.	UR5.4
FR6	Administrator Data Visibility	The system must provide full data visibility for administrators.	UR2.1, UR2.2, UR2.3
FR7	Administrator Data Management	The system must allow administrators to add, delete, and update details for all types of users.	UR2.5, UR2.6, UR2.7, UR2.9
FR8	Administrator View Routes	The system must enable administrators to view all routes.	UR2.4
FR9	Administrator Creation of Routes	The system must enable administrators to crate a new route.	UR2.8

ID	Name	Description	User Requirement
FR10	Staff Data Visibility	The system must provide full data visibility for staff members.	UR3.1, UR3.2, UR3.3, UR3.4
FR11	Staff Data Restriction	The system must prevent staff members from editing or deleting user data.	UR3.5
FR12	Driver View Routes	The system must enable drivers to view all routes.	UR4.2
FR13	Driver Select Route	The system must allow drivers to view select routes active route.	UR4.9
FR14	Driver View Route Map	The system must provide a map view of the route for drivers.	UR4.3
FR15	Driver Set User Status	The system must allow drivers to set the status of users as picked or not picked and add observations.	UR4.4
FR16	Driver View Bus list	The system must allow drivers to view the different bus options.	UR4.7
FR17	Driver View Bus Distribution	The system must allow drivers to view the different bus distributions.	UR4.8
FR18	Driver Reset trip	The system must allow drivers to reset the trip.	UR4.5
FR19	Driver Return trip	The system must allow drivers to perform a return trip.	UR4.6
FR20	Secure Data Storage	The system must store user data securely and ensure only authorized access.	UR5.3, UR1.6, UR4.1
FR21	User Profile Management	The system must allow daycare center users to view and edit their own profile.	UR1.1, UR1.2
FR22	User History Log	The system must provide daycare center users access to a history log of pick-up and drop-off dates.	UR1.4
FR23	User History Filter	The system must provide daycare center users access to filter their history log and short it alphabetically.	UR1.5
FR24	Administrator Creation of Bus	The system must enable administrators to crate a new bus distribution.	UR1.10

ID	Name	Description	User Requirement
NFR1	Performance	The system must handle up to 50 simultaneous users without performance degradation.	
NFR1	Scalability	The system must be scalable to support future growth in user base and features.	
NFR1	Security	The system must implement data encryption both at rest and in transit to protect sensitive information.	
NFR1	Usability	The system must have an intuitive and user-friendly interface for all types of users.	
NFR1	Maintainability	The system must be designed for easy maintenance and updates without significant downtime.	
NFR1	Compliance	The system must comply with relevant data protection regulations, such as GDPR.	
NFR1	Compatibility	The system must be compatible with various web browsers and mobile devices.	
NFR1	Availability	The system must have an internet connection. The application will be available continuously, except for maintenance tasks.	
NFR1	Web-Based System	The system must be accessible via a web browser.	

Table 3.2: System Requirements

3.3.- Use Cases

The use case model will graphically illustrate the interaction between the actors and the system. There will be four actors on the website, as there are distinctions between the two types of system users and the specific permissions and views assigned to each.

3.3.1.- Description of System Actors

This subsection provides detailed descriptions of the different actors interacting with the system. Each actor has specific roles and permissions, which are essential to understanding

their interactions within the application.

- **Name: Administrator**
  - **Description:** Represents those employees who have management roles within the company, such as managers, treasurers, administrative heads, etc. The administrator has full access to all system functionalities, including the ability to view and manage data for users, staff, and drivers. Administrators can also create, edit, and delete user accounts, assign and revoke administrative permissions, and manage routes.
- **Name: Staff Member (without administrative permissions)**
  - **Description:** Represents employees who do not have management roles and are not drivers, such as psychologists, assistant technicians, therapists, and cleaning staff. Staff members have access to view user data, worker data, and route information. They do not have the ability to edit or delete any data. Their access is restricted to viewing the information necessary for their duties.
- **Name: Driver**
  - **Description:** Represents employees responsible for driving the company van and transporting day center users to and from the center. They do not need to view user information as it is not relevant to their job role. Drivers have access to view routes and route maps. They can update the pick-up status of users and make observations. Drivers can also view and select bus routes, and reset trips if needed. They do not have access to personal data of users.
- **Name: Day Center User**
  - **Description:** Represents the users of the day center. Since most of them are elderly, it is common for a family member or caregiver to manage this profile. Day center users have access to view and edit their personal profiles, change their passwords, and view their history log of pick-up and drop-off events. They cannot access or modify data of other users.

3.3.2.- Use Case Model

This section presents the use case diagrams for the application, categorized by the different actors involved. These diagrams illustrate the interactions between the users and the system, highlighting the distinct roles and permissions of each actor. By visually mapping out these

interactions, we can clearly define the functionalities and responsibilities assigned to each user type within the application.

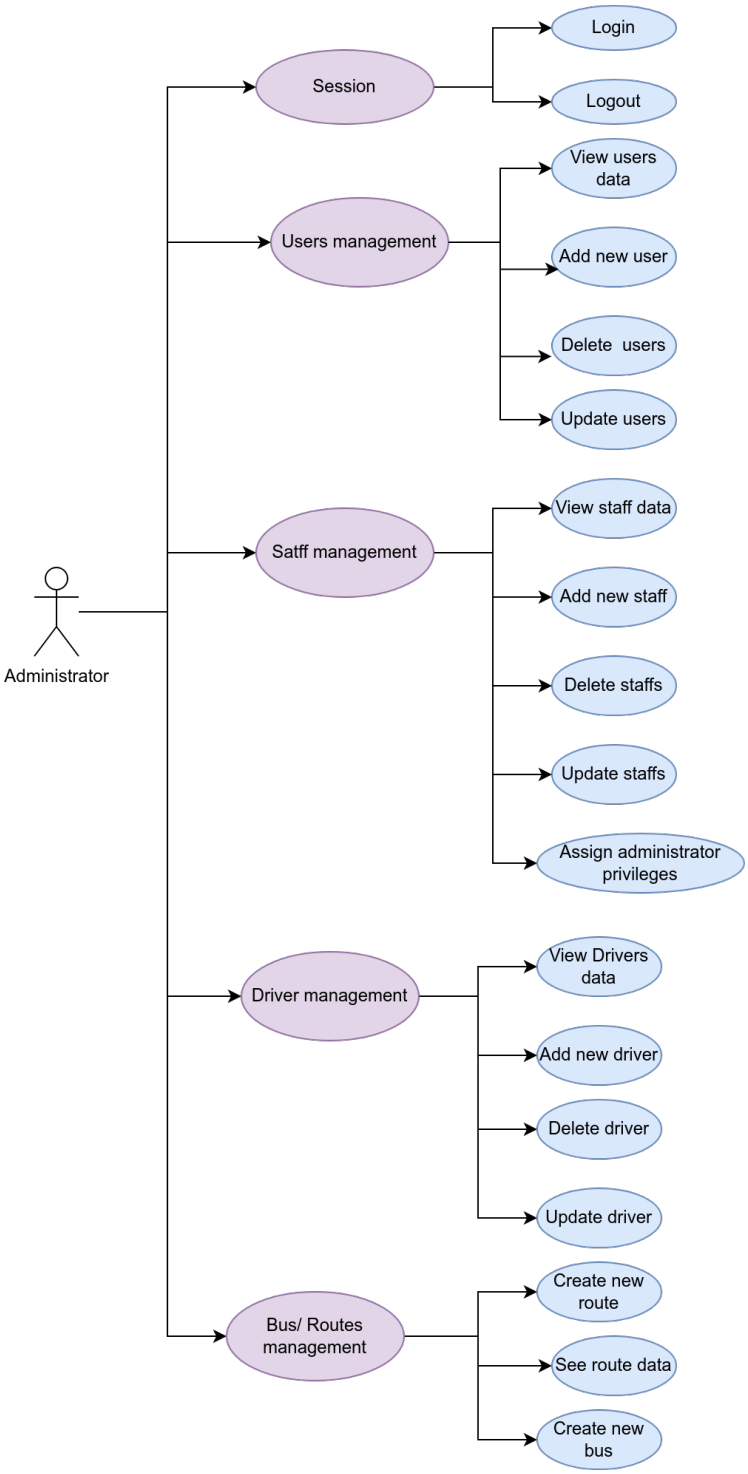

Figure 3.1: Model for the Administrator Use Cases

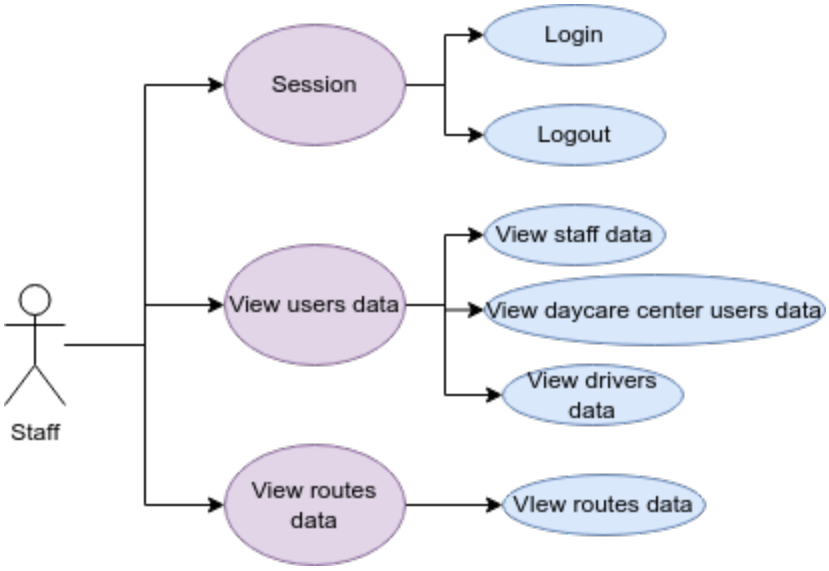

Figure 3.2: Model for the Staff Use Cases

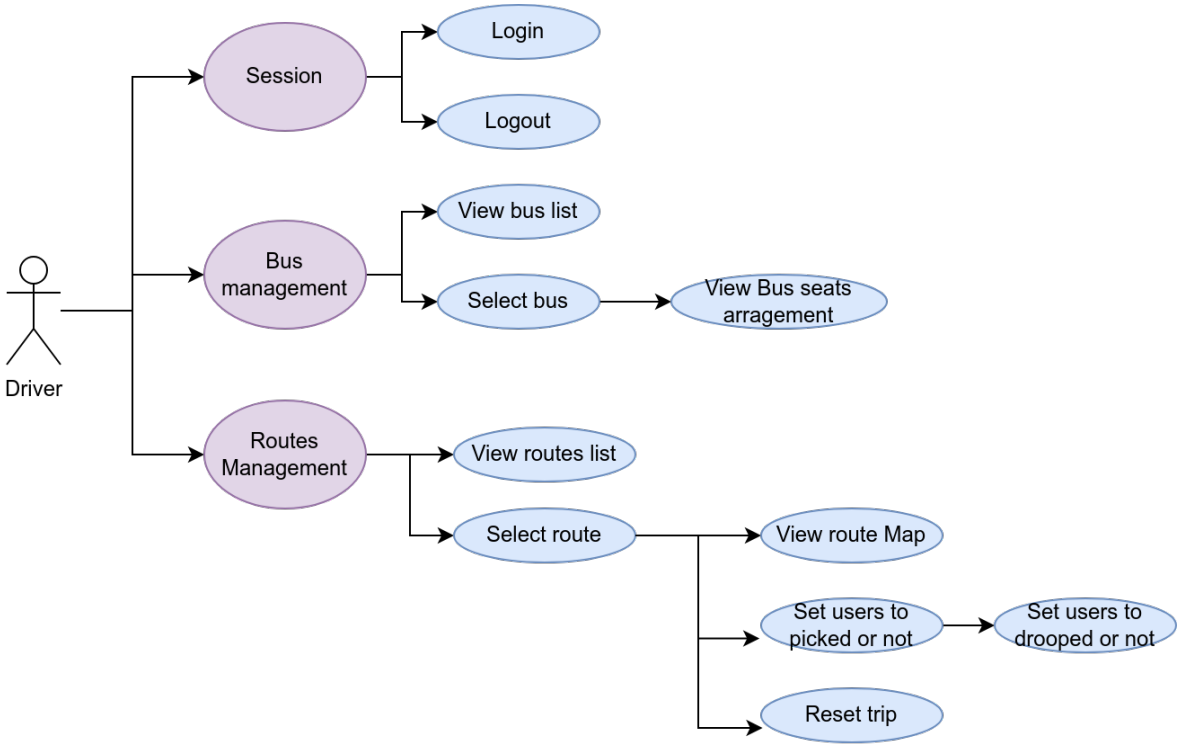

Figure 3.3: Model for the Driver Use Cases

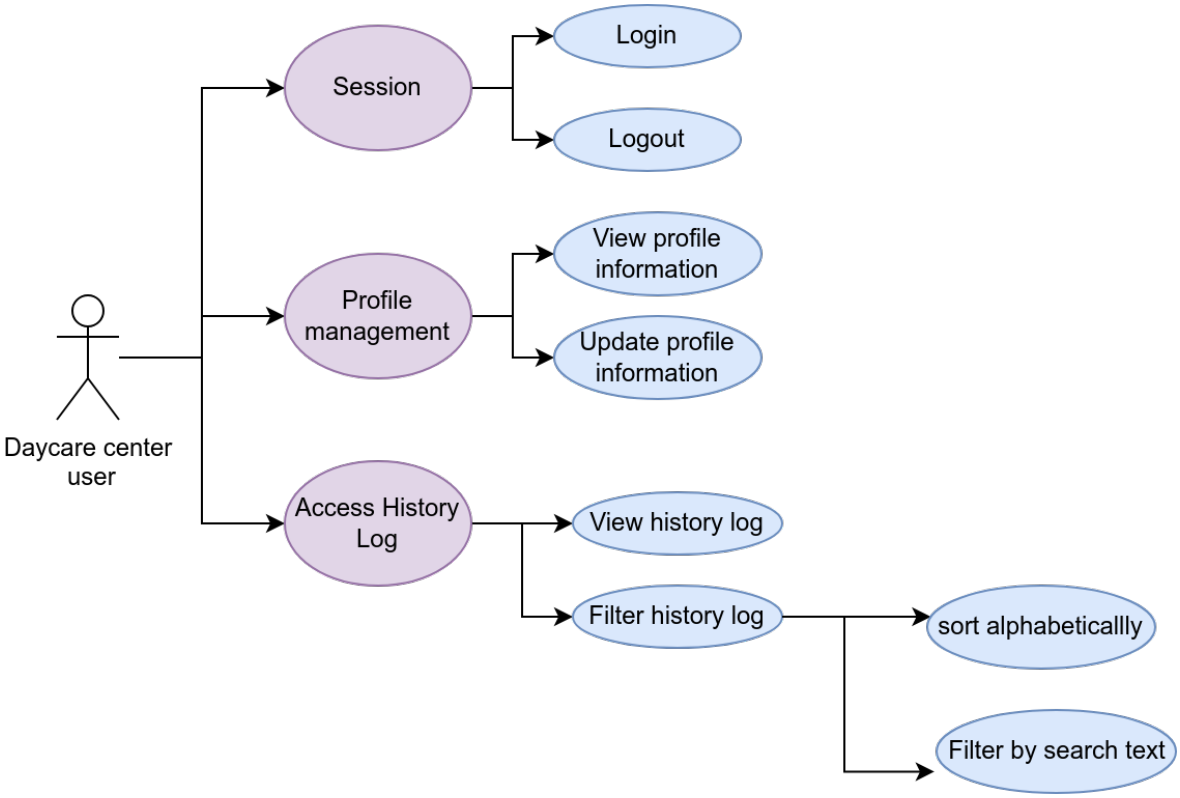

Figure 3.4: Model for the Daycare center user Use Cases

3.3.3.- Description of Use Cases

3.3.3.1.- Use Case: Staff & Administrator Login

Name:	Staff/ Administrator Login
Description:	Allows the user to access the main menu of the application.
Actors:	Staff
Preconditions:	The user must be registered in the application database.
Normal Flow:	1. The user opens the application on their device. 2. The user selects the option to be logged as a staff member. 3. The user enters their login credentials. 4. The user accesses the main menu of the application.

Alternative Flow:	1. The system checks the validity of the credentials against the database. 2. If the credentials are invalid, the system notifies the user that the credentials are incorrect.
Post-conditions:	The user is logged into the application.

Table 3.3: Use Case: Staff & Administrator Login

3.3.3.2.- Use Case: Driver Login

Name:	Driver Login
Description:	Allows the driver to access the main menu of the application.
Actors:	Driver
Preconditions:	The driver must be registered in the application database.
Normal Flow:	1. The driver opens the application on their device. 2. The driver selects the option to log in as a driver. 3. The driver enters their login credentials. 4. The driver accesses the main menu of the application.
Alternative Flow:	1. The system checks the validity of the credentials against the database. 2. If the credentials are invalid, the system notifies the driver that the credentials are incorrect.
Post-conditions:	The driver is logged into the application.

Table 3.4: Use Case: Driver Login

3.3.3.3.- Use Case: Daycare center user Login

Name:	Daycare Center User Login
Description:	Allows the daycare center user to access the main menu of the application.
Actors:	Daycare Center User
Preconditions:	The daycare center user must be registered in the application database.
Normal Flow:	1. The daycare center user opens the application on their device. 2. The daycare center user selects the option to log in as a daycare center user. 3. The daycare center user enters their login credentials. 4. The daycare center user accesses the main menu of the application.
Alternative Flow:	1. The system checks the validity of the credentials against the database. 2. If the credentials are invalid, the system notifies the daycare center user that the credentials are incorrect.
Post-conditions:	The daycare center user is logged into the application.

Table 3.5: Use Case: Daycare Center User Login

3.3.3.4.- Use Case: Logout

Name:	Logout
Description:	Allows the user to log out of the application, ending the current session.
Actors:	Staff/ Administrator/ Daycare center user/ Driver
Preconditions:	The user must be logged into the application.

Normal Flow:	1. The user selects the logout option from the application menu. 2. The system logs the user out and ends the current session.
Alternative Flow:	1. If the system encounters an error during the logout process, it notifies the user and provides an option to try again.
Post conditions:	The user is logged out of the application, and the session is ended.

Table 3.6: Use Case: Logout

3.3.3.5.- Use Case: View Users Data

Name:	View Users Data
Description:	Allows the user to view the list of daycare center users along with their details.
Actors:	Staff, Administrator
Preconditions:	The user must be logged into the application and be part of the staff or administrator group.
Normal Flow:	1. The administrator/ staff selects the "Users List" option in the main panel. 2. The administrator/ staff view the list of daycare center users with their details.
Alternative Flow:	None
Post-conditions:	

Table 3.7: Use Case: View Users Data

3.3.3.6.- Use Case: View Drivers Data

Name:	View Drivers Data
Description:	Allows the user to view the list of daycare center users along with their details.
Actors:	Staff, Administrator
Preconditions:	The user must be logged into the application and be part of the staff or administrator group.
Normal Flow:	1. The administrator/ staff selects the "Drivers List" option in the main panel. 2. The administrator/ staff view the list of drivers with their details.
Alternative Flow:	None
Post-conditions:	

Table 3.8: Use Case: View Drivers Data

3.3.3.7.- Use Case: View Users Data

Name:	View Staff Data
Description:	Allows the user to view the list of staff members along with their details.
Actors:	Staff, Administrator
Preconditions:	The user must be logged into the application and be part of the staff or administrator group.
Normal Flow:	1. The administrator/ staff selects the "Staff List" option in the main panel. 2. The administrator/ staff view the list of staff members with their details.
Alternative Flow:	None
Post-conditions:	

Table 3.9: Use Case: View Staff Data

3.3.3.8.- Use Case: View Routes Data

Name:	View Routes Data
Description:	Allows the user to view the list of routes along with their details.
Actors:	Staff, Administrator
Preconditions:	The user must be logged into the application and be part of the staff or administrator group.
Normal Flow:	1. The administrator/ staff selects the "Routes List" option in the main panel. 2. The administrator/ staff view the list of routes with their details.
Alternative Flow:	None
Post-conditions:	

Table 3.10: Use Case: View Routes Data

3.3.3.9.- Use Case: Add New Daycare Center User

Name:	Add New Daycare Center User
Description:	Allows the user to create a new daycare center user in the system.
Actors:	Administrator
Preconditions:	The user must be logged into the application and be part of the staff or administrator group.

Normal Flow:	1. The administrator selects the "Add New User" option in the main panel. 2. The system displays a form for entering the new user's details. 3. The administrator fills in the required information (e.g., name, contact details, medical information). 4. The administrator submits the form. 5. The system saves the new user's information and adds them to the users list.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the user and prompts for correction.
Post-conditions:	The new daycare center user is successfully added to the system and can now be managed by staff or administrators.

Table 3.11: Use Case: Add New Daycare Center User

3.3.3.10.- Use Case: Add New Staff Member

Name:	Add New Staff Member
Description:	Allows the administrator to create a new staff member in the system.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator selects the "Add New Staff" option in the main panel. 2. The system displays a form for entering the new staff member's details. 3. The administrator fills in the required information (e.g., name, contact details, role). 4. The administrator submits the form. 5. The system saves the new staff member's information and adds them to the staff list.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction.
Post-conditions:	The new staff member is successfully added to the system and can now be managed by administrators.

Table 3.12: Use Case: Add New Staff Member

3.3.3.11.- Use Case: Add New Driver

Name:	Add New Driver
Description:	Allows the administrator to create a new driver in the system.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator selects the "Add New Driver" option in the main panel. 2. The system displays a form for entering the new driver's details. 3. The administrator fills in the required information (e.g., name, contact details, driver's license number). 4. The administrator submits the form. 5. The system saves the new driver's information and adds them to the drivers list.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction.
Post-conditions:	The new driver is successfully added to the system and can now be managed by administrators.

Table 3.13: Use Case: Add New Driver

3.3.3.12.- Use Case: Add New Route

Name:	Add New Route
Description:	Allows the administrator to create a new route in the system. The administrator can provide a name, a description, select the users for the route, and assign a driver. The total distance (round trip) and the order of user visits will be calculated.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator selects the "Add New Route" option in the main panel. 2. The system displays a form for entering the new route's details. 3. The administrator fills in the required information: • Route name • Route description • Select users for the route (selected users are removed from the options list and appear below) • Select a driver for the route (only one driver can be selected; selecting a new driver replaces the previous selection) 4. The administrator submits the form. 5. The system saves the new route's information, calculates the total distance (round trip), and determines the order of user visits.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction. 2. If a selected user is deselected, the user is removed from the selected list and reappears in the options list.
Post-conditions:	The new route is successfully added to the system and can now be managed by administrators. The total distance and order of visits are calculated and stored.

Table 3.14: Use Case: Add New Route

3.3.3.13.- Use Case: Add New Bus

Name:	Add New Bus

Description:	Allows the administrator to create a new bus in the system. The administrator can provide a name, a description, select the users for the bus, and assign a driver. The distributions of the bus seats will be calculated.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.
Normal Flow:	1. The administrator selects the "Add New Bus" option in the main panel. 2. The system displays a form for entering the new bus' details. 3. The administrator fills in the required information: • Bus name • Bus description • Select users for the bus (selected users are removed from the options list and appear below) • Select a driver for the bus (only one driver can be selected; selecting a new driver replaces the previous selection) 4. The administrator submits the form. 5. The system saves the new bus' information and calculates the distribution of the seats.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction. 2. If a selected user is deselected, the user is removed from the selected list and reappears in the options list.
Post-conditions:	The new bus is successfully added to the system and can now be managed by drivers. The distribution of the seats are calculated and stored.

Table 3.15: Use Case: Add New Bus

3.3.3.14.- Use Case: Delete User

Name:	Delete User
Description:	Allows the administrator to delete a user from the system. The administrator can navigate to the users view and delete a user from the list.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.
Normal Flow:	1. The administrator navigates to the "Users" tab in the main panel. 2. The system displays a list of users. 3. The administrator selects the delete option next to the user they wish to delete. 4. The system deletes the user from the database and updates the user list.
Alternative Flow:	1. If the system encounters an error during the deletion process, it notifies the administrator and the user remains in the system.
Post-conditions:	The user is successfully deleted from the system and the user list is updated.

Table 3.16: Use Case: Delete User

3.3.3.15.- Use Case: Delete Driver

Name:	Delete Driver
Description:	Allows the administrator to delete a driver from the system. The administrator can navigate to the drivers view and delete a driver from the list.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Drivers" tab in the main panel. 2. The system displays a list of drivers. 3. The administrator selects the delete option next to the driver they wish to delete. 4. The system deletes the driver from the database and updates the driver list.
Alternative Flow:	1. If the system encounters an error during the deletion process, it notifies the administrator and the driver remains in the system.
Post-conditions:	The driver is successfully deleted from the system and the driver list is updated.

Table 3.17: Use Case: Delete Driver

3.3.3.16.- Use Case: Delete Staff

Name:	Delete Staff
Description:	Allows the administrator to delete a staff member from the system. The administrator can navigate to the staff view and delete a staff member from the list.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Staff" tab in the main panel. 2. The system displays a list of staff members. 3. The administrator selects the delete option next to the staff member they wish to delete. 4. The system deletes the staff member from the database and updates the staff list.
Alternative Flow:	1. If the system encounters an error during the deletion process, it notifies the administrator and the staff member remains in the system.
Post-conditions:	The staff member is successfully deleted from the system and the staff list is updated.

Table 3.18: Use Case: Delete Staff

3.3.3.17.- Use Case: Update Driver

Name:	Update Driver
Description:	Allows the administrator to update a driver's information in the system. The administrator can navigate to the drivers view and update the necessary fields.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Drivers" tab in the main panel. 2. The system displays a list of drivers. 3. The administrator selects the driver they wish to update. 4. The system displays a form with the driver's current information (being the DNI and username fields disabled). 5. The administrator updates the necessary fields: Name, Last Name, Phone Number, Address, Roles, Driver License. 6. The administrator submits the form. 7. The system saves the updated information and displays a confirmation message.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction.
Post-conditions:	The driver's information is successfully updated in the system.

Table 3.19: Use Case: Update Driver

3.3.3.18.- Use Case: Update Staff

Name:	Update Staff
Description:	Allows the administrator to update a staff member's information in the system. The administrator can navigate to the staff view and update the necessary fields.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Staff" tab in the main panel. 2. The system displays a list of staff members. 3. The administrator selects the staff member they wish to update. 4. The system displays a form with the staff member's current information (being the DNI and username fields disabled). 5. The administrator updates the necessary fields:Name, Last Name, Phone Number, Address, Roles, Admin (checkbox). 6. The administrator submits the form. 7. The system saves the updated information and displays a confirmation message.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction.
Post-conditions:	The staff member's information is successfully updated in the system.

Table 3.20: Use Case: Update Staff

3.3.3.19.- Use Case: Update Daycare Center User

Name:	Update Daycare Center User
Description:	Allows the administrator to update a daycare center user's information in the system. The administrator can navigate to the users view and update the necessary fields.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Users" tab in the main panel. 2. The system displays a list of users. 3. The administrator selects the user they wish to update. 4. The system displays a form with the user's current information (being the DNI and username fields disabled). 5. The administrator updates the necessary fields:Name, Last Name, Phone Number, Address, Allergies, Disabilities, Medication, Others, Illnesses, Can't sit with. 6. The administrator submits the form. 7. The system saves the updated information and displays a confirmation message.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction.
Post-conditions:	The user's information is successfully updated in the system.

Table 3.21: Use Case: Update Daycare Center User

3.3.3.20.- Use Case: Assign or Revoke User Permissions

Name:	Assign or Revoke User Permissions
Description:	Allows the administrator to assign or revoke user permissions. The administrator can navigate to the staff view, select a staff member, and assign or revoke permissions to the staff members.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Staff" tab in the main panel. 2. The system displays a list of staff members. 3. The administrator selects the staff member they wish to assign or revoke permissions. 4. The system displays a form with the staff member's current information. 5. The administrator can assign or revoke admin permissions by checking or unchecking the "Admin" checkbox. 6. If permissions are updated, the administrator submits the form. 7. The system updates the staff member's information and updates the staff list.
Alternative Flow:	1. If the required information is incomplete or incorrect, the system notifies the administrator and prompts for correction.
Post-conditions:	The staff member's permissions are successfully updated and the staff list is updated.

Table 3.22: Use Case: Assign or Revoke User Permissions

3.3.3.21.- Use Case: Change User Password

Name:	Change User Password
Description:	Allows the administrator to change the password of a user in the system. The administrator can navigate to the user view, select a user, and change their password.
Actors:	Administrator
Preconditions:	The administrator must be logged into the application and have the appropriate permissions.

Normal Flow:	1. The administrator navigates to the "Staff", "Drivers", or "Users" tab in the main panel. 2. The system displays a list of staff members, drivers, or users. 3. The administrator selects the user they wish to update. 4. The system displays a form with the user's current information. 5. The administrator clicks on the "Change password" button. 6. The system displays a dialog to set a new password. 7. The administrator enters the new password and confirms it. 8. The administrator clicks "Save". 9. The system saves the new password and displays a confirmation message.
Alternative Flow:	1. If the new password and the confirmation do not match, the system notifies the administrator and prompts for correction. 2. If the administrator cancels the operation, the user's password remains unchanged. 3. If the system encounters an error during the password change process, it notifies the administrator and the user's password remains unchanged.
Post-conditions:	The user's password is successfully updated in the system.

Table 3.23: Use Case: Change User Password

3.3.3.22.- Use Case: View Bus List

Name:	View Bus List
Description:	Allows the driver to view the list of available buses in the system. The driver can navigate to the bus view and see the list of buses.
Actors:	Driver
Preconditions:	The driver must be logged into the application.
Normal Flow:	1. The driver navigates to the "Bus" tab in the main panel. 2. The system displays a list of available buses.
Post-conditions:	The driver sees the list of available buses.

Table 3.24: Use Case: View Bus List

3.3.3.23.- Use Case: Select Bus and View Seats

Name:	Select Bus and View Seats
Description:	Allows the driver to select a bus from the list and view its seating arrangement. The driver can navigate to the bus view, select a bus, and see the seats.
Actors:	Driver
Preconditions:	The driver must be logged into the application. The driver must have already navigated to the bus list.
Normal Flow:	1. The driver navigates to the "Bus" tab in the main panel. 2. The system displays a list of available buses. 3. The driver selects a bus from the list. 4. The system displays the seating arrangement for the selected bus.
Post-conditions:	The driver sees the seating arrangement of the selected bus.

Table 3.25: Use Case: Select Bus and View Seats

3.3.3.24.- Use Case: View Available Routes

Name:	View Available Routes
Description:	Allows the driver to view the list of available routes in the system. The driver can navigate to the routes view and see the list of routes.
Actors:	Driver
Preconditions:	The driver must be logged into the application.
Normal Flow:	1. The driver navigates to the "Routes" tab in the main panel. 2. The system displays a list of available routes.
Post-conditions:	The driver sees the list of available routes.

Table 3.26: Use Case: View Available Routes

3.3.3.25.- Use Case: Select Route and View Users

Name:	Select Route and View Users
Description:	Allows the driver to select a route from the list and view the users assigned to it. The driver can navigate to the routes view, select a route, and see the users.
Actors:	Driver
Preconditions:	The driver must be logged into the application. The driver must have already navigated to the routes list.
Normal Flow:	1. The driver navigates to the "Routes" tab in the main panel. 2. The system displays a list of available routes. 3. The driver selects a route from the list. 4. The system displays the users assigned to the selected route.
Post-conditions:	The driver sees the users assigned to the selected route.

Table 3.27: Use Case: Select Route and View Users

3.3.3.26.- Use Case: Mark Users as Picked Up or Not, Add Observations

Name:	Mark Users as Picked Up or Not, Add Observations
Description:	Allows the driver to mark users as picked up or not picked up and add observations if desired.
Actors:	Driver
Preconditions:	The driver must be logged into the application and have selected a route.
Normal Flow:	1. The driver selects a route from the "Routes" tab. 2. The system displays the users assigned to the selected route. 3. For each user, the driver can: • Mark the user as "Picked Up" • Mark the user as "Not Picked Up" • Add an observation 4. The system saves the status updates and any observations.
Alternative Flow:	None
Post-conditions:	The users' statuses are successfully updated in the system, and any observations are saved.

Table 3.28: Use Case: Mark Users as Picked Up or Not, Add Observations

3.3.3.27.- Use Case: View Route Map

Name:	View Route Map
Description:	Allows the driver to view the map of the selected route. The driver can navigate to the routes view, select a route, and view its map.
Actors:	Driver
Preconditions:	The driver must be logged into the application and have selected a route.

Normal Flow:	1. The driver navigates to the "Routes" tab in the main panel. 2. The system displays a list of available routes. 3. The driver selects a route from the list. 4. The system displays the users assigned to the selected route. 5. The driver clicks on the "View map" button. 6. The system displays the map of the selected route.
Alternative Flow:	None
Post-conditions:	The driver sees the map of the selected route.

Table 3.29: Use Case: View Route Map

3.3.3.28.- Use Case: View History

Name:	View History
Description:	Allows the daycare center user to view their event history. The user can see all actions such as pickup, drop off, missing, and arrived, along with the date and any observations.
Actors:	Daycare Center User
Preconditions:	The user must be logged into the application.
Normal Flow:	1. The user logs into the application. 2. The user navigates to the "Events" section. 3. The system displays the user's event history, including actions, dates, and observations.
Post-conditions:	The user can see their complete event history.

Table 3.30: Use Case: View History

3.3.3.29.- Use Case: Sort and Filter History

Name:	Sort and Filter History
Description:	Allows the daycare center user to sort their event history by clicking on the arrows next to "Action" or "Date" and filter the actions by searching for specific text.
Actors:	Daycare Center User
Preconditions:	The user must be logged into the application and have event history to view.
Normal Flow:	1. The user logs into the application. 2. The user navigates to the "Events" section. 3. The system displays the user's event history. 4. The user clicks on the arrows next to "Action" or "Date" to sort the history. 5. The system sorts the event history based on the selected criterion. 6. The user enters text in the search bar to filter the events. 7. The system filters the event history based on the entered text and displays the matching results.
Post-conditions:	The user's event history is sorted and filtered based on their preferences.

Table 3.31: Use Case: Sort and Filter History

3.3.3.30.- Use Case: View Personal Information

Name:	View Personal Information
Description:	Allows the daycare center user to view their personal information, including basic information and other relevant details like allergies, disabilities, illnesses, and medication.
Actors:	Daycare Center User
Preconditions:	The user must be logged into the application.

Normal Flow:	1. The user logs into the application. 2. The user navigates to the main panel. 3. The user clicks on the "My Profile" button. 4. The system displays the user's personal information, including: • Basic Information: Username, Name, Last Name, DNI, Phone Number, Address. • Others: Allergies, Disabilities, Illnesses, Medication and others.
Post-conditions:	The user can view their complete personal information.

Table 3.32: Use Case: View Personal Information

3.3.3.31.- Use Case: Edit Personal Information

Name:	Edit Personal Information
Description:	Allows the daycare center user to edit their personal information, with the exception of the Username and DNI. Changes to the basic information need to be saved manually, while changes to the "Others" section are updated automatically.
Actors:	Daycare Center User
Preconditions:	The user must be logged into the application.

Normal Flow:	1. The user logs into the application. 2. The user navigates to the main panel. 3. The user clicks on the "My Profile" button. 4. The system displays the user's personal information. 5. To edit basic information (Name, Last Name, Phone Number, Address): (a) The user modifies the desired fields. (b) The user clicks the "Save changes" button. (c) The system updates the basic information. 6. To edit information in the "Others" section (Allergies, Disabilities, Illnesses, Medication): (a) The user navigates to the specific section. (b) The user adds a new entry by typing into the input field and clicking "Add to list". (c) The system automatically updates the list with the new entry. (d) To delete an entry, the user clicks the red button next to the item. (e) The system automatically removes the entry from the list. 7. To change the profile photo: (a) The user clicks the blue button above the profile photo. (b) The system opens a pop-up to select a new photo from the device. (c) The user selects and uploads the new photo. (d) The system updates the profile photo.
Post-conditions:	The user's personal information is updated in the system.

Table 3.33: Use Case: Edit Personal Information

3.3.3.32.- Use Case: Change Password

Name:	Change Password
Description:	Allows the daycare center user to change their password. The user can navigate to their profile and initiate a password change process.
Actors:	Daycare Center User
Preconditions:	The user must be logged into the application.
Normal Flow:	1. The user logs into the application. 2. The user navigates to the main panel. 3. The user clicks on the "My Profile" button. 4. The system displays the user's personal information. 5. The user clicks on the "Change password" button. 6. The system displays a pop-up to set a new password. 7. The user enters the new password and confirms it. 8. The user clicks "Save". 9. The system saves the new password and displays a confirmation message.
Alternative Flow:	1. If the new password and the confirmation do not match, the system notifies the user and prompts for correction. 2. If the user cancels the operation, the password remains unchanged. 3. If the system encounters an error during the password change process, it notifies the user and the password remains unchanged.
Post-conditions:	The user's password is successfully updated in the system.

Table 3.34: Use Case: Change Password

3.4.- Architectural Design

3.4.1.- System Functionality

The AFADLA system's architecture ensures seamless interaction between users and the backend services through a well-defined workflow involving multiple components hosted on different platforms. Here is a detailed explanation of the system functionality:

1. **User Access**:

- Users access the system by navigating to `afadla.com`, a domain registered with Cloudflare.
- This domain is configured to route traffic to the frontend and backend services.

2. **Frontend Hosting**:

- The frontend code is hosted on Vercel, a platform for frontend deployment and serverless functions.
- When a user accesses `afadla.com`, they are served the frontend application from Vercel.

3. **Backend Interaction**:

- The frontend application makes API calls to `api.afadla.com`, which is another subdomain registered with Cloudflare.
- This subdomain is configured to route traffic to an AWS EC2 instance where the backend application is deployed.

4. **Backend Hosting**:

- The backend application, hosted on an AWS EC2 instance, handles all business logic and data processing.
- It receives requests from the frontend and processes them accordingly.

5. **Database Operations**:

- The backend application interacts with a DynamoDB database, also hosted on AWS.
- It performs necessary operations such as fetching, storing, and updating data.

6. **DNS Configuration**:

- The DNS records in Cloudflare ensure that `afadla.com` points to Vercel, while `api.afadla.com` points to the EC2 instance on AWS.
- This setup ensures that all requests are properly routed through the necessary components.

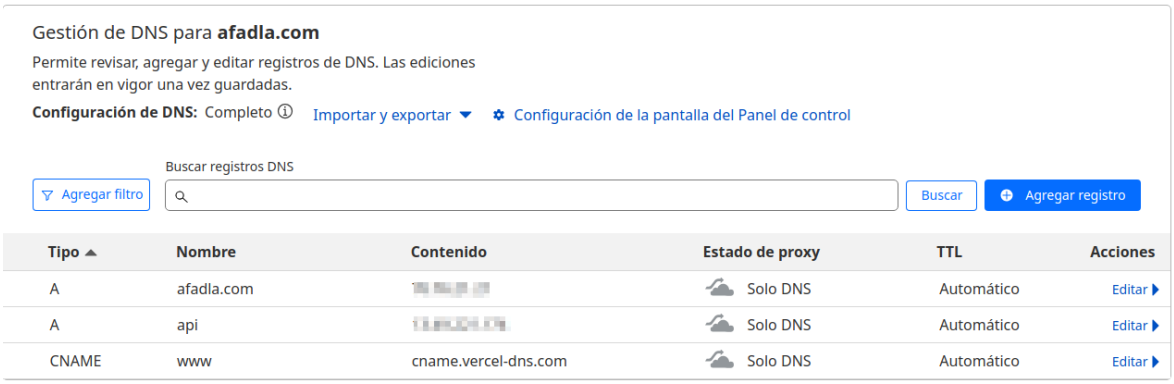

Figure 3.5: DNS configuration in cloudflare

7. **Security Measures**:

- For security reasons, a rule is configured in Cloudflare to reject any requests originating outside of Spain. This enhances security by ensuring that only users from Spain can access the system.

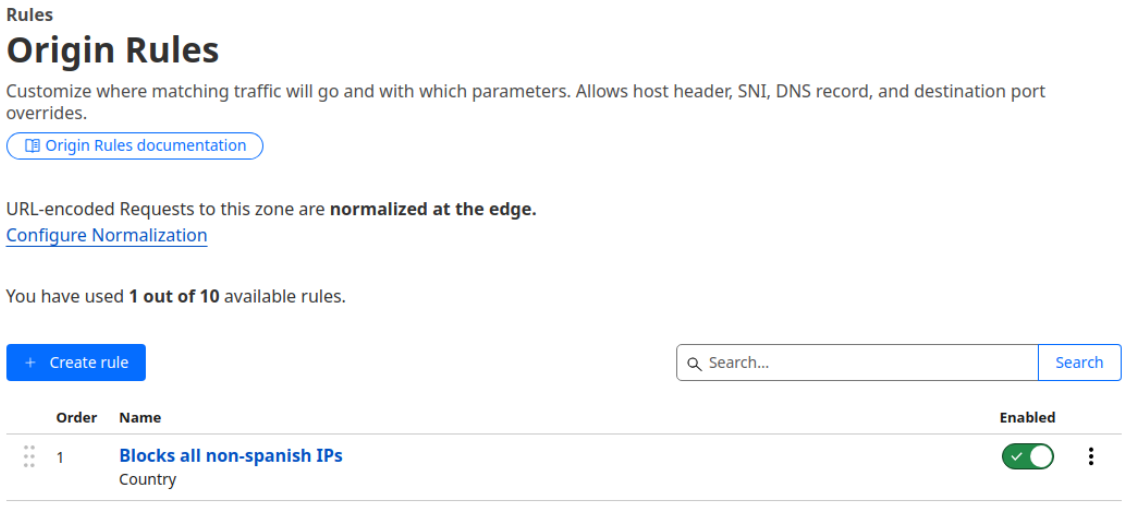

Figure 3.6: DNS configuration in Cloudflare

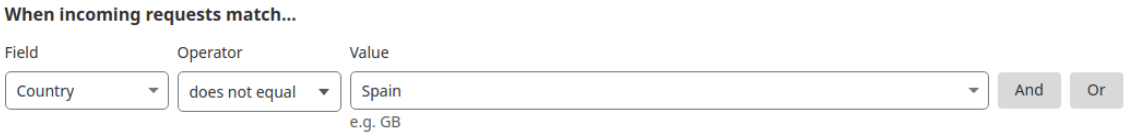

Figure 3.7: DNS rule detail

By leveraging these services, the system benefits from high availability, scalability, and reliability. Using Cloudflare for DNS management adds an extra layer of performance and security, ensuring that the users have a smooth and secure experience while interacting with the application.

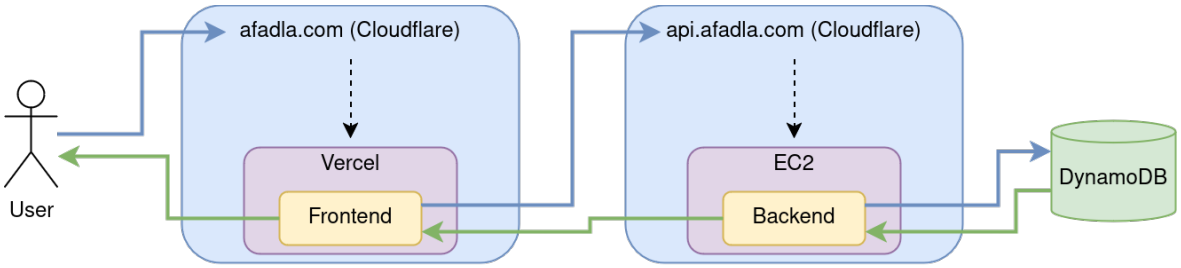

Figure 3.8: System Functionality Diagram

3.4.2.- System Architecture

The architecture used to develop the AFADLA web application employs the Model-View-Controller (MVC) pattern. This architectural style separates the application into three layers based on their responsibilities. The functionality of each layer is as follows:

- **Model**: This layer interacts with the data. It contains mechanisms that allow accessing and modifying information, which is usually stored in a database. In the case of the AFADLA web application, the model layer connects with DynamoDB for data storage and AWS S3 for storing user profile images.
- **View**: This layer presents the model in a suitable format for interaction, typically through a user interface. The view allows interaction with the data but does not provide direct access to it. In this project, the view is implemented using React, which enables users to view and edit their profiles, as well as interact with other application features.
- **Controller**: This layer contains navigational code. It receives events that serve as a link between the views and the models. In AFADLA, the back-end is developed in Python and operates on local-host port 5000, handling the business logic and communication between the front-end and the model.

To better understand this pattern, see the next figure:

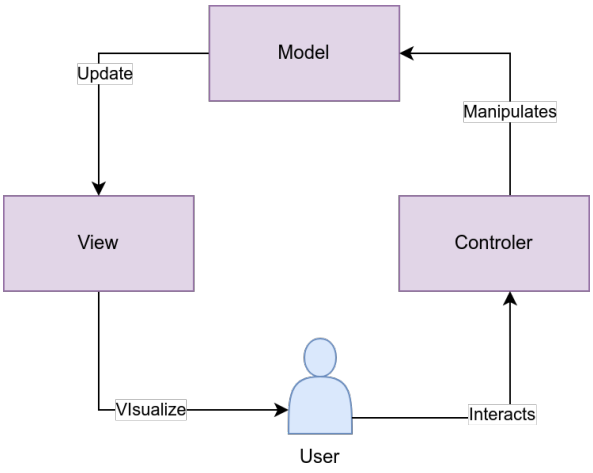

Figure 3.9: MVC pattern

The MVC pattern functions as follows:

1. The user interacts with the application through the controller.

2. The controller receives the user's interactions and communicates them to the model.
3. The model performs the actions received through the controller and interacts with the database.
4. The model returns the results of its actions and updates the view.
5. The user sees the changes in the view following their interaction.

The reasons for using this pattern include:

- It clearly separates each type of logic, making maintenance easier.
- It facilitates the scalability of the application.
- It simplifies the creation of variations of the same data.
- It reduces complexity, as there is no life cycle for the pages.

3.4.3.- Database Model

The database model for the AFADLA project is implemented using Amazon DynamoDB. DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability. The following characteristics are used for the tables in DynamoDB:

- **Deletion Protection:** Disabled
- **Read Capacity Mode:** On-Demand
- **Write Capacity Mode:** On-Demand

The On-Demand capacity mode is particularly suitable for this project because it allows DynamoDB to automatically manage the throughput capacity based on the actual workload. This setup is cost-effective and efficient, especially for applications like AFADLA, where the number of requests is not expected to be very high. Reserving at least one virtual capacity unit might not be practical or economical given the expected usage patterns. By using the On-Demand mode, we can ensure that the system scales up and down seamlessly, providing the required throughput without incurring unnecessary costs. This way, we pay only for the actual read and write operations performed, which is more economical for applications with variable or unpredictable traffic patterns.

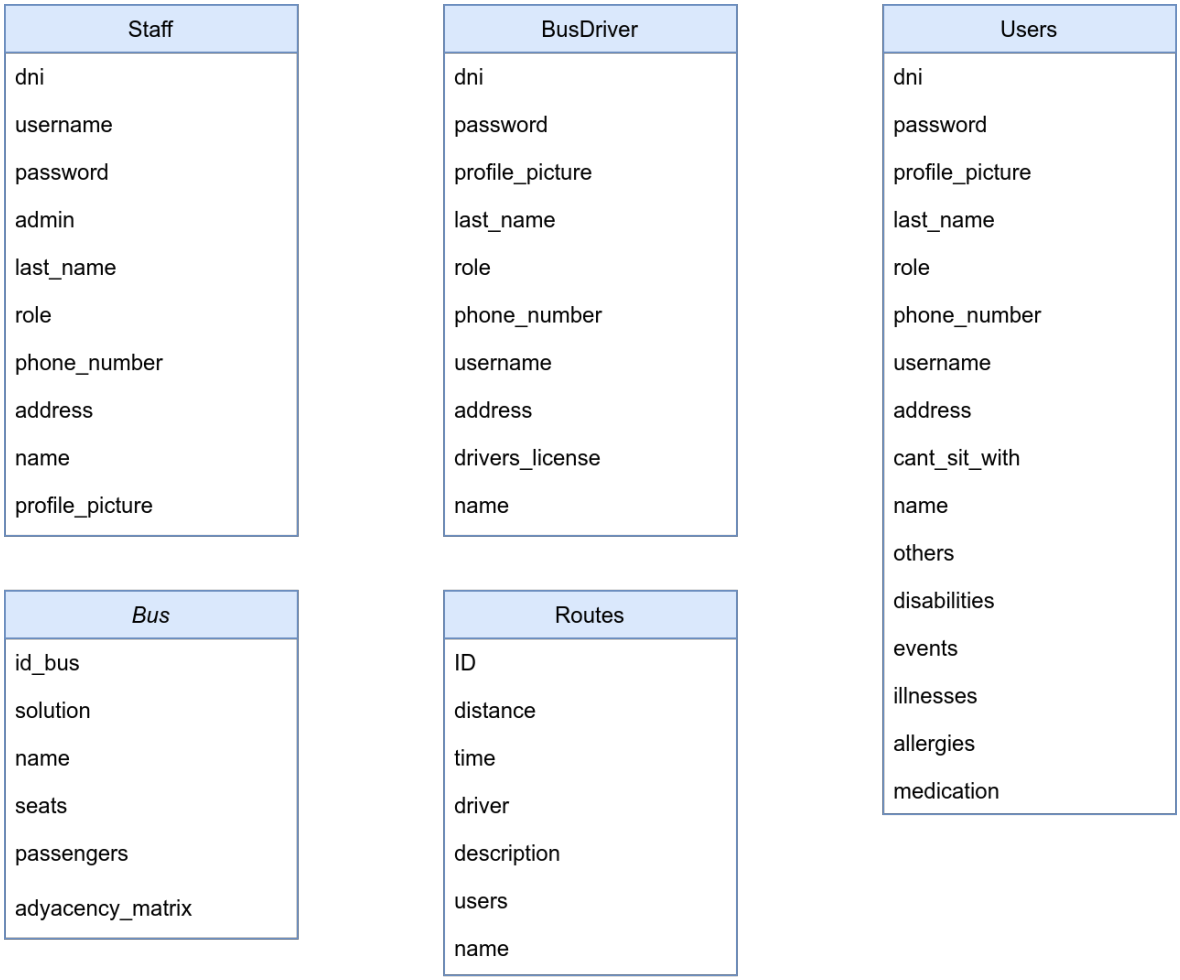

Figure 3.10: Database Model diagram

3.4.3.1.- Tables and Their Attributes

1. **Table: Bus**

- **id_bus:** The primary key of the table, a unique identifier (GUID).
  Example: `8c9a19dc-92fa-49a1-afaf-5ce35af3d107`
- **solution:** A JSON object mapping users to their respective seats using their DNI. Each element in the array corresponds to the placement of a user in the bus. For example, consider the following JSON object:

  ```
  [ { "S" : "4" }, { "S" : "8" }, { "S" : "12" } ]
  ```

  This implies that the user identified by the first entry in the `passengers` list is seated in position 4, the second user in position 8, and the third user in position 12.
- **name:** Text value representing the name given to the specific bus configuration.
  Example: `Bus 25a108f7-c0fd-4857-8fbc-3c4ed47434d2`
- **seats:** JSON array representing the distribution of free and aisle seats in the bus.
  Example: `["L":["N":"-1", "N":"0", "N":"-1", "N":"-1"], "L":["N":"0", "N":"0", "N":"-1", "N":"-1"], "L":["N":"0", "N":"0", "N":"-1", "N":"-1"], "L":["N":"0", "N":"0", "N":"0", "N":"-1"]]`
- **passengers:** JSON array listing the passengers to be seated in the bus. This list is used to retrieve the appropriate information for each user. For example, consider the following JSON array:

  ```
  [ { "S" : "39817107V" }, { "S" : "09817107V" }, { "S" : "68676326R" } ]
  ```

  Here, the array contains the DNIs of three passengers. These DNIs are used to fetch detailed information about each user. The `solution` array then indicates the specific seats assigned to these users.
- **adyacency_matrix:** JSON array representing the adjacency matrix for seat arrangement.
  Example: `["L":["N":"1", "N":"0", "N":"0", "N":"1", "N":"1", "N":"1", "N":"0", "N":"1"], "L":["N":"0", "N":"1", "N":"0", "N":"0", "N":"0", "N":"0", "N":"0", "N":"0"], "L":["N":"0", "N":"0", "N":"1", "N":"1",`

```
"N":"1", "N":"1", "N":"1", "N":"1"], "L":["N":"1", "N":"0", "N":"1",
"N":"1", "N":"0", "N":"1", "N":"0", "N":"0"], "L":["N":"1", "N":"0",
"N":"1", "N":"0", "N":"1", "N":"0", "N":"1", "N":"1"], "L":["N":"1",
"N":"0", "N":"1", "N":"1", "N":"1", "N":"1", "N":"0", "N":"1"], "L":["N":"0",
"N":"0", "N":"1", "N":"0", "N":"1", "N":"0", "N":"1", "N":"1"], "L":["N":"1",
"N":"0", "N":"1", "N":"1", "N":"1", "N":"1", "N":"1", "N":"1"]]
```


Interpretation

The `passengers` array provides a list of user DNIs, while the `solution` array maps these users to specific seats. Formally, let `passengers` $= [\text{DNI}_1, \text{DNI}_2, \ldots, \text{DNI}_n]$ be the list of DNIs, and `solution` $= [s_1, s_2, \ldots, s_n]$ be the list of seat positions. Then, the seat s_i is assigned to the user with DNI DNI_i for $i = 1, 2, \ldots, n$.

The reason for maintaining these as two separate lists instead of a single dictionary correlating the DNIs with their positions is that some passengers may not have an available seat in a given bus. This separation allows for a clear distinction between all potential passengers and the subset of passengers who are assigned seats. In cases where there are more passengers than available seats, this approach ensures that all DNIs are accounted for in the `passengers` list, while only those who have been allocated seats are represented in the `solution` list.

3.4.3.2.- Tables and Their Attributes

2. **Table: Users**

- **dni:** The primary key of the table, a unique identifier for each user being this equal to their national ID (DNI). Example: `08734345R`
- **username:** A text value representing the username of the user. Example: `johndoe89`
- **address:** A text value representing the address of the user. Example: `Avenida de Asturias, 72, Villablino, León`
- **allergies:** A JSON array representing the allergies of the user. Example: `["S":"onion", "S":"carrots"]`
- **cant_sit_with:** A JSON array representing the users the person cannot sit with. Example: `["S":"1", "S":"2"]`
- **disabilities:** A JSON array representing the disabilities of the user. Example: `["S":"blindness"]`

- **events:** A JSON array representing the events related to the user. Each event includes a date, event type, and observations. Example: `["M":"date":"S":"2024-05-05 22:46:04", "event_type":"S":"Arrived", "observations":"S":"-"]`
- **illnesses:** A JSON array representing the illnesses of the user. Example: `[ "S" : "diabetes" , "S" : "heart problems" ]`
- **last_name:** A text value representing the last name of the user. Example: `Doe`
- **medication:** A JSON array representing the medication of the user. Example: `["S":"ventolin"]`
- **name:** A text value representing the name of the user. Example: `Johnes`
- **others:** A JSON array representing other relevant information about the user. Example: `["S":"Does nor like to watch television"]`
- **password:** A text value representing the encrypted password of the user. Example: `JDJhJDEyJHJlY3VFeVc`
- **phone_number:** A text value representing the phone number of the user. Example: `591459523`
- **profile_picture:** A text value representing the filename of the profile picture of the user. Example: `placeholder.png`

3.4.3.3.- Tables and Their Attributes

3. **Table: BusDriver**

- **id_driver:** The primary key of the table, a unique identifier (GUID). Example: `2345425E`
- **name:** Text value storing the driver's first name. Example: `Angel`
- **last_name:** Text value storing the driver's last name. Example: `Ferrnández Sal`
- **phone_number:** Text value storing the driver's phone number. Example: `3455346`
- **driver_license:** Text value storing the driver's license number. Example: `234567t`
- **username:** Text value storing the username. Example: `Gelo`
- **password:** Text value representing the encrypted password of the driver. Example: `JDJhJDEyJHJlY3VFeVc`
- **address:** A text value representing the address of the driver. Example: `dsgsdgs`

- **profile_picture:** A text value representing the filename of the profile picture of the driver. This field is always null but is left for future extension. Example: `Null`
- **role:** A JSON array representing the roles assigned to the driver. Example: `["driver"]`

4. **Table: Routes**

- **id_route:** The primary key of the table, a unique identifier (GUID). Example: `a2de20b5-e1bf-40e1-a24a-b4459f64ba8b`
- **name:** Text value storing the name of the route. Example: `estimated route for January`
- **description:** Text value storing the description of the route. Example: `route January`
- **distance:** Numerical value representing the distance of the route in meters. Example: `23918.755`
- **time:** Numerical value representing the estimated time to complete the route in seconds. Example: `1591.1`
- **driver:** Text value storing the name of the driver assigned to the route. Example: `Gelo`
- **users:** JSON array listing the users (passengers) assigned to the route. Example: `["Sara", "Pedro", "Rafael", "Teresa"]`

5. **Table: Staff**

- **dni:** The primary key of the table, a unique identifier being this equal to the national ID of the user (DNI) (string). Example: `09817107V`
- **name:** Text value storing the staff member's first name. Example: `Silvia`
- **last_name:** Text value storing the staff member's last name. Example: `Rodríguez Bares`
- **phone_number:** Text value storing the staff member's phone number. Example: `654356789`
- **address:** Text value storing the staff member's address. Example: `Avenida Constantino Gancedo`

- **roles:** List value storing the roles assigned to the staff member. Example: `["receptionist", "assistant"]`
- **username:** Text value storing the username for login purposes. Example: `silvia`
- **admin:** Boolean value indicating if the staff member has administrative privileges. Example: `False`
- **password:** Text value storing the encrypted password for login. Example: `JDJhJDEyJHJlY3VFeVc`
- **profile_picture:** Text value storing the file name of the profile picture. Currently, this is empty for future use. Example: `Empty`

3.4.4.- Using typing in python

Python is dynamically typed, which means that the type of a variable is interpreted at runtime, and variables are not bound to a specific type. This flexibility can make Python quick and easy to write, but it can also lead to issues when maintaining or debugging code, particularly in large projects. To address this, Python introduced the `typing` [2] module, which provides a way to specify type hints.

Type hints are annotations that specify the expected type of variables, function parameters, and return values. Although Python remains dynamically typed and these annotations are not enforced at runtime, they serve as valuable documentation and allow for the use of static type checkers such as mypy. By using type hints, developers can catch type errors early in the development process, leading to more robust and reliable code.

3.4.4.1.- Benefits of Using typing

- **Improved Code Documentation:** Type hints act as documentation for the expected types of variables and function parameters. This makes the code more readable and easier to understand, as other developers can quickly see what types are expected without having to read through the entire implementation.
- **Enhanced Code Quality:** Static type checkers can analyze the code and identify potential type-related errors before runtime. This helps to catch bugs early, reducing the likelihood of runtime errors and improving the overall quality of the code.
- **Better IDE Support:** Modern Integrated Development Environments (IDEs) leverage type hints to provide better code completion, navigation, and error checking. This makes the development process smoother and more efficient.

- **Facilitates Refactoring:** When refactoring code, type hints provide a clear contract of what types are expected, making it easier to modify and extend the code without introducing errors. This is particularly useful in large code-bases where understanding all the dependencies can be challenging.

- **Support for Complex Data Structures:** The typing module provides constructs to define complex data structures such as lists, dictionaries, and custom generic types. This allows developers to specify and enforce the types of elements within these structures, ensuring consistency throughout the code.

3.4.5.- Using typescript

TypeScript is a strongly typed programming language developed and maintained by Microsoft. It is a superset of JavaScript, which means that any valid JavaScript code is also valid TypeScript code. TypeScript extends JavaScript by adding static types, which can help developers catch errors early and write more robust, maintainable code.

3.4.5.1.- Benefits of Using TypeScript

- **Static Typing:** One of the main advantages of TypeScript[15] is its static typing. This feature allows developers to define the types of variables, function parameters, and return values, which helps in catching type-related errors during the development phase rather than at runtime. Static typing enhances code quality and reduces the likelihood of bugs.

- **Improved IDE Support:** TypeScript is supported by most modern Integrated Development Environments (IDEs) such as Visual Studio Code, WebStorm, and Sublime Text. These IDEs provide features like autocompletion, type checking, and navigation, which can significantly boost productivity and reduce errors.

- **Enhanced Code Readability:** TypeScript's explicit type annotations improve code readability, making it easier for developers to understand the codebase. This is particularly beneficial in large projects or when working in teams, as it clarifies the intended use of variables and functions.

- **Refactoring Support:** With TypeScript, refactoring code becomes safer and more efficient. The type system ensures that changes in one part of the codebase are propagated correctly throughout the project, reducing the risk of introducing bugs during refactoring.

- **Modern JavaScript Features:** TypeScript supports all modern JavaScript features, including ECMAScript 6 (ES6) and beyond. This means developers can use the latest JavaScript syntax and features, even in environments that do not yet support them natively, thanks to TypeScript's transpilation process.
- **Large Community and Ecosystem:** As a product of Microsoft, TypeScript has a large and active community. There are many libraries, tools, and frameworks built with or for TypeScript, making it easier to find resources and get support.
- **Integration with Existing JavaScript Code:** TypeScript integrates seamlessly with existing JavaScript code. This allows developers to gradually adopt TypeScript in their projects without needing to rewrite the entire codebase.
- **Optional Typing:** TypeScript allows optional typing, which means developers can gradually introduce type annotations to their codebase. This flexibility makes it easier to adopt TypeScript incrementally.

3.4.6.- Security Measures

3.4.6.1.- Cookie Control

In this system, cookies are used to ensure that users are logged in to access various pages. After a user logs in, they must maintain their session to access specific routes such as `/admin`, `/user`, or `/driver`. This is managed by setting and checking cookies on both the client and server sides.

The `Driver` component, for example, includes the following relevant code for handling cookies[6][7] :

When the `Driver` component is rendered, the `getServerSideProps` function is executed on the server side to fetch necessary data. Here's how it uses cookies:

1. **Parsing Cookies:** The function `parseCookies(context)` is used to parse cookies from the server-side request context. This helps in extracting the `dni` and `access_token` values which are essential for user authentication.

```
export async function getServerSideProps
    (context: GetServerSidePropsContext) {
    // parse the cookies in the context of the server-side request
    const cookies = parseCookies(context);
```

```
// Extract username and access token from cookies
const dni = cookies.dni;
const access_token = cookies.access_token;
```

2. **Fetching Data:** If both `access_token` and `dni` are present, the system makes multiple `fetch` requests to retrieve data about buses, routes, and users. These requests include appropriate headers to handle CORS and content type.

```
if (access_token && dni) {
    let buses = await fetch(backendUrl + '/buses', {
        method: 'GET',
        headers: new Headers({
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*',
        })
    }).then(data => data.json())
        .catch(err => console.log(err));

    let routes = await fetch(backendUrl + '/route/all', {
        method: 'GET',
        headers: new Headers({
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*',
        })
    }).then(data => data.json())
        .catch(err => console.log(err));

    let users = await fetch(backendUrl + '/user/all', {
        method: 'GET',
        headers: new Headers({
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*',
        })
    }).then(data => data.json())
        .catch(err => console.log(err));

    return {
```

```
            props: {
                buses: buses,
                routes: routes,
                users: users
            }
        };
    }
```

3. **Handling Unauthenticated Access:** If the `access_token` or `dni` is missing, it means the user is not logged in. The system then clears any existing cookies and redirects the user to the login page.

```
    setCookie("dni", "", { maxAge: 0, path: "/" });
    setCookie("role", "", { maxAge: 0, path: "/" });
    setCookie("access_token", "", { maxAge: 0, path: "/" });
    return {
        redirect: {
            permanent: false,
            destination: "/",
        },
        props: {}
    }
}
```

By using this method, the system ensures secure and proper handling of user sessions through cookies, providing a robust mechanism for controlling access to different parts of the application based on authentication status.

3.4.6.2.- API Access Control

In the AFADLA backend, an authentication system was initiated to ensure that each user type has appropriate access control policies. This system is crucial for maintaining security and defining permissions for various user roles. For example, only administrators are allowed to create new users.

Here is an example of the implementation for driver insertion:

```
@driver_routes.route("/driver", methods=["POST"])
@admin_required
def insert_driver():
    data = request.get_json()

    if not data:
        return jsonify({"error": "Driver data is required"}), 400

    data["password"] = DDBC.hash_password(data["password"])

    driver = BusDriver.from_json(data)

    try:
        dbc.insert_driver(driver)
        return jsonify({"message": "Driver created successfully"}), 200
    except EntityAlreadyExistsError as e:
        return jsonify({"error": str(e)}), 409
    except DriverInsertionError as e:
        return jsonify({"error": str(e)}), 500
```

In the example above, the `insert_driver` function is intended to be protected by an `admin_required` decorator to restrict access to administrators only. The implementation of this decorator is shown below:

```
def admin_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        token = request.headers.get("Authorization")
        staff = get_staff_from_token(token)[0]

        if staff is None or not staff.admin:
            return jsonify({"error": "Access forbidden: admin privileges required"}

        return f(*args, **kwargs)

    return decorated_function
```

Additionally, the `admin_or_user_required` decorator ensures that either an admin or

the user themselves can access certain resources:

```
def admin_or_user_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        return f(*args, **kwargs)

    return decorated_function
```

Currently, the implementation is in its initial stages, and future versions of the web application will complete the functionality. These future updates will include creating access control policies for day center users and drivers to prevent unauthorized requests.

Future Enhancements:

- **User Policies:** Define specific policies for day center users to restrict access to relevant parts of the system.
- **Driver Policies:** Implement detailed permissions for drivers to ensure they can only access and modify their relevant data.
- **Comprehensive Access Control:** Ensure that all endpoints have appropriate access controls to prevent unauthorized access and operations.

These enhancements will significantly improve the security and functionality of the AFADLA system by ensuring that only authorized users can perform specific actions based on their roles.

3.4.6.3.- Password Creation

The password creation process involves several security measures to ensure that user passwords are both strong and secure. The frontend application performs initial password validation to ensure that passwords meet certain complexity requirements. Specifically, the password must be at least 8 characters long, contain at least one uppercase letter, one lowercase letter, one number, and one special character. This helps mitigate the risk of weak passwords that are easy to guess or crack.

```
if (e.target.value.length < 8) {
```

```
    errors.push("Password must be at least 8 characters long");
}

if (!e.target.value.match(/[A-Z]/)) {
    errors.push("Password must contain at least one uppercase letter");
}

if (!e.target.value.match(/[a-z]/)) {
    errors.push("Password must contain at least one lowercase letter");
}

if (!e.target.value.match(/[0-9]/)) {
    errors.push("Password must contain at least one number");
}

if (!e.target.value.match(/[^A-Za-z0-9]/)) {
    errors.push("Password must contain at least one special character");
}
```

The validation is performed on the client-side to provide immediate feedback to the user. However, it is important to note that client-side validation alone is not sufficient for security purposes. Therefore, the backend also validates the password and hashes it before storing it in the database.

3.4.6.4.- Password Verification

When a user attempts to log in, the backend verifies the provided password against the stored hashed password. The password verification process involves decoding the stored hashed password from its Base64 representation and using the bcrypt[14] library to compare the hashed password with the plain text password provided by the user. The use of bcrypt ensures that the password verification process is secure and resistant to common attacks such as brute-force and rainbow table attacks.

```
def check_password(password: str, hashed_password_base64: str) -> bool:
    hashed_password = base64.urlsafe_b64decode(hashed_password_base64)
    return bcrypt.checkpw(password.encode("utf-8"), hashed_password)
```

3.4.6.5.- Password Reset

For this initial version, considering that most users lack email accounts or are elderly individuals who tend to forget their passwords, it will be easily changed by a user with administrator privileges. Additionally, users can change their passwords from their profile if they wish. If a user forgets their password, they must contact the center for an administrator to change it and provide a new one.

The password reset functionality is implemented through two main methods:

- **User-Initiated Password Change:** Users have the option to change their password through their profile page. The user must provide their DNI and the new password they wish to set. The system verifies the provided DNI and updates the password accordingly. This operation is handled by an endpoint in the user routes, which processes the request, validates the data, and updates the password in the database.
- **Administrator-Initiated Password Change:** Administrators have the ability to change the passwords of any user. This is particularly useful when a user forgets their password and needs assistance. The administrator provides the DNI of the user and the new password. The system validates the administrator's privileges, processes the request, and updates the password in the database. This operation is handled by an endpoint in the staff routes, ensuring that only users with administrative privileges can perform this action.

3.4.6.6.- Security in Uploading Images

The image upload functionality includes several security measures to prevent common vulnerabilities associated with file uploads. The backend verifies the existence of the uploaded file and checks if the file is of an allowed type. The file is then saved with a unique filename to prevent conflicts and ensure that the file can be easily identified.

```
if "file" not in request.files:
    return jsonify({"error": "No file submitted"}), 412

file = request.files["file"]

if file.filename == "":  # or file.content_length == 0:
    return jsonify({"error": "No file submitted"}), 412
```

```
if file and allowed_file(file.filename):
    filename = secure_filename(file.filename)
    new_filename = (
        filename.rsplit(".", 1)[0].lower()
        + "_"
        + file_hash_stream(file.stream)
        + "_"
        + str(time.mktime(datetime.now().timetuple()))
        + "."
        + filename.rsplit(".", 1)[1].lower()
    )

    filepath = os.path.join(
        current_app.config["UPLOAD_FOLDER"], new_filename
    )

    if dbc.photo_exists(new_filename):
        return jsonify({"error": "File already exists"}), 409

    if not os.path.exists(filepath):
        file.stream.seek(0)
        file.save(filepath)

    dbc.update_photo(user, new_filename)
```

Additionally, the server response headers are set to prevent caching of the image to ensure that the latest image is always displayed and to prevent unauthorized access to cached images. The use of a unique filename and checking for existing filenames helps prevent file overwriting and ensures that each image is uniquely identifiable.

The combination of frontend validation, secure password hashing and storage on the backend, and robust file upload handling ensures that the application adheres to best practices for security. These measures help protect user data and maintain the integrity and confidentiality of user accounts and uploaded files.

3.4.7.- Uploading Images

It has been implemented a feature that enables user profile images through an API endpoint. This route allows for both the retrieval and updating of a user's profile picture, using their DNI and username as primary keys or identifying information.

3.4.7.1.- Storage

User profile images are stored in an AWS S3 bucket, since they are designed to store and retrieve any amount of data, making S3 storage an optimal solution for storing user profile images.

3.4.7.2.- Automatically deleting old photos

Data compliance laws in the EU, notably the General Data Protection Regulation (GDPR), place strict requirements on the processing of personal data of EU citizens. When it comes to user profile photos, the following reasons explain why we cannot automatically delete past user photos:

Right to Access: GDPR gives individuals the right to access their personal data held by organizations. Automatically deleting past user photos could infringe on this right, as users might request access to their previously uploaded images.

Data Retention and Erasure: While GDPR also provides the "right to be forgotten", meaning users can request their data to be deleted, it doesn't mean that companies should automatically erase data without a specific request. In some cases, the law requires businesses to retain certain data for specific periods due to reasons like legal disputes, audits, or other regulatory requirements.

Record-keeping: For transparency and accountability, GDPR requires organizations to maintain records of their data processing activities. Deleting past user photos automatically could interfere with these record-keeping obligations, especially if the action lacks a documented and lawful reason.

Consent and Purpose Limitation: GDPR mandates that data should only be processed for the specific purpose for which it was collected and only if there's a valid lawful basis (e.g., consent, contractual necessity). If users provided their photos for a specific purpose, automatically deleting them without an associated reason or without notifying the user might not align with the original purpose of data collection.

Potential Legal Claims: In some situations, past data, including photos, might be necessary to establish, exercise, or defend against legal claims. Automatically deleting this information could jeopardize the organization's legal position.

In summary, while GDPR emphasizes data minimization and user rights, it also requires careful consideration of data retention based on both the regulation's principles and other potential legal obligations. Automating deletion processes without these considerations can lead to non-compliance.

3.4.7.3.- Database Reference

Instead of storing the image directly, we store the S3 file path in the database, which points to the respective image.

3.4.7.4.- Image Retrieval

- On receiving a GET request, our backend checks for the image in a local cache.
- If not found locally, it downloads the image from S3, stores it in the local cache, and then serves it to the requester.
- Subsequent requests for the same image will be served from the local cache.

3.4.8.- Unique Image Naming

- To ensure uniqueness and mitigate risks of naming collisions, we use the hash of the initial 65536 bytes of the image.
- This method limits RAM usage and optimizes processing.
- The hashed value is appended to the image filename, and this convention is followed both in the S3 bucket and the database.

3.4.8.1.- Security Considerations

A filename cleansing function is in place to prevent potential code injections, bolstering our system's security.

3.4.9.- Deployment

When transitioning a web application from development to production, the choice of server architecture and configurations becomes crucial for performance and reliability. In devel-

opment environments, lightweight web servers like Flask's built-in server are often used for their simplicity and ease of use. However, for deployment in a production environment, more robust solutions like Gunicorn are preferred.

3.4.9.1.- Why Use Gunicorn for Production

- Production Readiness: Flask's built-in server is not designed to handle production workloads. It is single-threaded and lacks the advanced features required for high availability and concurrency. Gunicorn, on the other hand, is a pre-fork worker model server designed for production use. It can manage multiple worker processes to handle concurrent requests efficiently.
- Concurrency Handling: Gunicorn supports multiple workers and can handle many requests simultaneously, improving the application's ability to serve multiple users concurrently without significant performance degradation.
- Robustness and Reliability: Gunicorn includes many production-grade features like graceful shutdowns, timeouts, and configurable logging. These features enhance the stability and maintainability of the application in a live environment.
- Compatibility and Integration: Gunicorn is compatible with various web frameworks and can be integrated with other tools and services (like Nginx) to build a more resilient and scalable web architecture.

3.4.9.2.- Memory Usage and Worker Calculation

Given that the server is a tg4.small instance with 1GB of RAM, it's critical to optimize the number of Gunicorn workers to balance concurrency and resource constraints. Running too many workers can exhaust the available memory, leading to degraded performance or even system crashes.

Calculating the Number of Workers Gunicorn's documentation provides a rule of thumb for calculating the number of workers:

$$\text{Number of Workers} = 2 \times \text{Number of CPU Cores} + 1 \tag{3.1}$$

However, this calculation assumes sufficient memory to support all workers. Given the constraint of 1GB RAM and the observation that 6 workers consume nearly 100% of the

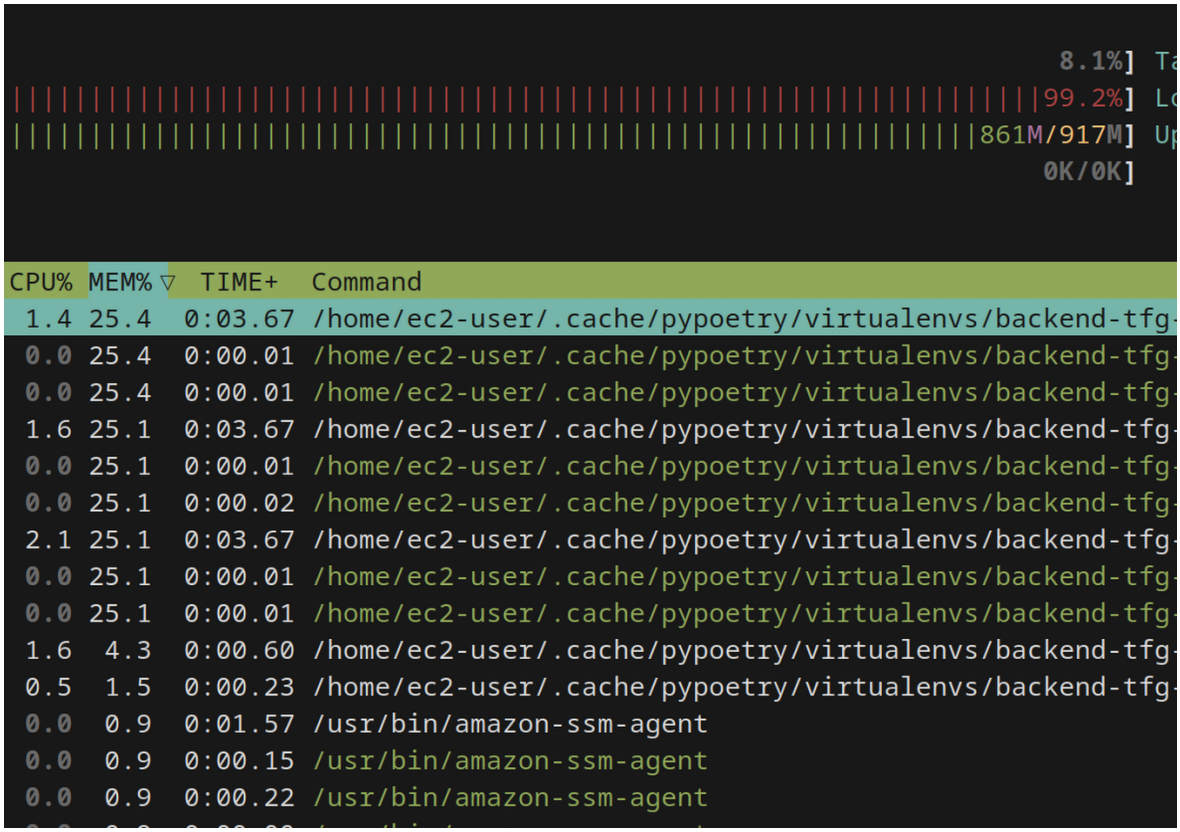

Figure 3.11: Resource monitoring when running 6 workers of Gunicorn

RAM, we need a more memory-focused approach.

3.4.9.3.- Memory Considerations

Baseline Memory Usage Determine the memory footprint of each Gunicorn worker. If 4 workers consume 857MB of RAM, then each worker approximately uses:

$$\frac{857 \text{ MB}}{4} \approx 214 \text{ MB per worker} \tag{3.2}$$

Memory Overhead Allocate some buffer for system processes and other applications running on the server. Assume 200MB for system overhead, leaving 800MB for Gunicorn workers.

Optimal Worker Count Divide the available memory by the memory usage per worker to find the optimal number of workers:

$$\text{Optimal Workers} = \frac{800 \text{ MB}}{214 \text{ MB per worker}} \approx 3.7 \tag{3.3}$$

Since we can't have a fraction of a worker, we round down to 3 workers.

3.4.9.4.- Handling Concurrent Users

With an estimated maximum of 20 concurrent users, the server needs to ensure these users are efficiently managed. Each worker can handle multiple requests, but if requests are long-running or resource-intensive, increasing the number of workers might be necessary despite the memory constraints.

3.5.- Workarounds

3.5.0.1.- Caching and URL Consistency

A challenge in the frontend is the static nature of the image URL. Even after an image update, the URL remains unchanged. This leads to the image being cached and not reflecting the updated version, even after a page reload.

To address this, specific headers are attached to the GET response to instruct browsers not to cache the image:

```
response.headers["Pragma"] = "no-cache"
response.headers["Expires"] = "Fri, 30 Oct 1998 14:19:41 GMT"
response.headers["Cache-Control"] = "no-cache, must-revalidate"
```

On the front end, the image URL is appended with '#${new Date().getTime()}'. This modification ensures the URL is distinct every time, forcing the browser to fetch the most recent image version instead of relying on cached data.

3.5.1.- Bus User Sorting Algorithm

3.5.1.1.- Algorithm Implementation

The algorithm uses a backtracking approach to find the optimal seating arrangement for users based on their compatibility and available seats. Here's a clear and concise explanation of the algorithm:

1. **Decision Sequence:** The sequence $\langle x_0, x_1, \ldots, x_{P-1} \rangle$ represents the seat number each

passenger will occupy, where x_0 is the seat number for passenger 0, x_1 is for passenger 1, and so on.

2. Objective Function and Constraints:

- Objective: Maximize the number of seated users N_k where $x[k] \neq -1$ for $0 \leq k < P$.
- Explicit constraints: $\forall k\ x_k \in 0 \ldots M-1$ for $0 \leq k < P$.
- Implicit constraints: Based on the user compatibility matrix U and the number of occupied seats N_k where $x_k \neq -1$ for $0 \leq k < P \leq M$.

3. Algorithm Steps:

1. Prepare Traversal Level: Initialize $x[k] = -1$.
2. Check if a sibling exists at level k: $x[k] < M \times M$.
3. Move to the next sibling level: $x[k] = x[k] + 1$.
4. If $x[k]$ is valid, check if it satisfies all constraints using the function `can_sit`.
5. If the current configuration is better than the best found so far, update the best configuration.

4. Primary Functions:

- `can_sit`: Determines if a user can sit in a particular seat based on the user adjacency matrix, seat matrix, and current decision array.
- `value`: Calculates the number of users who have been seated based on the current decision array.
- `sit_users`: Recursively attempts to seat users in the optimal arrangement, updating the best configuration found.

3.5.1.2.- Example

Consider a scenario with 8 users and a 4x4 seating arrangement. The incompatibilities are as follows:

- User 0 cannot sit next to User 1 and User 2.
- User 1 cannot sit next to any user.
- User 2 cannot sit next to User 0 and User 1.
- User 3 cannot sit next to User 1, User 6, and User 7.
- User 4 cannot sit next to User 1, User 3, and User 5.
- User 5 cannot sit next to User 1 and User 6.
- User 6 cannot sit next to User 1, User 0, User 3, and User 5.
- User 7 cannot sit next to User 1.

The user compatibility matrix U and the seat matrix S are defined as follows. Note that users with a 0 in the matrix cannot sit together.

$$U = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$S = \begin{bmatrix} -1 & 0 & -1 & -1 \\ 0 & 0 & -1 & -1 \\ 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

In the matrix S:

- Positions with -1 correspond to aisles or the driver's seat.
- Positions with 0 are free seats that users can occupy.

Given the input matrices U and S, the algorithm proceeds as follows:

Initialization

- *x_best* is initialized to store the best seating arrangement.
- *v_best* is initialized to 0 to store the value of the best arrangement.

Backtracking Search

- The algorithm starts with user 0 and tries to seat them in all possible seats (from 0 to 15, as there are 16 seats in total).
- For each seat, it checks if it is valid using `can_sit`.

Seating Users

- For user 0, it finds that seat 1 is a valid position.
- For user 1, it finds that seat 4 is valid, given the current configuration.
- For user 2, it finds that seat 8 is valid.
- This process continues until all users are seated, ensuring at each step that the constraints are satisfied.

Updating Best Configuration

- Whenever a valid seating configuration is found for all users, it updates *x_best* if the current configuration has more users seated than the previous best.

Final Solution After the backtracking search completes, the algorithm determines the optimal seating arrangement as follows: User 0 sits at position 1, user 1 sits at position 4, user 2 sits at position 8, user 3 sits at position 5, user 4 sits at position 9, user 5 sits at position 12, user 6 sits at position 14 and finally user 7 sits at position 13.

$$Solution = \begin{bmatrix} -1 & 0 & -1 & -1 \\ 1 & 3 & -1 & -1 \\ 2 & 4 & -1 & -1 \\ 5 & 7 & 6 & -1 \end{bmatrix}$$

This arrangement ensures that all constraints are satisfied:

- No two incompatible users are seated next to each other.
- The maximum number of users is seated.

3.5.1.3.- Library in C

In this section, it will be explained the architectural design decisions required to integrate the algorithm responsible for the most efficient seat allocation with the rest of the program (backend).

The algorithm has been implemented in C to have greater control over low-level operations and due to its faster performance compared to Python when performing operations on various matrices.

The primary reason for choosing C over numpy is to achieve finer control over memory management and to maximize performance. Although numpy leverages C under the hood, using C directly allows for more optimized and precise handling of computational tasks, which is crucial for the performance-intensive operations involved in the seat allocation algorithm.

This development in C is used as a library in this project. The library is compiled into a `.so` file using a makefile. This `.so` file can be utilized from Python as if it were a regular library. However, it is necessary to define the input and output types of this library/function. Therefore, we need a 'wrapper' that acts as an interface between the rest of the code in Python and the library in C.

3.5.1.4.- Wrapper in Python

To integrate the C library with Python, a wrapper is created using the `ctypes` library. This wrapper allows Python to call the functions defined in the C library by specifying the path to the shared object file (`.so`) and defining the function signatures.

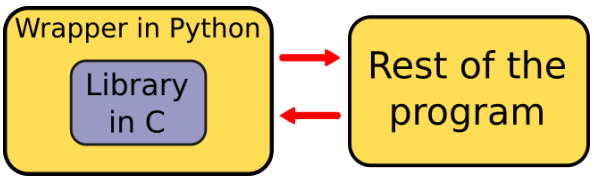

Figure 3.12: Representation of the integration of the library with the program.

Explanation of the Wrapper

The wrapper works as follows:

1.Loading the Shared Object File:

```
so_file = "../bin/bus.so"
lib = CDLL(so_file)
```

The `CDLL` function from the `ctypes` library is used to load the shared object file `bus.so`. This file contains the compiled C code and can be called as if it were a Python library.

2. Defining Function Signatures:

```
lib.solution.argtypes = [
    ctypes.POINTER(ctypes.POINTER(ctypes.c_int)),  # U argument type
    ctypes.POINTER(ctypes.POINTER(ctypes.c_int)),  # S argument type
    ctypes.c_int,  # n_users argument type
    ctypes.c_int  # n_seats argument type
]
lib.solution.restype = ctypes.POINTER(ctypes.c_int)  # Return type
```

The `argtypes` attribute specifies the argument types for the `solution` function in the C library. It expects two 2D arrays (`U` and `S`), and two integers (`n_users` and `n_seats`). The

`restype` attribute specifies the return type of the function, which is a pointer to an array of integers.

3. Calling the C Function from Python:

```
def bus_sitter(U: list[list[int]], A: list[list[int]]) -> list[int]:
    sol = lib.solution(
        (ctypes.POINTER(ctypes.c_int) * len(U))(
            *[ctypes.cast((ctypes.c_int * len(row))(*row),
             ctypes.POINTER(ctypes.c_int))
              for row in U]),
        (ctypes.POINTER(ctypes.c_int) * len(A))(
            *[ctypes.cast((ctypes.c_int * len(row))(*row),
            ctypes.POINTER(ctypes.c_int))
              for row in A]),
        ctypes.c_int(len(U)),
        ctypes.c_int(len(A))
    )

    return [sol[i] for i in range(1, len(U))]
```

The `bus_sitter` function prepares the input data by converting the Python lists into the appropriate C types using `ctypes`. The function then calls `lib.solution` with these converted arguments and processes the result, converting it back into a Python list.

Integration with the Rest of the Program

To facilitate the use of this wrapper, the `Bus` class is defined to manage the seating of passengers and interface with the C library. The `Bus` class handles:

- Initialization and validation of passengers.
- Solving the seating arrangement using the `solve` method, which calls the C library function.
- Conversion to and from JSON and DynamoDB formats for easy storage and retrieval.
- Equality and hashing operations to allow comparisons and use in data structures.

This integration allows the high-performance C algorithm to be used within a Python application, combining the speed of C with the ease of use and flexibility of Python.

3.5.2.- Technologies and Tools

3.5.2.1.- pnpm

When it comes to package managers for JavaScript and Node.js, pnpm[12] (Performant npm) offers several distinct advantages over npm (Node Package Manager) that make it a superior choice for this project. pnpm excels in performance and efficiency by using a unique method of storing packages in a content-addressable storage and creating hard links to the project's $\texttt{node}_{m}odulesdirectory$.

In the context of using Next.js, pnpm enhances the development experience by significantly reducing the time required for installing and updating dependencies, which is crucial for rapid iteration in modern web development. pnpm also ensures disk space efficiency and consistency across different environments, which is essential for maintaining the reliability of Next.js applications.

For developers, pnpm provides user-friendly commands that are compatible with npm, enhanced commands for workspace and monorepo management, and comprehensive documentation with a growing community. These features collectively improve developer productivity and make pnpm an excellent choice for modern JavaScript and Node.js projects, including those built with Next.js, ensuring an optimized, secure, and scalable development workflow.

Furthermore, pnpm's support for seamless integration with various CI/CD pipelines ensures that the deployment process remains efficient and consistent, which is vital for Next.js applications that often require frequent updates and rapid deployments. In summary, pnpm's advantages in performance, efficiency, reliability, and security make it a better choice than npm for this project's needs, especially when leveraging the capabilities of Next.js.

3.5.2.2.- Poetry

For Python projects, Poetry[13] is a powerful dependency management and packaging tool that offers several benefits over traditional tools like pip and setuptools. One of the primary reasons for choosing Poetry for this project is to avoid dealing with `requirements.txt` files. Poetry simplifies the process of managing dependencies, virtual environments, and package publishing by using a single `pyproject.toml` file to specify project dependencies and configurations. This approach makes the project structure cleaner and more maintain-

able.

Poetry's unified dependency management is particularly beneficial for syncing project dependencies and versions across different development environments, such as a laptop, a desktop PC, and the server where the backend will be hosted. By using `poetry.lock`, Poetry ensures that the exact versions of dependencies are consistently installed across all environments, reducing the risk of discrepancies and version conflicts.

Additionally, Poetry automatically creates and manages virtual environments, ensuring that dependencies are isolated and do not conflict with other projects. This helps maintain a clean development environment and reduces the risk of dependency issues. Poetry's advanced and efficient dependency resolver is capable of handling complex dependency trees, further minimizing version conflicts and ensuring a smooth development workflow.

3.5.2.3.- OSMnx

For creating the route graph and obtaining the route data, we used OSMnx[11]. OSMnx is a Python package that allows for the easy download, analysis, and visualization of street networks from OpenStreetMap[8]. It is particularly useful for geographic data science, urban planning, and transportation engineering. In our project, OSMnx was used to geocode addresses, calculate routes between multiple points, and visualize these routes on a map.

3.6.- Routing Algorithm

3.6.1.- Introduction

Route optimization is a critical aspect of logistics and transportation, often addressed through the Traveling Salesman Problem (TSP). The TSP seeks the shortest possible route that visits a given set of nodes exactly once and returns to the origin node. While exact solutions are computationally infeasible for large instances due to their NP-hard nature, various heuristic and approximation algorithms provide practical solutions. This section examines the Minimum Spanning Tree (MST) based approximation, TSP approximation algorithm provided by NetworkX, and the Greedy TSP algorithm, discussing their respective advantages and disadvantages. We then present our approach, which initially targets the furthest node from the origin to mitigate inefficient backtracking.

3.6.2.- Minimum Spanning Tree (MST) Approximation

3.6.2.1.- Description

The MST-based heuristic, often referred to as the Christofides algorithm, constructs a minimum spanning tree of the graph, which connects all nodes with the minimum possible total edge weight. The MST is then augmented with a minimal set of additional edges to form an Eulerian circuit, which can be transformed into a Hamiltonian circuit approximating the TSP solution.

3.6.2.2.- Advantages

- Simplicity: The MST can be computed efficiently using algorithms like Prim's or Kruskal's, making it straightforward to implement.
- Bounded Performance: The Christofides algorithm guarantees a solution within 1.5 times the optimal TSP route length, providing a reliable upper bound.

3.6.2.3.- Disadvantages

- Inefficiency for Sparse Graphs: The MST-based approach may add redundant edges, especially in sparse graphs, leading to suboptimal routes.
- No Guarantee of Feasibility: The transformation from an Eulerian circuit to a Hamiltonian circuit can sometimes result in infeasible paths due to overlapping edges.

3.6.3.- TSP Approximation Algorithm

3.6.3.1.- Description

The TSP approximation algorithm implemented in NetworkX utilizes various heuristic methods, such as nearest neighbor, genetic algorithms, or simulated annealing, to find a near-optimal solution. These heuristics iteratively improve the route by exploring permutations and adjustments.

3.6.3.2.- Advantages

- Flexibility: Heuristic methods can adapt to different types of graphs and constraints, offering versatility in solution approaches.

- Potential for High-Quality Solutions: With sufficient iterations, these algorithms can approach near-optimal solutions, especially when combined with techniques like local search.

3.6.3.3.- Disadvantages

- Computational Overhead: Heuristic algorithms, particularly those requiring numerous iterations or complex operations, can be computationally expensive, consuming significant time and memory.

- No Deterministic Guarantees: Unlike MST-based approaches, heuristics do not provide a fixed approximation ratio, leading to variability in solution quality.

3.6.4.- Greedy TSP Algorithm

3.6.4.1.- Description

Our implemented Greedy TSP algorithm prioritizes selecting the next node based on a specific criterion, such as the nearest neighbor. Our approach begins by identifying the furthest node from the origin, aiming to reduce back-and-forth travel. The algorithm iteratively adds the closest unvisited node to the current path until all nodes are visited, finally returning to the origin.

3.6.4.2.- Advantages

- Simplicity and Speed: The greedy approach is computationally efficient, as it involves straightforward distance calculations and immediate decision-making.

- Reduced Redundancy: By starting with the furthest node, the algorithm minimizes the likelihood of repetitive back-and-forth trips, potentially leading to more direct routes.

3.6.4.3.- Disadvantages

- Suboptimal Solutions: The greedy approach does not guarantee optimality, often resulting in longer paths compared to more sophisticated algorithms.

- Locally Optimal Decisions: The algorithm's local decision-making can miss globally optimal solutions, especially in complex graphs with non-uniform node distributions.

3.6.5.- Our Approach

Given the limitations observed with the NetworkX TSP approximation, particularly its extensive computational requirements and memory consumption, we adopted a modified Greedy TSP algorithm. Our approach initiates the route with the furthest node from the origin, aiming to avoid redundant travel. This method has shown promising results in reducing computation time and resource usage while producing reasonably efficient routes. The detailed implementation involves:

1. Initial Selection: Identifying and traveling to the furthest node from the origin.
2. Greedy Iteration: Continuously selecting the nearest unvisited node.
3. Completion: Returning to the origin after visiting all nodes.

3.6.6.- Performance Evaluation of Our Implementation

To evaluate the performance of our Greedy TSP algorithm, we conducted a series of tests to measure the execution time for various stages of the process. The results demonstrate the efficiency of our approach, particularly in terms of time complexity. Below is a detailed breakdown of the performance metrics:

- **Converting addresses to node IDs:** This step involves mapping given addresses to their corresponding node IDs in the graph. As observed in the logs, this process begins at `[2024-07-14 18:18:55.052793]` and completes by `[2024-07-14 18:18:59.676364]`, taking approximately 4.62 seconds. This duration is acceptable given the necessity of accurate geocoding and node identification.
- **Running the Greedy TSP algorithm:** The core TSP computation starts at `[2024-07-14 18:18:59.676388]` and finishes at `[2024-07-14 18:18:59.762905]`, requiring a mere 0.09 seconds. This rapid execution underscores the algorithm's efficiency, significantly reducing computational overhead compared to more complex TSP approximations.
- **Determining the order of user appearance:** Once the TSP path is calculated, the next step is to determine the sequence in which users (represented by their addresses) appear in the optimized route. This operation starts at `[2024-07-14 18:18:59.762928]` and concludes at `[2024-07-14 18:19:03.478283]`, taking approximately 3.72 seconds. This step is crucial for translating node paths back to user-friendly information.

The overall execution from receiving the request to delivering the final ordered user list is remarkably swift. The entire process, as evidenced by the HTTP log entry `127.0.0.1 - - [14/Jul/2024 18:19:03] "POST /route HTTP/1.1" 200 -`, completes in just over 8 seconds. This performance metric highlights the practicality of our implementation for real-time applications, ensuring prompt and efficient route optimization suitable for logistics and transportation services.

Process Stage	Duration (seconds)
Converting addresses to node IDs	4.62
Running Greedy TSP	0.09
Finding order of users	3.72
Total Time	**8.43**

Figure 3.13: Performance Metrics of Greedy TSP Implementation

3.6.7.- Conclusion

The comparison of MST-based, heuristic TSP approximations, and the Greedy TSP algorithm reveals distinct advantages and limitations for each method. While MST-based and heuristic approaches can offer structured and high-quality solutions, their computational demands can be prohibitive for large-scale applications. In contrast, our Greedy TSP algorithm, which prioritizes selecting the next node based on proximity, demonstrates significant computational efficiency and practicality.

This implementation begins by identifying the furthest node from the origin, aiming to reduce unnecessary back-and-forth travel. This method has shown promising results in terms of both speed and resource usage. Performance evaluation indicates that the entire process, from converting addresses to node IDs, running the Greedy TSP, and determining the user order, completes in just over 8 seconds. Specifically, converting addresses to node IDs takes approximately 4.62 seconds, running the Greedy TSP algorithm requires only 0.09 seconds, and determining the order of users takes around 3.72 seconds.

These metrics highlight the practicality of our approach for real-time applications. The swift execution and reduced computational overhead make our Greedy TSP algorithm suitable for logistics and transportation services, where timely and efficient route optimization is crucial. By starting with the furthest node, we effectively minimize redundant travel, thereby achieving a balance between computational efficiency and route quality. This approach offers a viable solution for scenarios requiring prompt and efficient routing decisions, reinforcing the utility and robustness of our Greedy TSP algorithm.

3.7.- Generation of the Map with the Route Overlay

In this application, it is essential to visualize the optimized route on a map to ensure clarity and usability for end users. This section details the method used to generate a map with the route overlay, highlighting the implementation specifics and the configuration settings employed.

The function `get_graph` is responsible for creating a visual representation of the route on the map. The route is represented by a list of node IDs, `self.path`, generated by the Greedy TSP algorithm, which identifies the optimal sequence of nodes to visit. The steps involved in generating the map are as follows:

1. **Filepath Configuration:** The function begins by defining the filepath where the generated map image will be saved. The filepath is constructed using the `UPLOAD_FOLDER` configuration from the application settings, combined with the name of the graph and a `.png` extension.

2. **File Existence Check:** Before generating a new map, the function checks if a file already exists at the specified filepath. If the file exists, the function returns the existing filepath, avoiding redundant computations.

3. **Graph Plotting:** If the file does not exist, the function proceeds to plot the graph route using the `plot_graph_route` function from the OSMnx library. The following configuration settings are used to customize the appearance of the map and route:

```
ox.plot.plot_graph_route(
    maps.G,
    self.path,
    bgcolor='linen',
    route_color='orangered',
    route_linewidth=2,
    route_alpha=0.8,
    orig_dest_size=50,
    node_alpha=0,
    edge_color='k',
    edge_linewidth=1,
    save=True,
    show=False,
    close=False,
```

```
    filepath=filepath,
    dpi=600
)
```

- `maps.G`: The graph object containing the nodes and edges.
- `self.path`: The list of node IDs representing the optimal route.
- `bgcolor='linen'`: Sets the background color of the map.
- `route_color='orangered'`: Specifies the color of the route overlay.
- `route_linewidth=2`: Defines the line width of the route.
- `route_alpha=0.8`: Sets the transparency level of the route.
- `orig_dest_size=50`: Determines the size of the markers for the origin and destination nodes.
- `node_alpha=0`: Sets the transparency of the nodes, making them invisible.
- `edge_color='k'`: Specifies the color of the edges in the graph.
- `edge_linewidth=1`: Defines the line width of the edges.
- `save=True`: Ensures the plot is saved to a file.
- `show=False`: Prevents the plot from being displayed interactively.
- `close=False`: Keeps the plot open for further manipulation if needed.
- `filepath=filepath`: Specifies the filepath where the plot will be saved.
- `dpi=600`: Sets the resolution of the saved image.

4. **Return Filepath:** After successfully generating and saving the map, the function returns the filepath to the caller.

This method ensures that the route is clearly and accurately represented on a high-resolution map, providing users with a visual guide to the optimized path. The use of various customization options allows for a clear distinction between the route and the underlying map features, enhancing the overall user experience.

3.8.- Feasibility of Running the Algorithm in a Lambda Function

In this section, we discuss the feasibility of deploying the backend, which includes our routing algorithm, as a serverless function using AWS Lambda. While serverless architectures offer several benefits, there are significant challenges and limitations that make it less suitable for this specific use case.

3.8.1.- Cost Considerations

AWS Lambda pricing is based on the number of requests and the duration of code execution, measured in milliseconds. For computationally intensive tasks such as our routing algorithm, which involves graph processing and potentially complex pathfinding computations, the execution time can be substantial.

- Execution Time: The greedy TSP algorithm and the necessary shortest path calculations can consume significant computational resources, especially for larger graphs. As execution time increases, so does the cost.
- Request Volume: If the routing service is used frequently, the number of requests can accumulate rapidly, leading to high costs.

While Lambda functions are cost-effective for sporadic and short-running tasks, the sustained and resource-intensive nature of our algorithm could result in prohibitively high operational expenses. Speed and Performance

AWS Lambda imposes limits on the amount of memory and CPU resources available to functions. For memory-intensive tasks such as loading large graphs and performing complex calculations, these limits can be restrictive.

- Resource Limits: The maximum memory allocation for a Lambda function is 10 GB, which may not be sufficient for large city-wide road networks. The CPU power is also limited and shared with other functions, potentially impacting performance.
- Cold Starts: Lambda functions can experience latency due to cold starts, where the environment must be initialized before execution begins. This can add delay, particularly problematic for real-time routing requests where fast response times are critical.

These limitations can lead to suboptimal performance and slower response times, making Lambda a less attractive option for our backend.

3.8.2.- Vendor Lock-In and Migration Concerns

Deploying the backend on AWS Lambda can lead to vendor lock-in, which is a significant consideration for the long-term flexibility and scalability of the project.

- Vendor Lock-In: Lambda functions are closely integrated with AWS services and infrastructure, making it difficult to migrate to another cloud provider without substantial rework.
- Portability: If we decide to move to another cloud provider or an on-premises solution in the future, the tight coupling with AWS-specific services and configurations would complicate the migration process.

By using traditional server-based deployments or containerized solutions, we can maintain greater control over our infrastructure and avoid these pitfalls, ensuring easier portability and flexibility to switch providers if needed.

3.8.3.- Alternative Solutions

Given the constraints and limitations of AWS Lambda for this use case, alternative solutions are more appropriate:

- Dedicated Servers: Deploying the backend on dedicated servers or virtual machines provides greater control over resources, potentially reducing costs for sustained heavy workloads.
- Containerization: Using Docker containers allows for better resource management and portability across different cloud providers, avoiding vendor lock-in.
- Managed Kubernetes: Platforms like Amazon EKS, Google Kubernetes Engine (GKE), or Azure Kubernetes Service (AKS) offer scalable and flexible environments for running our backend, balancing cost, performance, and portability.

These alternatives offer the necessary computational power and flexibility to efficiently run our routing algorithm while providing better control over costs and future-proofing against vendor lock-in.

3.9.- Testing

3.9.1.- Bus User Sorting Algorithm Library

As C does not have a built-in testing suite, we have implemented a specific approach to ensure the correctness and performance of our C library. This process involves creating two versions of the program: a preliminary unoptimized version and a new optimized version that interfaces well with Python. Here are the steps we have followed:

- We developed a preliminary version of the program without any optimizations. This version is verified manually to produce the correct results and remains constant throughout the testing process.
- We created a new version of the program that includes the necessary changes for a good interface with Python, along with various optimizations.
- In the `Makefile`, we added a `test` entry that compiles both the baseline (preliminary) version and the new version, and performs a `diff` of their outputs.
- If the output of the `diff` command is empty, it indicates that the new version produces the same results as the baseline version, confirming its correctness. Any differences in the output indicate an error.

```
test: debug baseline
diff --color=always -y -w -B --suppress-common-lines <(./bin/baseline) <(./bin/main
```

This approach is known as regression testing[1], where we compare two versions of a program to ensure that the new version performs correctly and produces the same results as the original version.

3.9.2.- Performed Tests

To check the exact tests that were performed, check the annex.

4. User manual

4.1.- Introduction

To access the web application, users need to visit the URL `https://www.afadla.com/`.

4.2.- Login

Upon navigating to this address, they will be presented with the login panel, as shown in Figure 4.1.

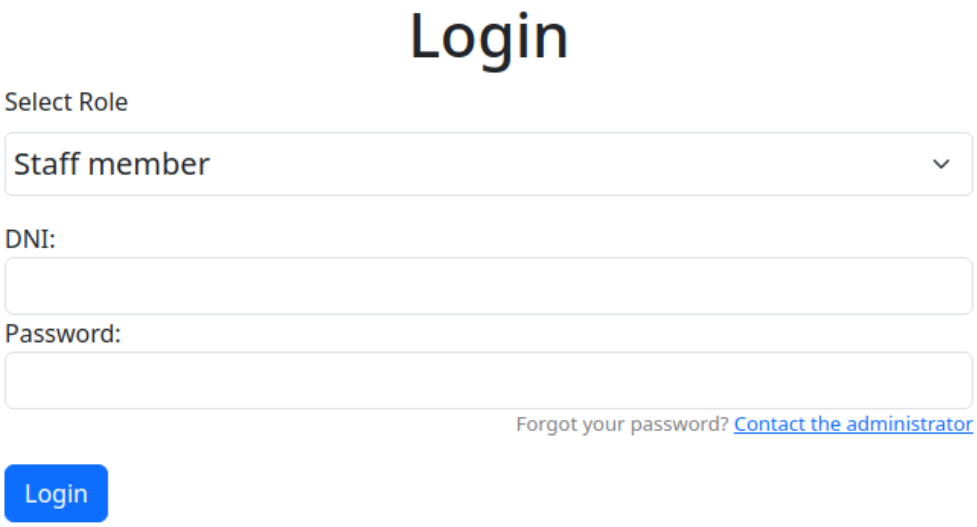

Figure 4.1: Login panel

The panel includes a drop-down menu where users can select one of three distinct roles: Staff member, Driver, or User (with "User" referring to the elderly day center clients), as shown in Figure 4.2. Depending on their role, users must choose the appropriate option from the drop-down menu. Following this, they are required to enter their DNI (National Identification Document) and password.

To successfully log in to the system, the user must have been previously created by an administrator and must know their password.

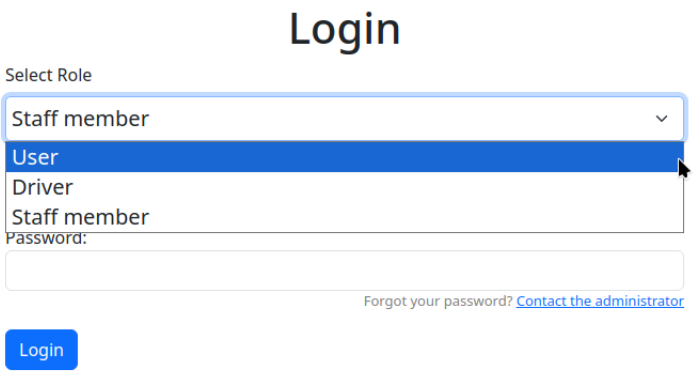

Figure 4.2: Login panel detail

4.3.- Staff View

4.3.1.- Staff View without administrator privileges

Once logged in as a staff member without administrator privileges, the first screen displayed will be as shown in Figure 4.3. In this view, the user can see the various sections available, such as Users, Staff, Drivers, and Routes.

In the image(Figure 4.3), the red highlight shows the current section being viewed, while the blue buttons are disabled, indicating that a staff member without administrator privileges does not have permissions to edit or create new entries. This ensures that only authorized personnel can make changes to the system, maintaining security and data integrity.

In the Users tab (Figure 4.3), the user can view a list of users along with their details such as DNI, name, last name, phone number, address, allergies, illnesses, disabilities, medication, others, and username. The buttons for adding new users, staff, drivers, routes, and buses appear disabled, as indicated by the blue color, showing that the staff member does not have permissions to edit or create new entries. The red highlight shows the section currently being viewed.

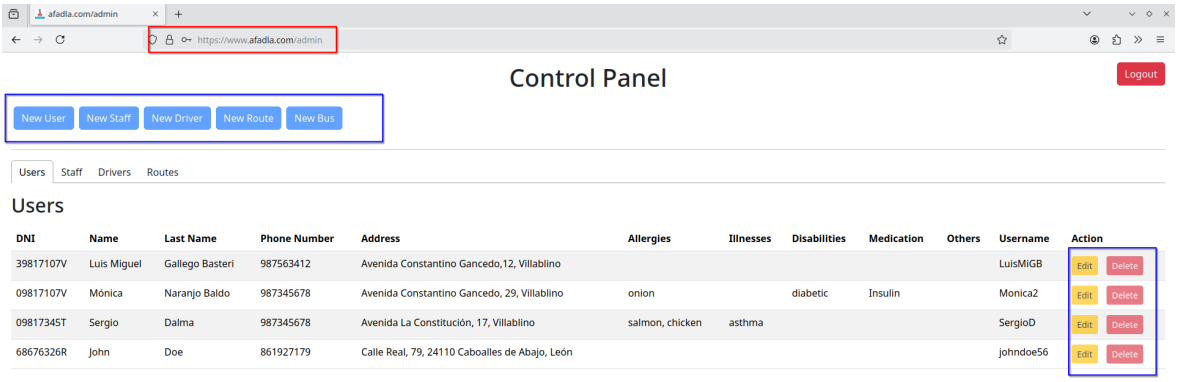

Figure 4.3: Staff view without administrator privileges

In the Routes tab (Figure 4.4), staff can view the routes along with the details such as the route name, driver, time, distance, description, and users assigned to the route. This allows staff members to see route information without making any changes, ensuring that only authorized personnel can edit or add new routes.

Figure 4.4: Routes view without administrator privileges

In the Drivers tab (Figure 4.5), staff members can view information about the drivers, including their DNI, name, last name, phone number, address, role, username, and driver's license number. As with the other tabs, the edit and delete actions are disabled for staff members without administrator privileges, ensuring that only administrators can make changes to driver information.

Figure 4.5: Drivers view without administrator privileges

In the Staff tab (Figure 4.6), staff members can see details about other staff members, including their DNI, name, last name, phone number, address, role, username, and whether they have administrator privileges. This tab helps staff members to know the roles and contact details of their colleagues, while the edit and delete actions remain disabled for non-administrative staff.

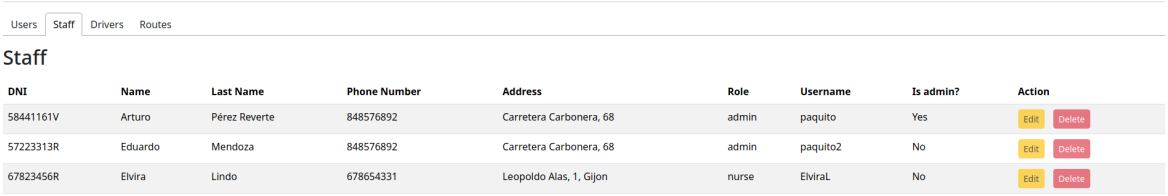

Figure 4.6: Staff view without administrator privileges

4.3.2.- Staff with Administrator Privileges

When logged in as a staff member with administrator privileges, the interface provides additional functionality compared to regular staff members. As shown in Figure 4.7, administrators have access to buttons that enable the creation and editing of users, staff, drivers, routes, and buses. These buttons are highlighted in blue, indicating that they are active and available for use.

Administrators can navigate through the various tabs(Users, Staff, Drivers, and Routes)to view and manage the corresponding records. They have the authority to edit and delete entries in each tab, ensuring that they can maintain and update the system as needed.

In the Users tab (Figure 4.7), administrators can view, edit, and delete user records. The buttons for adding new entries are active, allowing administrators to manage user information effectively.

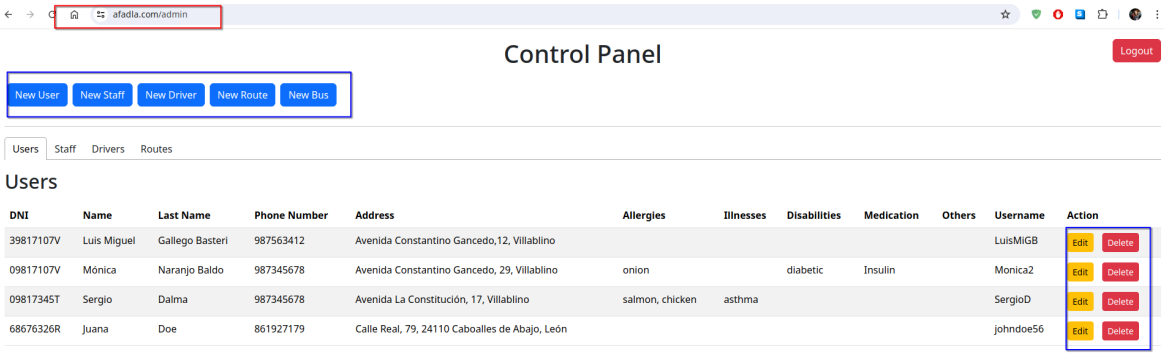

Figure 4.7: Administrator view with full privileges

4.3.2.1.- Creating a New User

To create a new user as an administrator, follow these steps:

1. **Navigate to the Users Tab**: First, go to the Users tab where you can see a list of all current users. Ensure you are logged in as an administrator, as only administrators have the permissions to add new users. Click on the "New User" button to open the user creation form

(Figure 4.8).

2. **Fill in the User Details**: Fill in the necessary details for the new user. This includes the user's DNI, name, last name, phone number, address, allergies, disabilities, medication, illnesses, and any other relevant information. Each field should be completed accurately to ensure proper records.

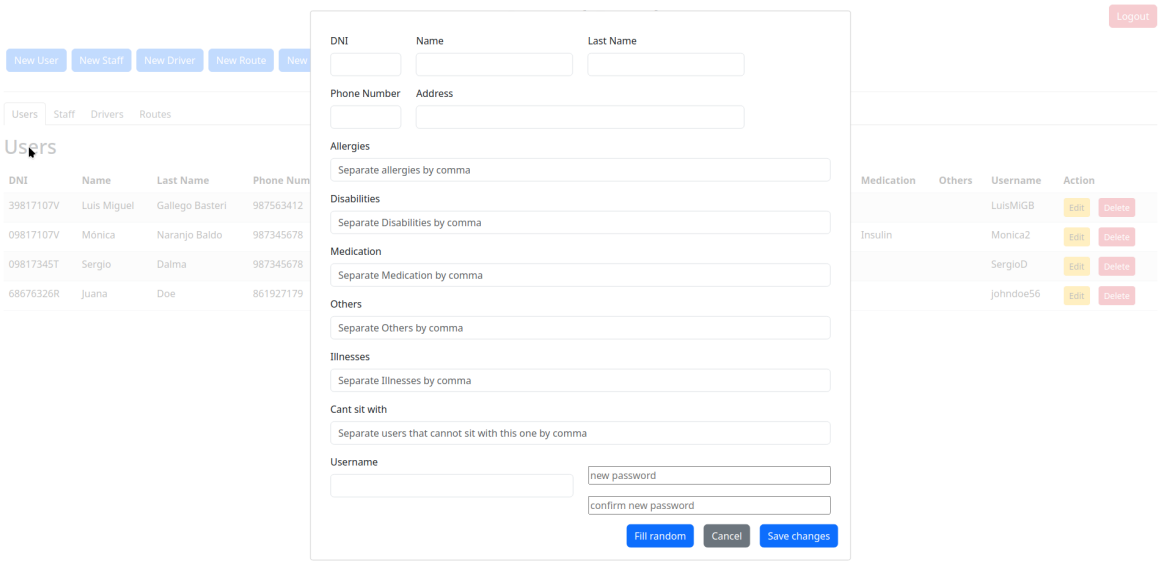

Figure 4.8: New User Form Fields

3. **Password Requirements**: Set a password for the new user. If the password does not meet the requirements, an error message will be displayed, as shown in Figure 4.9.

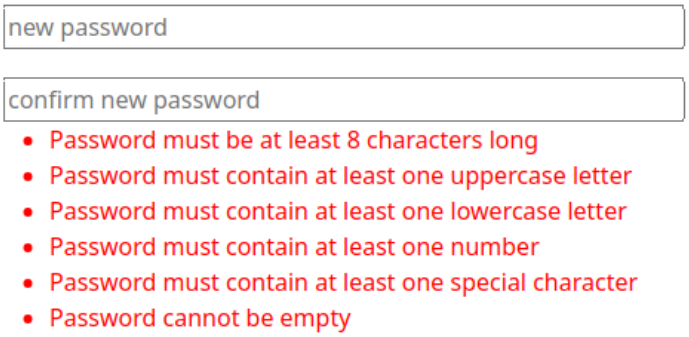

Figure 4.9: Password Requirements

4. **Confirm Password**: Re-enter the password to confirm it. If the passwords do not match, an error message will be displayed, indicating that the passwords do not match (Figure 4.10).

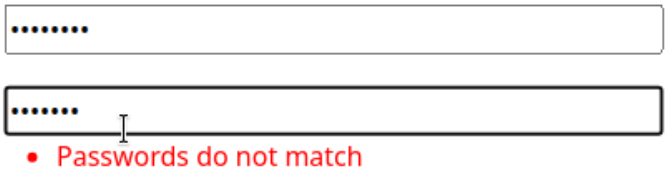

Figure 4.10: Password Mismatch Error

5. **Required fields Format Validation**: Ensure that the required fields (DNI, name, last name, email, phone number, address and username) entered matches the required format. If it does not, an error message will prompt you to correct it (Figure 4.11).

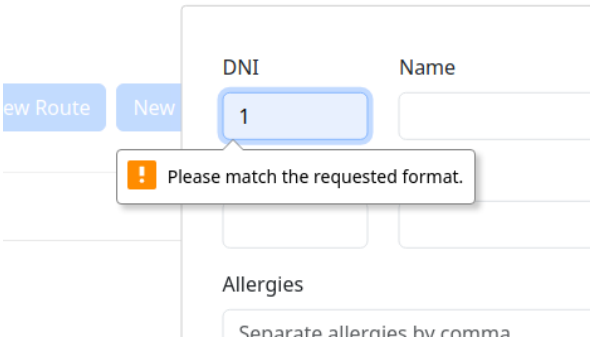

Figure 4.11: DNI Format Validation

6. **Complete the Form**: Ensure all required fields are filled out. If any required fields are left empty, you will be prompted to complete them before proceeding (Figure 4.12).

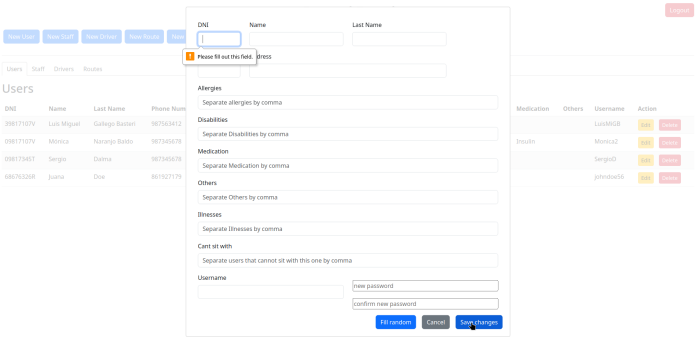

Figure 4.12: Required Field Prompt

7. **Save the New User**: Once all details are correctly filled in and validated, click the "Save Changes" button to add the new user to the system. If the form is filled out correctly, the user will be successfully added and will appear in the Users list.

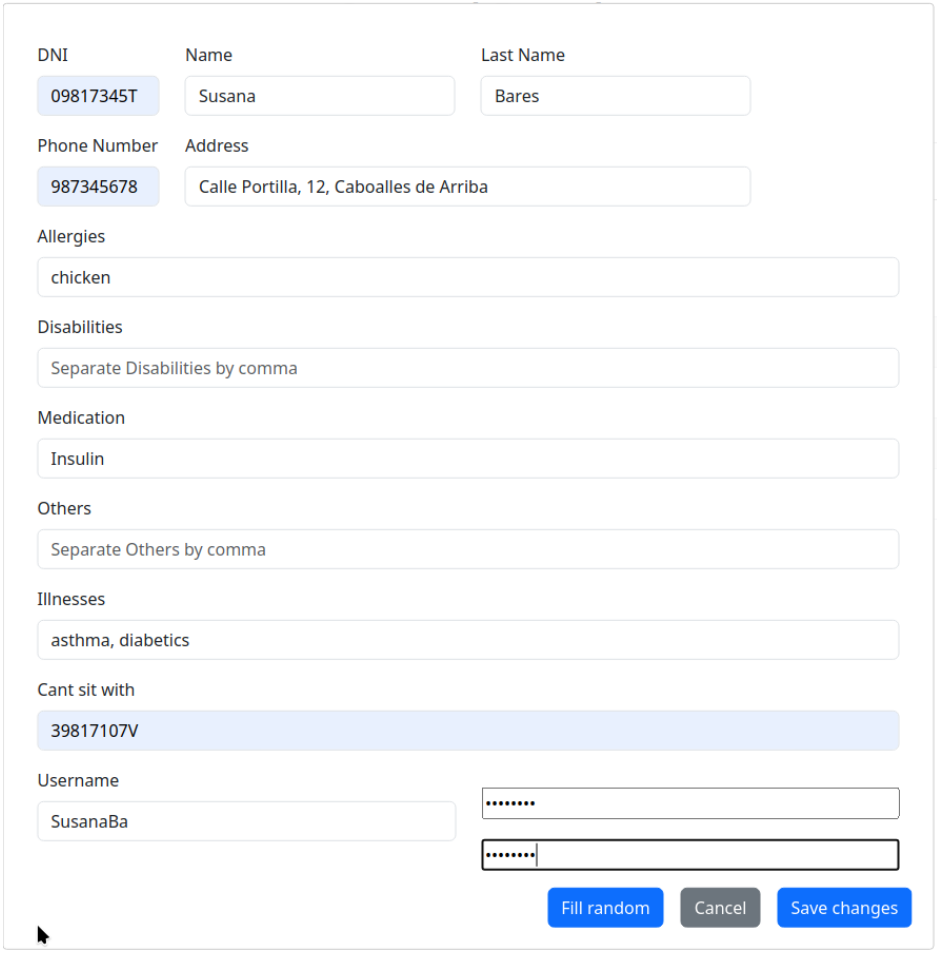

Figure 4.13: New User Form Completed

By following these steps, administrators can efficiently manage user records, ensuring that all necessary information is accurately recorded and maintained. If at any point during the process the "Cancel" button is clicked, the add new user view will be closed, and the main panel will be displayed again. This allows administrators to easily exit the user creation process without saving any changes, returning to the main interface where they can continue to manage the system.

4.3.2.2.- Creating a New Staff

Creating a new staff member involves similar steps to adding a new user, with additional options for administrator privileges.

1. **Initiate the New Staff Creation**: Click on the "New Staff" button to open the form.

2. **Fill in the Staff Details**: Complete the form with required details: DNI, name, last name, phone number, address, roles, username, and password (confirm password).

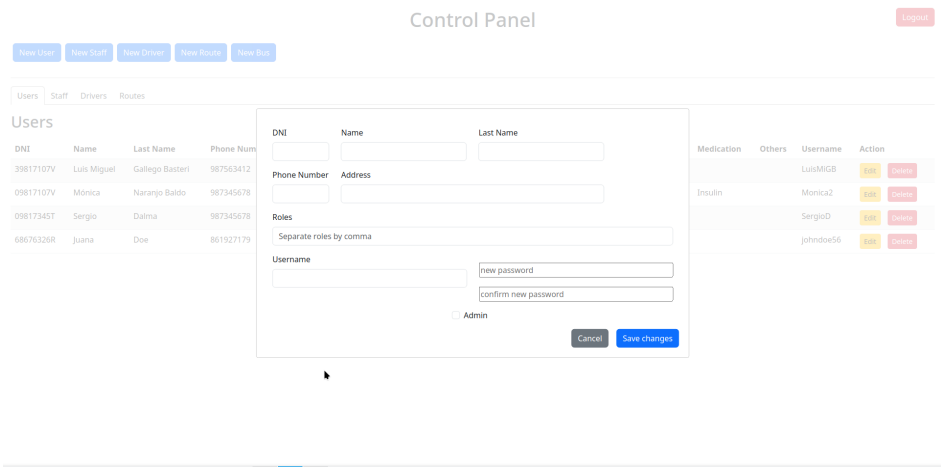

Figure 4.14: New Staff Member Form

4. **Assign Administrator Privileges**: Click the "Admin" checkbox if the staff member needs administrator rights.

5. **Validate fields**: Ensure the password meets criteria and the other fields are correctly formatted.

6. **Handle Validation Errors**: Correct any errors, such as mismatched passwords.

7. **Save the New Staff Member**: Click "Save Changes" to add the staff member to the system. If the form is filled correctly, the staff member will appear in the Staff list.

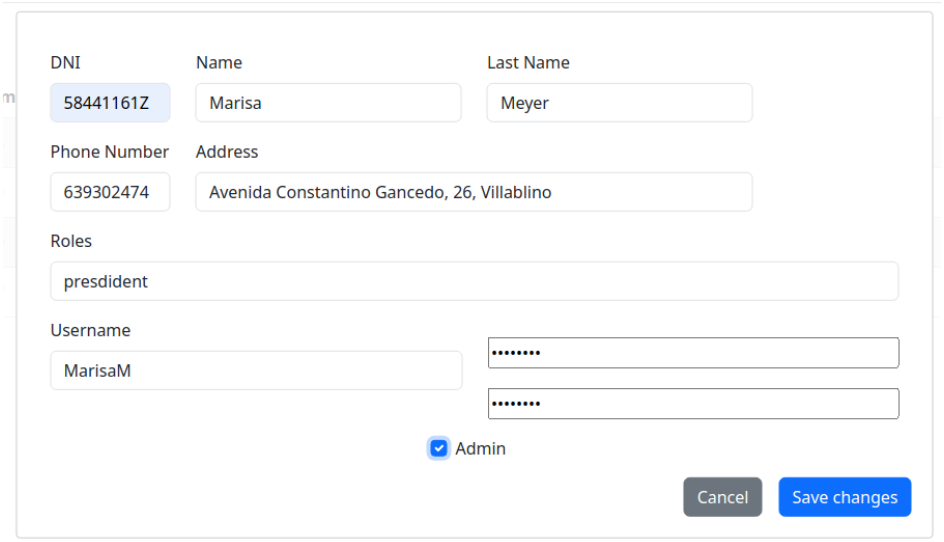

Figure 4.15: New Staff Member Form Completed

If "Cancel" is clicked, the form will close, returning to the main control panel without saving any changes.

4.3.2.3.- Creating a New Driver

Creating a new driver involves similar steps to adding a new user, with specific fields for driver-related information.

1. **Initiate the New Driver Creation**: Click on the "New Driver" button to open the form.

2. **Fill in the Driver Details**: Complete the form with required details: DNI, name, last name, phone number, address, roles, driver license, username, and password (confirm password).

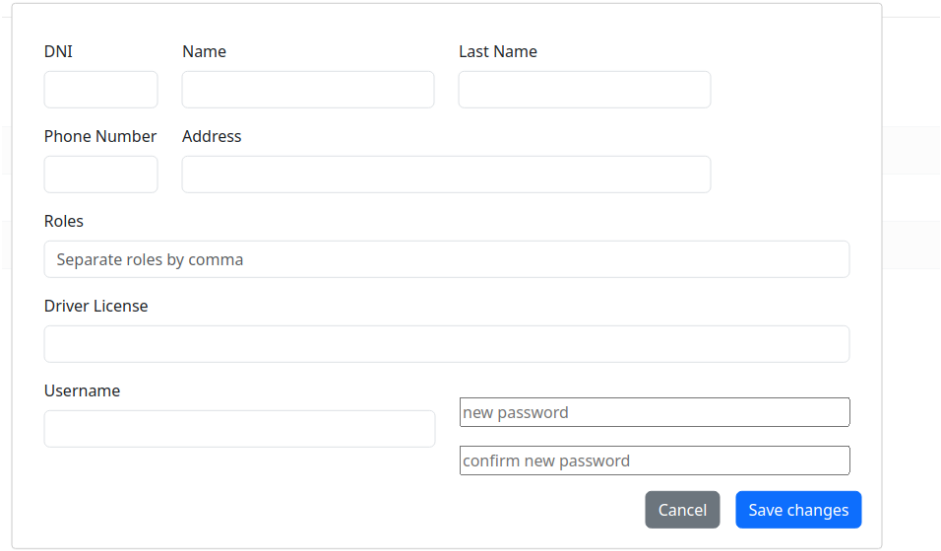

Figure 4.16: New Driver Form

5. **Validate fields**: Ensure the password meets criteria and the other fields are correctly formatted.

4. **Handle Validation Errors**: Correct any errors, such as mismatched passwords.

5. **Save the New Driver**: Click "Save Changes" to add the driver to the system. If the form is filled correctly, the driver will appear in the Drivers list.

If "Cancel" is clicked, the form will close, returning to the main control panel without saving any changes.

4.3.2.4.- Creating a New Route

Creating a new route involves selecting users and a driver for the route.

1. **Initiate the New Route Creation**: Click on the "New Route" button to open the form.

2. **Fill in the Route Details**: Complete the form with the route name and description.

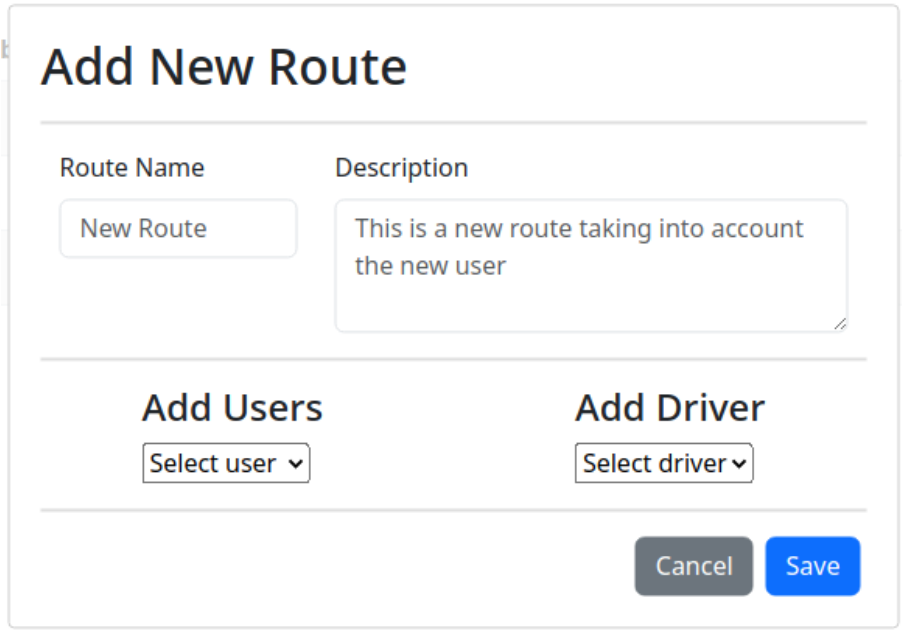

Figure 4.17: New Route Form

3. **Add Users to the Route**: Select users from the dropdown menu to add them to the route. Click the "Add" button after selecting each user.

Figure 4.18: Select Users for Route

4. **Add a Driver to the Route**: Select a driver from the dropdown menu. Click the "Add" button after selecting the driver.

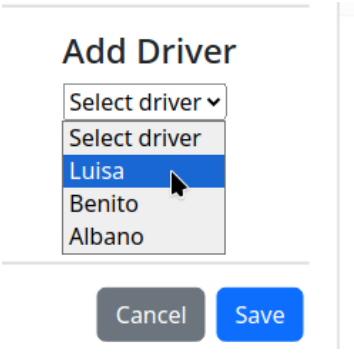

Figure 4.19: Select Driver for Route

5. **Review and Finalize**: Review the selected users and driver. Make any necessary adjustments by clicking "Remove" next to any user or driver.

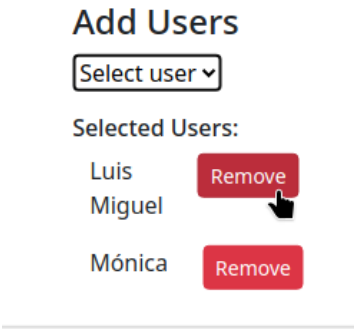

Figure 4.20: Remove user from Route

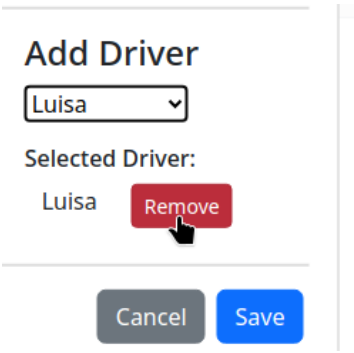

Figure 4.21: Remove Driver for Route

6. **Save the New Route**: Click "Save" to add the route to the system. If the form is filled correctly, the route will appear in the Routes list.

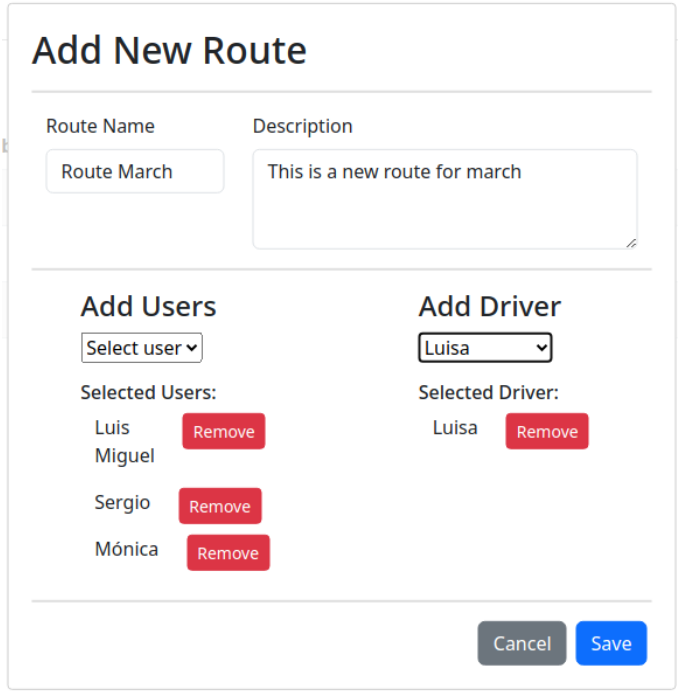

Figure 4.22: New Route Form Completed

If "Cancel" is clicked, the form will close, returning to the main control panel without saving any changes.

4.3.2.5.- Creating a New Bus

Creating a new bus is similar to creating a new route, involving selecting users and a driver for the bus.

1. **Initiate the New Bus Creation**: Click on the "New Bus" button to open the form.

2. **Fill in the Bus Details**: Complete the form with the bus name and description.

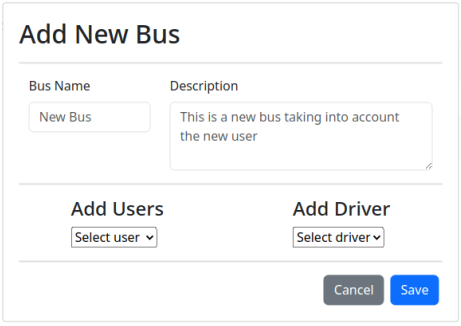

Figure 4.23: New Bus Form

3. **Add Users to the Bus**: Select users from the dropdown menu to add them to the bus.

4. **Add a Driver to the Bus**: Select a driver from the dropdown menu.

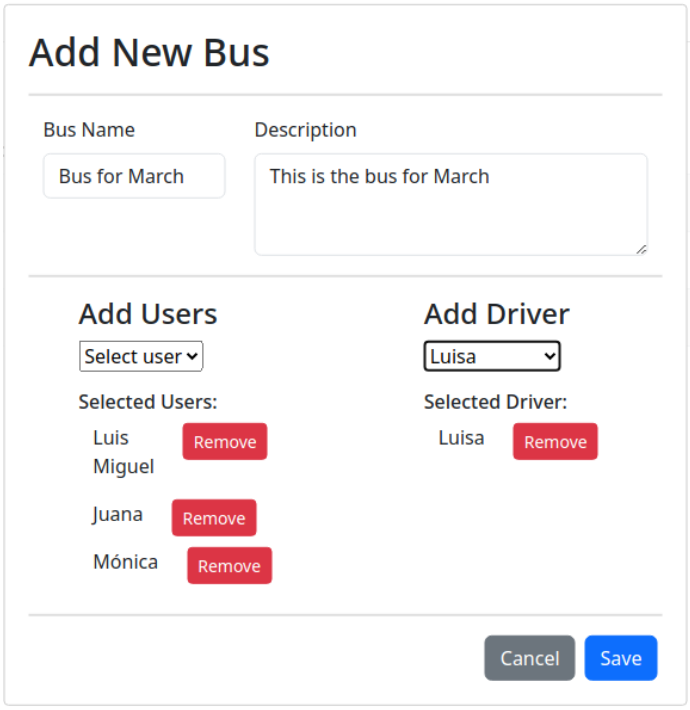

Figure 4.24: New Bus Form Completed

5. **Save the New Bus**: Click "Save" to add the bus to the system. If the form is filled correctly, the bus will appear in the Buses list.

4.3.2.6.- Editing User Details

To edit the details of any user type (User, Driver, or Staff), follow these steps:

1. **Click the Edit Button**: Locate the user in the respective list (Users, Drivers, Staff) and click the "Edit" button next to their name. This will open the edit form with the current details of the user.

2. **Editable Fields**: In the edit form, the fields that can be modified will be enabled, while others will remain disabled. For instance, the DNI field is not editable. The following images show examples of the edit forms for different user types:

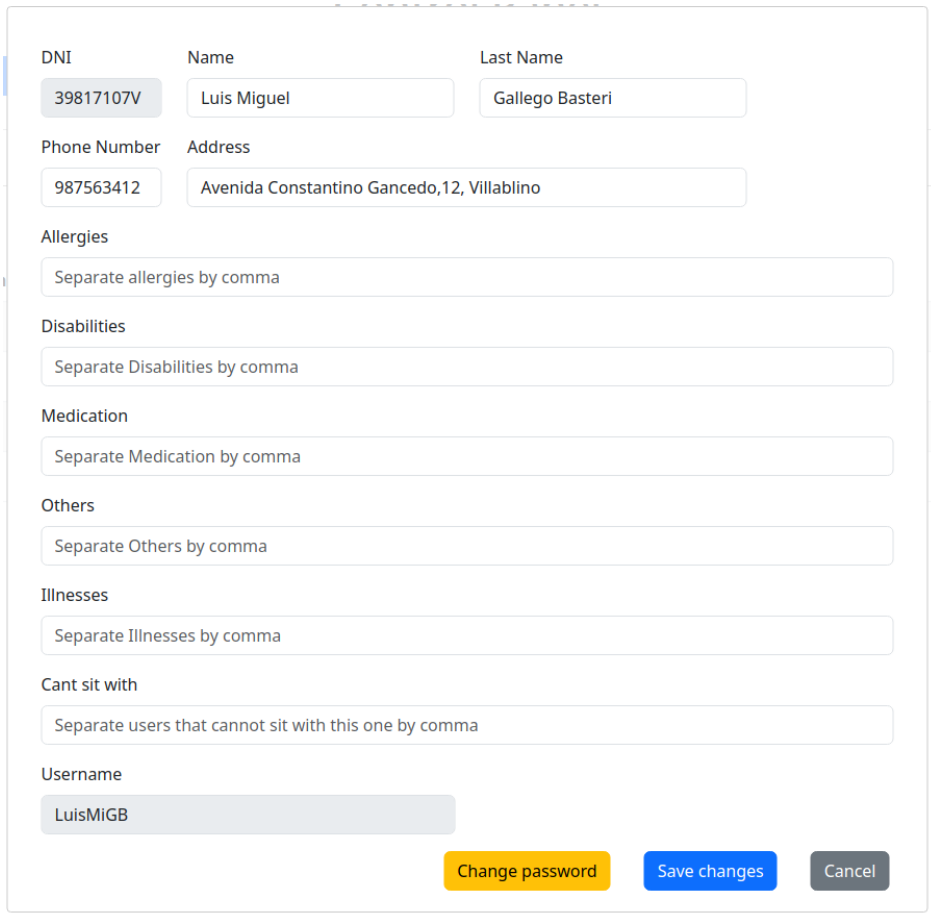

Figure 4.25: Edit User Form

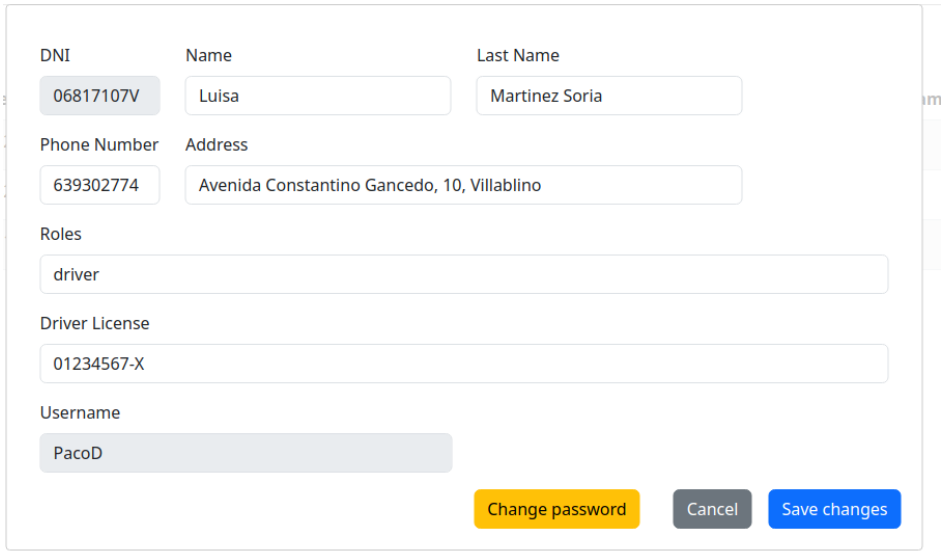

Figure 4.26: Edit Driver Form

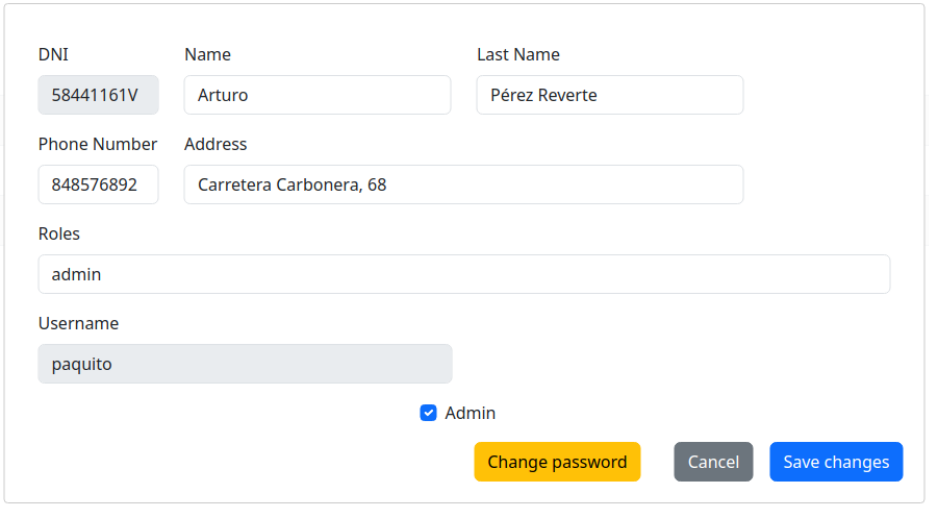

Figure 4.27: Edit Staff Form

3. **Update the Information**: Modify the necessary fields. Ensure that all mandatory fields are completed and that any new information meets the required criteria.

4. **Save Changes or Cancel**:

- Click the "Save Changes" button to update the user details. If the form is filled out correctly, the changes will be saved, and the user will see the updated details in the list.
- Click the "Cancel" button to discard any changes and return to the main control panel without saving.

4.3.2.7.- Deleting a User

Administrators have the ability to delete any user, regardless of their type (User, Driver, or Staff), by following these steps:

1. **Locate the User**: In the respective list (Users, Drivers, Staff), find the user you want to delete.

2. **Click the Delete Button**: Click the "Delete" button next to the user's name. This will remove the user from the system.

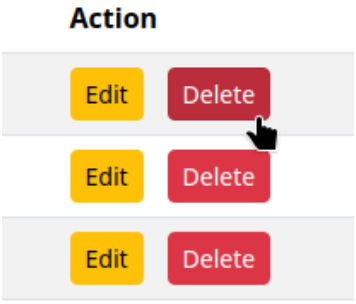

Figure 4.28: Delete User Button

4.3.2.8.- Updating User Details

For each user, within the edit panel, there is an option to change the password. It can be done easily by following these steps:

1. **Access the Edit Panel**: Click on the "Edit" button corresponding to the user whose password you wish to change. This will open the edit panel where you can view and modify the user's information.

2. **Select the Change Password Option**: Within the edit panel, click on the "Change password" button (see Figure 4.29).

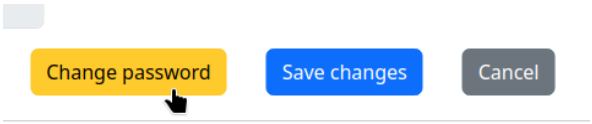

Figure 4.29: Change Password Button

3. **Enter the New Password**: A modal will appear where you need to enter the new password and confirm it (see Figure 4.30). It is important to ensure that the password meets the established security requirements and that both entries match exactly.

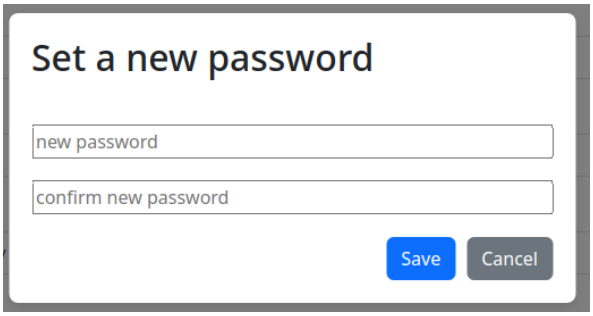

Figure 4.30: Set a New Password Modal

4. **Save the Changes**: Once the new password is entered, click the "Save" button to confirm the change.

5. **Cancel the Operation**: If you wish to cancel the password change process at any time, you can click the "Cancel" button, which will close the modal and no changes will be made.

4.4.- Daycare center user view

4.4.1.- Main Screen

Once logged in, the user is greeted with a welcome message and their profile picture, as seen in Figure 4.31. This screen provides an overview of the user's recent events and actions within the daycare center.

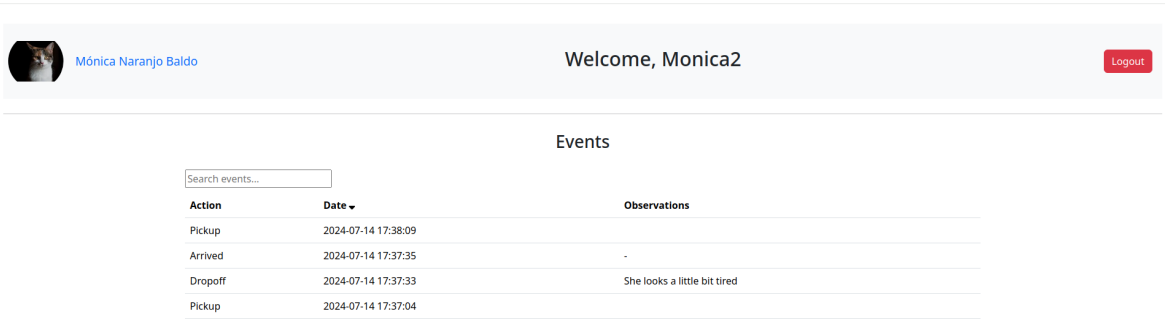

Figure 4.31: Main Screen for Daycare Center User

On this main screen, users can:

- View a list of their recent events, such as pickups, arrivals, and drop-offs.
- Each event is displayed with its action type, date, and any observations noted by the driver.
- Use the search bar to filter events by keywords.

4.4.2.- Search and Sorting Functions

The search bar allows users to filter events by keywords, searching through all three columns: Action, Date, and Observations. For instance, if a user types "tired" into the search bar, it will filter and display only the events that have the word "tired" in any of the columns, as shown in Figure 4.32.

Figure 4.32: Search Events by Observations

Users can also sort the events in ascending or descending order by clicking on the arrows next to the column names. This helps in quickly finding the most recent events or actions. Figure 4.33 demonstrates how to sort events by date.

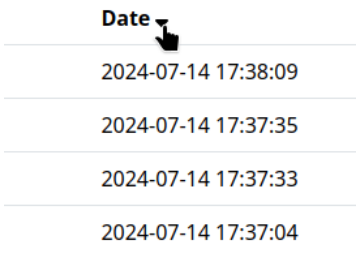

Figure 4.33: Sort Events by Date

4.4.3.- Profile Details

If the user clicks on their name next to the profile picture, they will enter their profile details screen as shown in Figure 4.35.

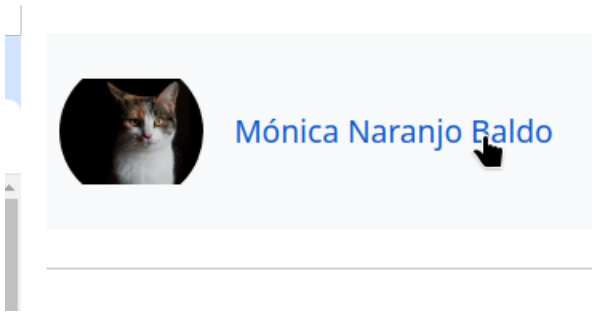

Figure 4.34: Click on Name to Enter Profile Details

In the profile details screen (Figure 4.35), users can view and edit their basic information such as name, DNI, phone number, and address. Additionally, they can manage other information such as allergies, disabilities, medication, illnesses, and users they cannot sit with.

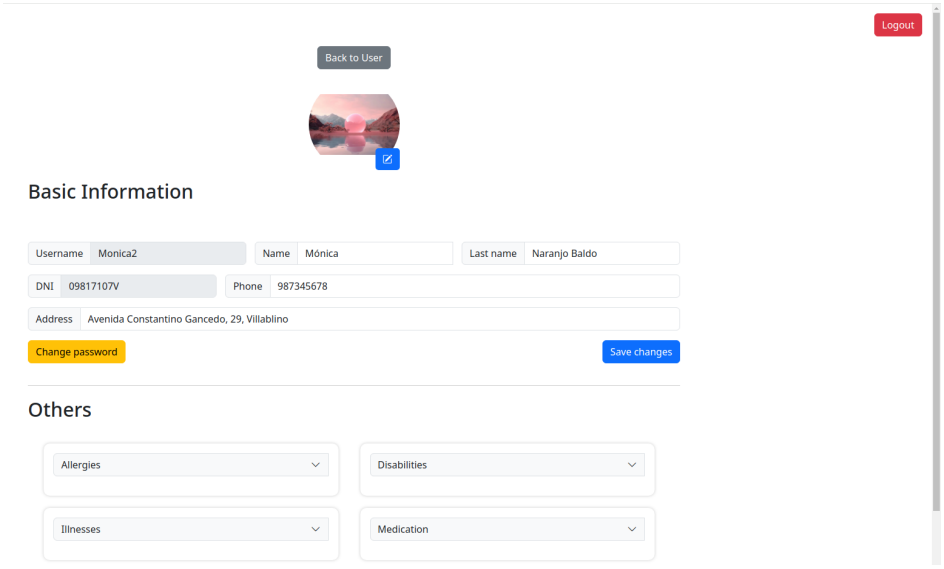

Figure 4.35: User Profile Details

4.4.3.1.- Basic Information

In the Basic Information section (Figure 4.42), users can view and edit(the non-disabled fields) their uDNI, sername, name, last name, phone number, and address. To save any changes, users should click the "Save changes" button.

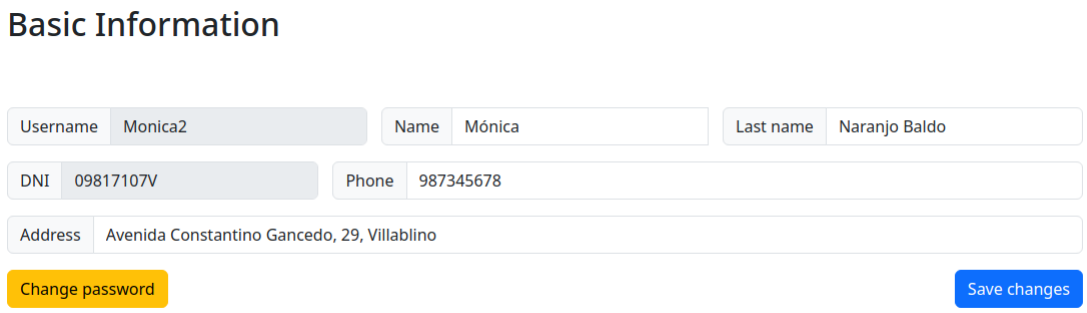

Figure 4.36: Editing Basic Information

Users can change their password by clicking the "Change password" button in this section (Figure 4.42). This will open a modal where users can enter and confirm their new password having the same functionality as the change password option from the administrator(Figure 4.37).

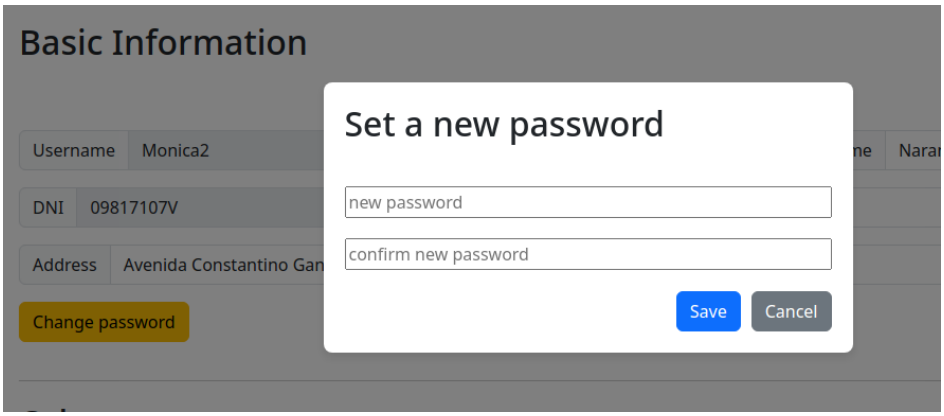

Figure 4.37: Changing Password

4.4.4.- Others Section

In the Others section, users can manage additional information such as allergies, disabilities, illnesses, medication, and other notes. They can add new entries by typing in the respective fields and clicking "Add to list," or remove existing entries by clicking the red "x" button. Changes are updated automatically.

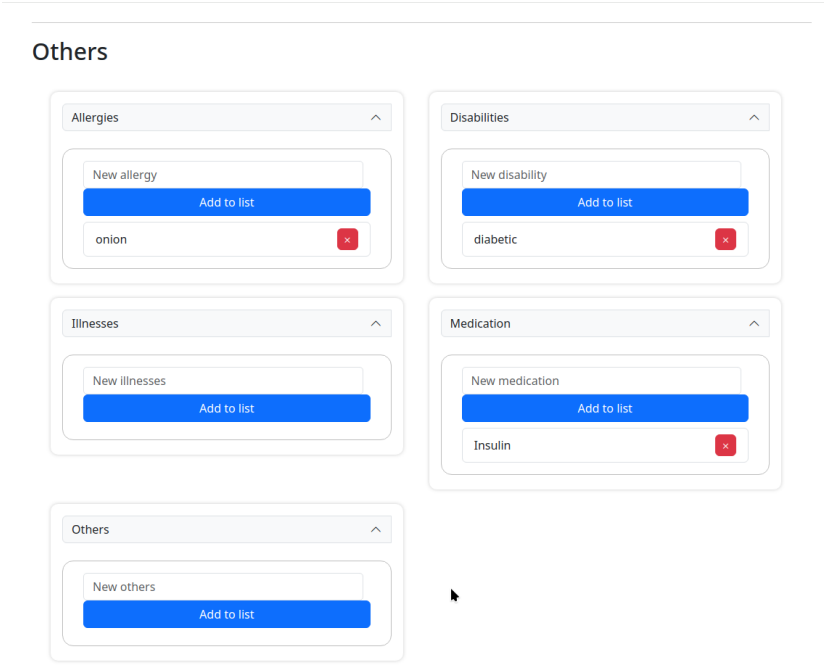

Figure 4.38: Manage Additional Information

To change the profile picture, users can click on the small icon on the current profile picture, which will open a file selection dialog to choose a new image from their device.

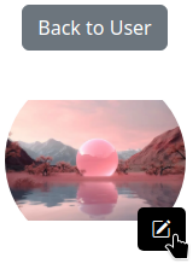

Figure 4.39: Change Profile Picture

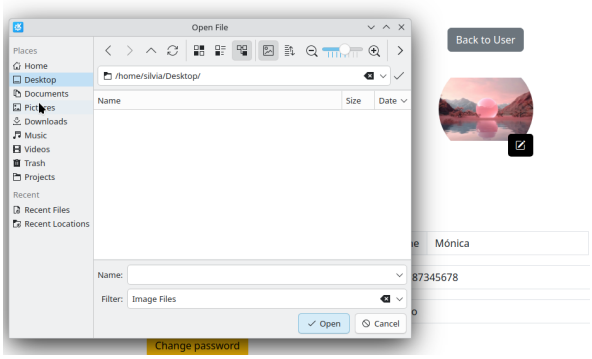

Figure 4.40: Select New Profile Picture

To return to the main panel, users can click the "Back to User" button.

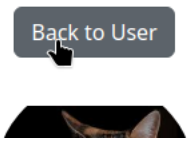

Figure 4.41: Back to Main Screen

4.5.- Driver View

When a driver logs in successfully, the main screen they see is as shown in Figure 4.42. This view allows drivers to select and manage the buses and routes.

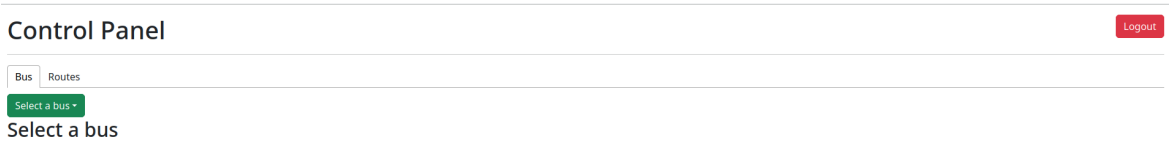

Figure 4.42: Main Screen for Driver

4.5.1.- Route Selection and Management

Drivers can select a route from the "Routes" tab. Clicking on the "Select a route" dropdown menu displays available routes, as shown in Figure 4.43.

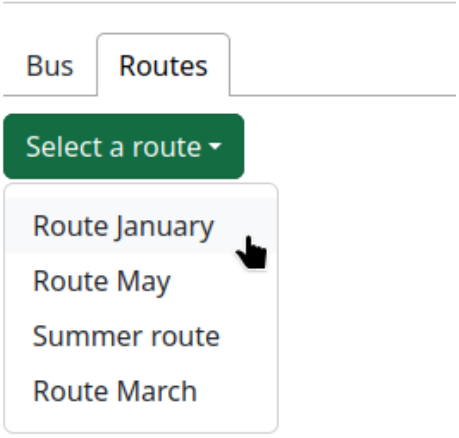

Figure 4.43: Select a Route

Upon selecting a route, the driver can view the route details and manage pickups and drop-offs. The route is displayed with the users' details and action buttons for each user, as seen in Figure 4.44.

Figure 4.44: Route Detail

4.5.1.1.- Managing Pickups and Drop-offs

Reset Trip: The "Reset trip" button allows the driver to start the journey from the beginning.

Visited or Missing: Clicking the "Visited" or "Missing" buttons sends an event to the user indicating they have been picked up or marked as missing, respectively. The observation field can be filled in before marking a user and is optional.

Drop-off: Once all users have been picked up or marked as missing, the driver proceeds to the drop-off phase, where each user is dropped off at their destination.

View Map: Clicking "View map" shows the route on a map, providing a visual representation of the journey.

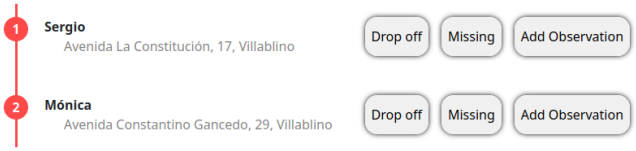

Figure 4.45: Drop Off User

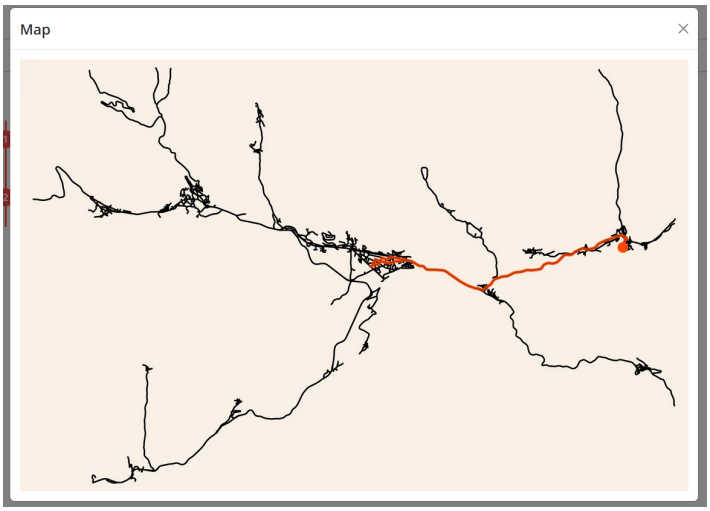

Figure 4.46: Route Map

4.5.1.2.- Adding Observations

The driver can add observations for each user during the trip. Clicking "Add Observation" opens a modal to enter the observations, as shown in Figure 4.47.

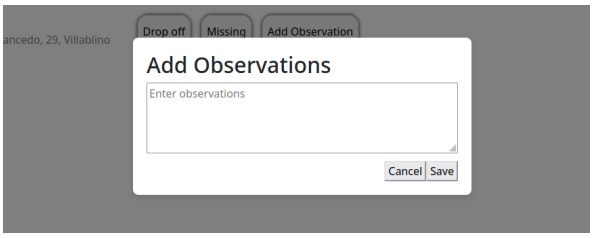

Figure 4.47: Add Observation

4.5.2.- Bus View

From the bus view, drivers can select a bus and see its seating arrangement.

- **Select a Bus**: Click on the "Select a bus" dropdown menu to choose a bus.

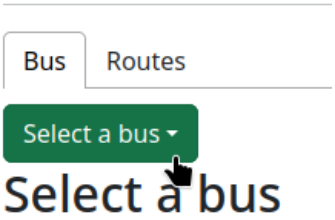

Figure 4.48: Select a Bus

- **Seating Arrangement**: The seating arrangement displays user profile pictures,driving wheel (driver seat) empty seats (X), and aisle spaces (black).

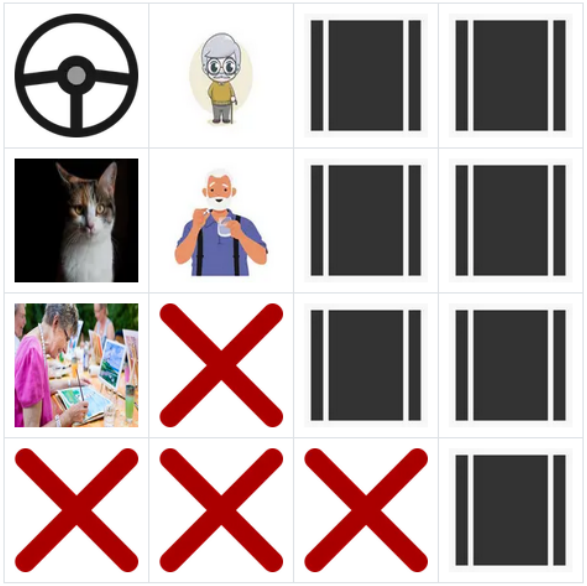

Figure 4.49: Bus Seating Arrangement

4.6.- Logout

At any time, any user can log out of the application by clicking the red "Logout" button located at the top right corner of the screen. This will redirect the user to the login page.

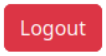

Figure 4.50: Logout Button

5. Technical manual

5.1.- Introduction

This chapter provides a comprehensive guide for developers to understand, deploy, and maintain the software components of the project, which includes a frontend developed in Next.js, a backend in Flask, and an auxiliary component written in C. Each component is deployed in specific environments, with detailed instructions for setup and configuration.

5.2.- Frontend: Next.js Application

The frontend application is developed using Next.js and managed with the ‘pnpm‘ package manager. It is automatically built and deployed on Vercel, linked to the main branch of the GitHub repository. The package configuration and the project layout are crucial for understanding and maintaining the frontend.

5.2.1.- Package Configuration

The ‘package.json‘ file outlines the dependencies and scripts used in the project:

```
{
  "name": "frontend-web-tfg",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "@fortawesome/free-solid-svg-icons": "^6.5.1",
    "@fortawesome/react-fontawesome": "^0.2.0",
    "axios": "^1.6.8",
    "bootstrap": "^5.3.3",
    "bootstrap-icons": "^1.11.3",
    "cookies-next": "^2.1.2",
```

```
    "eslint": "^8.57.0",
    "eslint-config-next": "^14.1.4",
    "next": "^14.1.4",
    "nookies": "^2.5.2",
    "react": "^18.2.0",
    "react-bootstrap": "^2.10.2",
    "react-dom": "^18.2.0",
    "react-modal": "^3.16.1",
    "react-tabs": "^6.0.2",
    "validator": "^13.12.0"
  },
  "devDependencies": {
    "@types/node": "^20.11.30",
    "@types/react": "^18.2.73",
    "@types/react-dom": "^18.2.23",
    "@types/react-modal": "^3.16.3",
    "sass": "^1.72.0",
    "typescript": "^5.4.3"
  }
}
```

5.2.1.1.- Dependencies

- `@fortawesome/free-solid-svg-icons`, `@fortawesome/react-fontawesome`: Provides Font Awesome icons for use in React components.
- `axios`: A promise-based HTTP client for making API requests.
- `bootstrap`, `bootstrap-icons`: CSS framework for responsive design and icons.
- `cookies-next`, `nookies`: Libraries for handling cookies in Next.js.
- `eslint`, `eslint-config-next`: Tools for linting and maintaining code quality.
- `next`: The core framework for the Next.js application.
- `react`, `react-dom`, `react-bootstrap`, `react-modal`, `react-tabs`: Core libraries for building React components.
- `validator`: A library for validating and sanitizing strings.

5.2.1.2.- Scripts

- `dev`: Starts the development server.
- `build`: Builds the application for production.
- `start`: Starts the production server.
- `lint`: Lints the codebase using ESLint.

5.2.2.- Project Layout

The project is structured to maintain modularity and readability:

```
frontend-web-tfg/
 components/
    bus_stops.tsx
    bus.tsx
    event.tsx
    itemizer.tsx
    map.tsx
    modals/
    passwordComponent.tsx
    tables/
    tooltip.tsx
    types.tsx
 node_modules/
 pages/
    _app.tsx
    admin/
    driver/
    index.tsx
    user/
 public/
    favicon.ico
    images/
    next.svg
    seat.png
    vercel.svg
```

```
styles/
    admin.module.scss
    bus.module.scss
    bus_stops_modal.module.scss
    bus_stops.module.scss
    change_passwords_modal.module.scss
    globals.scss
    Itemizer.module.scss
    login.module.scss
    password_form.module.scss
    profile.module.scss
    Tooltip.module.scss
    user.module.scss
jsconfig.json
next.config.js
next-env.d.ts
package.json
pnpm-lock.yaml
README.md
tsconfig.json
```

5.2.3.- Running the Project

To run the frontend application:

1. Install the dependencies using `pnpm`:

   ```
   pnpm install
   ```

2. Start the development server:

   ```
   pnpm run dev
   ```

3. To build the project for production:

   ```
   pnpm run build
   ```

4. To start the production server:

```
pnpm run start
```

5.3.- Backend: Flask Application

The backend of the project is developed using Flask and deployed on an EC2 AWS server. Nginx is used to proxy connections, and Gunicorn runs the backend.

5.3.1.- Project Layout

The backend project is organized as follows:

```
backend_tfg/
 bin/
    bus.so
 src/
    app.py
    cache/
    database/
    entities/
    exceptions/
    __init__.py
    __pycache__/
    routes/
    static/
    templates/
    uploaded_images/
    utils/
 tests/
    bus_driver_test.py
    staff_test.py
    user_test.py
 LICENSE
 package.json
 package-lock.json
 poetry.lock
```

```
pyproject.toml
route_map.html
serverless.yml
test_route.py
wsi_handler.py
```

5.3.2.- Running the Backend

To run the backend application:

1. Install dependencies using Poetry:

   ```
   poetry install
   ```

2. Start the application with Flask (development mode):

   ```
   flask run
   ```

3. Start the application with Gunicorn:

   ```
   gunicorn --workers 2 --bind 0.0.0.0:8000 src.app:app
   ```

5.3.3.- Nginx Configuration

Nginx is configured to proxy requests to the Flask application running on Gunicorn. Key configurations include increasing buffer sizes and timeouts to handle large requests efficiently. The Nginx configuration also manages SSL certificates with Certbot for secure connections.

5.3.4.- Gunicorn Service Configuration

The Gunicorn service is managed using systemd. The service file specifies the user, environment variables, working directory, and the command to start Gunicorn. It ensures the service restarts automatically on failure to maintain high availability.

5.4.- Auxiliary Project: bus_sitter

The ‘bus_sitter‘ project is a C program compiled to a CPython extension and used by the backend for performance-critical operations. The compiled shared object file (‘bus.so‘) is located in the ‘bin/‘ directory and imported into the backend application.

5.4.1.- Compiling the `bus_sitter` Project

The `bus_sitter` project is a critical component written in C, which is compiled to a shared object file (`bus.so`) to be used as a CPython extension in the backend. This section provides instructions on how to compile the project using the provided `Makefile`.

5.4.1.1.- Makefile Overview

The `Makefile` defines several targets for building the project in different configurations. Below is a brief description of each target:

- `all`: Default target that compiles the project in debug mode.
- `debug`: Compiles the project with debugging information.
- `run`: Compiles the project in debug mode and runs the resulting executable.
- `release`: Compiles the project with optimizations enabled.
- `python`: Compiles the project as a shared object file for use with Python.
- `clean`: Removes all compiled files from the `bin/` directory.
- `baseline`: Compiles a baseline executable for testing.
- `test`: Compares the output of the debug and baseline executables.

5.4.1.2.- Compiling for Python

To compile the `bus_sitter` project as a CPython extension, follow these steps:

1. Navigate to the `bus_sitter` project directory:

```
cd path/to/bus-sitter
```

2. Ensure the source files are located in the appropriate directories as specified in the `Makefile`:

```
src/main.c
src/lib/utils.c
src/lib/algorithm.c
```

3. Run the `make python` command to compile the project into a shared object file:

```
make python
```

4. The compiled shared object file, `bus.so`, will be located in the `bin/` directory:

```
bin/bus.so
```

5.4.1.3.- Other Compilation Options

For development and testing purposes, you can compile the project in different modes:

- **Debug Mode**: Compiles the project with debugging information.

```
make debug
```

- **Release Mode**: Compiles the project with optimizations enabled.

```
make release
```

- **Running the Executable**: Compiles the project in debug mode and runs the executable.

```
make run
```

- **Cleaning Up**: Removes all compiled files from the `bin/` directory.

```
make clean
```

- **Baseline Compilation**: Compiles a baseline executable for testing.

```
make baseline
```

- **Testing**: Compares the output of the debug and baseline executables.

```
make test
```

These commands ensure that the `bus_sitter` project is correctly compiled for various development and production needs, facilitating its integration into the backend application.

5.5.- Conclusion

This technical manual provides a detailed guide for setting up, running, and maintaining the project's frontend and backend components. By following the outlined steps and configurations, developers can effectively manage the deployment and ensure the system's stability and performance.

6. Annex

6.1.- Tests

6.1.1.- Login Tests

Id	Description	Inputs	Expected output	Final output
L1	Test staff login with invalid password for an exiting staff member	Selected role: staff member, DNI:12345678B, password:123	Invalid	Invalid
L2	Test staff login with invalid DNI for an exiting staff member	Selected role: staff member, DNI:123, password:Asterijsko*	Invalid	Invalid
L3	Test staff login with valid credentials but having the user role selected	Selected role: User, DNI:12345678B, password:Asterijsko*	Invalid	Invalid
L4	Test staff login with valid credentials but having the driver role selected	Selected role: Driver, DNI:12345678B, password:Asterijsko*	Invalid	Invalid
L5	Test user with invalid password for an exiting staff member	Selected role: User, DNI:09817107V, password:123	Invalid	Invalid
L6	Test user login with invalid DNI for an exiting staff member	Selected role: User, DNI:123, password:Asterijsko*	Invalid	Invalid
L7	Test user login with valid credentials but having the user role selected	Selected role: staff member, DNI:09817107V, password:Asterijsko*	Invalid	Invalid
L8	Test user login with valid credentials but having the driver role selected	Selected role: Driver, DNI:09817107V, password:Asterijsko*	Invalid	Invalid

Id	Description	Inputs	Expected output	Final output
L9	Test driver login with invalid password for an exiting staff member	Selected role: Driver, DNI:06817107V, password:123	Invalid	Invalid
L10	Test driver login with invalid DNI for an exiting staff member	Selected role: Driver, DNI:123, password:Asterijsko*	Invalid	Invalid
L11	Test driver login with valid credentials but having the user role selected	Selected role: User, DNI:06817107V, password:Asterijsko*	Invalid	Invalid
L12	Test driver login with valid credentials but having the driver role selected	Selected role: staff member, DNI:06817107V, password:Asterijsko*	Invalid	Invalid
L13	Test staff member login with valid credentials and staff member role selected	Selected role: staff member, DNI:12345678B, password:Asterijsko*	Redirected to staff page	Redirected to staff page
L14	Test user login with valid credentials and user role selected	Selected role: user, DNI:09817107V, password:Asterijsko*	Redirected to user page	Redirected to user page
L15	Test driver login with valid credentials and driver role selected	Selected role: driver, DNI:06817107V, password:Asterijsko*	Redirected to driver page	Redirected to driver page

Table 6.1: Tests: Login tests

6.1.2.- Staff Tests

Id	Description	Inputs	Expected output	Final output
SM1	Test staff can view the daycare center users list	Being login as a staff without administrator privileges and have selected the Users tab panel	See daycare center users list	See daycare center users list

Id	Description	Inputs	Expected output	Final output
SM2	Test staff can view the staff members list	Being login as a staff without administrator privileges and have selected the Staff tab panel	See staff members list	See staff members list
SM3	Test staff can view the drivers list	Being login as a staff without administrator privileges and have selected the Drivers tab panel	See drivers list	See drivers list
SM4	Test staff can view the routes list	Being login as a staff without administrator privileges and have selected the Routes tab panel	See drivers routes	See drivers routes
SM5	Test staff cannot edit any information or create any user	Being login as a staff without administrator privileges clicking in the buttons	No action	No action

Table 6.2: Tests: Staff tests

6.1.3.- Administrator Tests

Id	Description	Inputs	Expected output	Final output
AD1	Administrator can view the daycare center users list	Being login as a staff with administrator privileges and have selected the Users tab panel	See daycare center users list	See daycare center users list
AD2	Test administrator can view the staff members list	Being login as a staff with administrator privileges and have selected the Staff tab panel	See staff members list	See staff members list
AD3	Administrator can view the drivers list	Being login as a staff with administrator privileges and have selected the Drivers tab panel	See drivers list	See drivers list

Id	Description	Inputs	Expected output	Final output
AD4	Administrator can view the routes list	Being login as a staff with administrator privileges and have selected the Routes tab panel	See drivers routes	See drivers routes
AD5	Administrator can create a user with only its required fields	Being login as a staff with administrator privileges, have clicked on the "New User" button and introduce: DNI- 39817107V, Name: Luis MIguel, Last Name-Gallego Basteri, phone number - 987563412, address: Avenida Constantino Gancedo,12, Villablino, Username: LuisMiGB, password:Asterijsko*	User created with that information and the users list is updated	User created with that information and the users list is updated
AD5	Administrator can create a user with some optional fields	Being login as a staff with administrator privileges, have clicked on the "New User" button and introduce: DNI-09817345T, Name: Sergio, Last Name-Dalma, phone number - 987563412, address: Avenida La Constitución, 17, Villablino, Username: SergioD, password:Asterijsko*, Allergies - salmon, chicken, Illnesses: asthma	User created with that information and the users list is updated	User created with that information and the users list is updated

Id	Description	Inputs	Expected output	Final output
AD6	Administrator can update the user information except their DNI and username	Being login as a staff with administrator privileges, have clicked on the "Edit" button of an user and introduce: Allergies: onion (being this filed empty at the beginning) - Phone Number: 654325001 (being this one equal to 987563412 at the beginning)	User updated with that information and the users list is updated	User updated with that information and the users list is updated
AD7	Administrator can update the password of an user	Being login as a staff with administrator privileges, have clicked on the "Edit" button of an user, the clicked on the "change password" button and introduce the new password along with its confirmation (password: Valentina+10, confirm new password: Valentina+10)	User password is updated, the set new password modal is close and the user is returned to the edition panel.	User password is updated, the set new password modal is close and the user is returned to the edition panel.

Id	Description	Inputs	Expected output	Final output
AD8	Administrator can create new staff member with administrator privileges	Being login as a staff with administrator privileges and have clicked on the "New Staff Button" and introduce: DNI- 29817345T, Name: Valentina, Last Name-Rilo, phone number - 987563412, address: Avenida La Constitución, 17, Villablino, Username: ValentinaR, password:Asterijsko* (same for confirm password), Roles:president, select IsAdmin	Staff member created with administrator privileges and staff list updated	Staff member created with administrator privileges and staff list updated
AD9	Administrator can create new staff member without administrator privileges	Being login as a staff with administrator privileges and have clicked on the "New Staff Button" and introduce: DNI-59817345T, Name: Luisa, Last Name-Rilo, phone number - 987563412, address: Avenida La Constitución, 17, Villablino, Username: LuisaR, password:Asterijsko* (same for confirm password), Roles:president	Staff member created without administrator privileges and staff list updated	Staff member created without administrator privileges and staff list updated

Id	Description	Inputs	Expected output	Final output
AD10	Administrator can create new driver	Being login as a staff with administrator privileges and have clicked on the "New Driver" button and introduce: DNI- 52317345T, Name: Albano, Last Name- Perez, phone number - 987563412, address: Avenida La Constitución, 17, Villablino, Username: Albano, password:Asterijsko* (same for confirm password), Roles:driver, Driver license:01234567-X	Driver member created and drivers list updated	Driver created and drivers list updated
AD11	Administrator can create a new route	Being logged in as a staff with administrator privileges and having clicked on the "New Route" button, and introducing: Route Name - Route one, Description - Description for route one, Selected Users - Luis Miguel, Sergio, Selected Driver - Luisa	Route created and routes list updated	Route created and routes list updated
AD12	Administrator can create a new bus	Being logged in as a staff with administrator privileges and having clicked on the "New Bus" button, and introducing: Bus Name - Bus one, Description - Description for bus one, Selected Users - Sergio, Juana, Luis Miguel, Selected Driver - Benito	Bus created and buses list updated	Bus created and buses list updated

Id	Description	Inputs	Expected output	Final output
AD13	Administrator can grant admin privileges	Being logged in as a staff with administrator privileges and having clicked on the "Edit" button for a staff member, check the "Admin" checkbox and save changes	Admin privileges granted and user details updated	Admin privileges granted and user details updated
AD14	Administrator can revoke admin privileges	Being logged in as a staff with administrator privileges and having clicked on the "Edit" button for a staff member, uncheck the "Admin" checkbox and save changes	Admin privileges revoked and user details updated	Admin privileges revoked and user details updated
AD15	Administrator can change user password	Being logged in as a staff with administrator privileges and having clicked on the "Edit" button for a user, click on "Change password", enter the new password as "NewPass123!" and confirm password, then click "Save"	Password changed and user details updated	Password changed and user details updated
AD16	Administrator can delete a user	Being logged in as a staff with administrator privileges and click on the "Delete" button next to the user. Confirm the deletion in the confirmation dialog.	User is deleted and removed from the users list	User is deleted and removed from the users list

Table 6.3: Tests: Administrator tests

6.1.4.- Driver Tests

Id	Description	Inputs	Expected output	Final output
DR1	Test driver login with correct credentials	Selected role: driver, DNI: 87654321A, password: DriverPassword123	Driver is logged in and directed to the main screen displaying the control panel	Driver is logged in and directed to the main screen displaying the control panel
DR2	Test driver views the list of buses	Log in as a driver, click on the "Bus" tab	A list of available buses is displayed	A list of available buses is displayed
DR3	Test driver selects a bus and views its layout	Log in as a driver, click on the "Bus" tab, select "Bus March" from the dropdown menu (existing this bus in the database)	The layout of the selected bus is displayed, showing the seating arrangement and available seats	The layout of the selected bus is displayed, showing the seating arrangement and available seats
DR4	Test driver views the list of routes	Log in as a driver, click on the "Routes" tab	A list of available routes is displayed	A list of available routes is displayed
DR5	Test driver selects a route	Log in as a driver, click on the "Routes" tab, select "Route March" from the dropdown menu	The details of the selected route are displayed	The details of the selected route are displayed
DR6	Test driver marks a user as visited	Log in as a driver, select a route, click on the "Visited" button next to a user	The user is marked as visited	The user is marked as visited
DR7	Test driver marks a user as missing	Log in as a driver, select a route, click on the "Missing" button next to a user	The user is marked as missing	The user is marked as missing

Id	Description	Inputs	Expected output	Final output
DR8	Test driver adds an observation	Log in as a driver, select a route, click on the "Add Observation" button next to a user, enter the observation and save	The observation is added to the user's record	The observation is added to the user's record
DR9	Test driver resets the trip	Log in as a driver, select a route, click on the "Reset trip" button	The trip is reset, and the route starts from the beginning	The trip is reset, and the route starts from the beginning
DR10	Test driver performs a drop-off	Log in as a driver, select a route, visits all users, click on the "Drop off" button next to a user	The user is marked as dropped off	The user is marked as dropped off
DR11	Test driver views the route map	Log in as a driver, select a route, click on the "View map" button	The map showing the route is displayed	The map showing the route is displayed

Table 6.4: Tests: Driver tests

6.1.5.- Daycare center User Tests

Id	Description	Inputs	Expected output	Final output
US1	Test user login with correct credentials	Selected role: daycare center user, DNI: 09817107V, password: MonicaPassword	User is logged in and directed to the main screen displaying a welcome message, profile picture, and a list of recent events	User is logged in and directed to the main screen displaying a welcome message, profile picture, and a list of recent events

Id	Description	Inputs	Expected output	Final output
US3	Test user filters events by description	Logged in as a user, input keyword "tired" into the search bar	Only events with "tired" in the description are displayed: Dropoff - 2024-07-14 17:37:33 - She looks a little bit tired	Only events with "tired" in the description are displayed: Dropoff - 2024-07-14 17:37:33 - She looks a little bit tired
US4	Test user filters events by action	Logged in as a user, input keyword "Arrived" into the search bar	Only events with "Arrived" in the action column are displayed 2024-07-14 18:49:36, Arrived - 2024-07-14 18:38:48, Arrived - 2024-07-14 17:37:35n	Only events with "Arrived" in the action column are displayed 2024-07-14 18:49:36, Arrived - 2024-07-14 18:38:48, Arrived - 2024-07-14 17:37:35
US5	Test user filters events by date	Logged in as a user, input keyword "2024-07-14 17:37:35" into the search bar	Events filtered by date	Only events with the date "2024-07-14 17:37:35" are displayed, Arrived - 2024-07-14 17:37:35
US6	Test user sorts events by date	Logged in as a user, clicks on the date column header	Events sorted by date	Events sorted by date
US7	Test user sorts events by action	Logged in as a user, clicks on the action column header	Events sorted by action	Events sorted by action
US8	Test user sorts events by observations	Logged in as a user, clicks on the observations column header	Events sorted by observations	Events sorted by observations

Id	Description	Inputs	Expected output	Final output
US9	Test user accesses personal info page	Logged in as a user, clicks on the name next to the profile picture	Redirected to personal info page	Redirected to personal info page
US10	Test user updates address and last name	Log in as a user, click on the name next to the profile picture, change the address to "New Address, 123", change the last name to "UpdatedLastName", click "Save changes"	Address and last name updated successfully, changes visible in the user profile	Address and last name updated successfully, changes visible in the user profile
US11	Test user changes password successfully	Log in as a user, click on the name next to the profile picture, click on "Change password", enter "NewPassword123*" in both fields, click "Save"	Password updated successfully, user logged in with new password	Password updated successfully, user logged in with new password

Id	Description	Inputs	Expected output	Final output
US12	Test user fails to change password due to not meeting requirements	Log in as a user, click on the name next to the profile picture, click on "Change password", enter "." as the new password and confirm it, click "Save"	Error message displayed correctly	Error message displayed correctly
US13	Test user updates profile picture	Log in as a user, click on the name next to the profile picture, click on the small pencil icon on the profile picture, select a new picture from the device, click "Open" and then "Save changes"	Profile picture updated successfully, new picture displayed in the user profile	Profile picture updated successfully, new picture displayed in the user profile

Id	Description	Inputs	Expected output	Final output
US14	Test user updates information in the "Others" section	Log in as a user, click on the name next to the profile picture, in the "Others" section, add a new entry under "Allergies" by typing "Peanuts" and clicking "Add to list", click "Save changes"	New allergy added successfully, displayed in the user profile	New allergy added successfully, displayed in the user profile

Table 6.5: Tests: Daycare center user tests

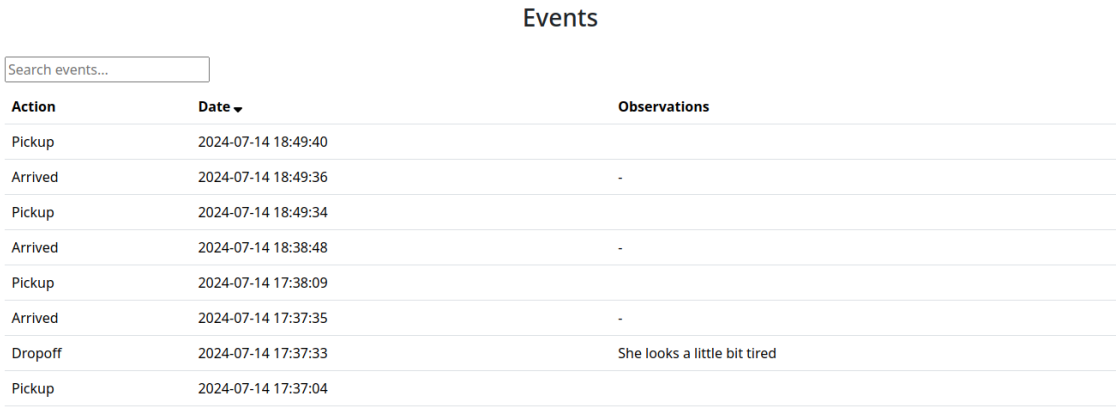

Figure 6.1: Test data for filter tests.

6.1.6.- Logout Tests

Id	Description	Inputs	Expected output	Final output
LO1	Test administrator logout	Log in as administrator, click on "Logout" button	Administrator is logged out and redirected to the login page	Administrator is logged out and redirected to the login page
LO2	Test staff member (non-admin) logout	Log in as staff member (non-admin), click on "Logout" button	Staff member is logged out and redirected to the login page	Staff member is logged out and redirected to the login page
LO3	Test driver logout	Log in as driver, click on "Logout" button	Driver is logged out and redirected to the login page	Driver is logged out and redirected to the login page
LO4	Test user logout	Log in as user, click on "Logout" button	User is logged out and redirected to the login page	User is logged out and redirected to the login page

Table 6.6: Tests: Login tests